

MTH00050 – Combinatorial Mathematics

Lecture 9: Network Flows

Lecturer: Bùi Văn Thạch

Lab instructors: Nguyễn Ngọc Toàn, Trần Thị Thảo Nhi, Lê Đức Khoan

{bvthach,nntoan,tttnhi,ldkhoan}@fit.hcmus.edu.vn

(Tentative) Schedule 2026

No.	Date	Location	Topic
1	13/01 16/01	C24 C24/E302	Introduction, Sets, Proofs
2	20/01 23/01	C24 C24/E302	General counting methods
3	27/01 30/01	C24 C24/E302	Inclusion-exclusion principle
4	03/02 06/02	C24 C24/E302	Recurrence relations
5	03/03 06/03	C24 C24/E302	Generating functions
			Mid-term test

No.	Date	Location	Topic
6	10/03 13/03	C24 C24/E302	Graphs (Part I)
7	17/03 20/03	C24 C24/E302	Graphs (Part II)
8	24/03 27/03	C24 C24/E302	Tree problems
9	31/03 03/04	C24 C24/E302	Path problems
10	07/04 10/04	C24 C24/E302	Network flows (Part I)
11	14/04 17/04	C24 C24/E302	Network flows (Part II)
			Final exam

Course topics

0. Introduction
1. Set and counting
2. General counting methods for arrangements and selections
3. Inclusion-exclusion principle
4. Recurrence relations
5. Generating functions
6. Graphs
7. Tree problems
8. Path problems
9. Network flows

Goals

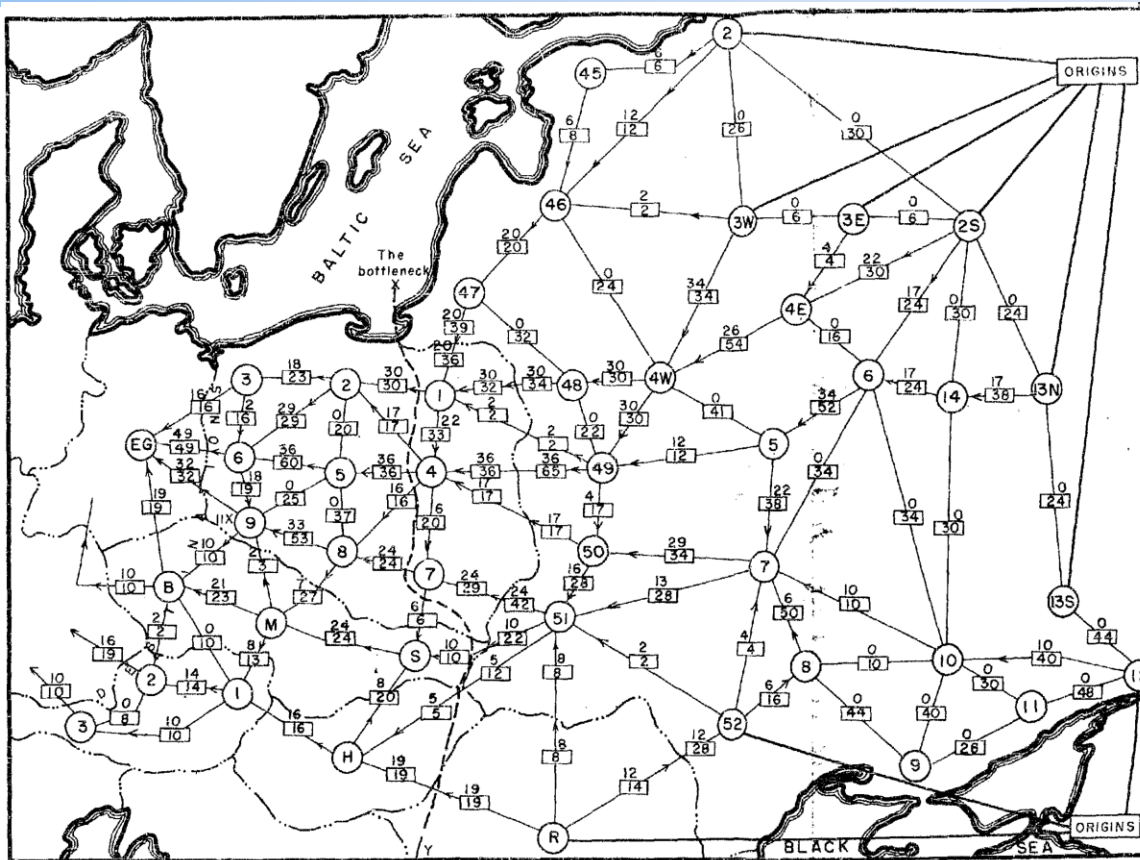
1. Understand the concept of flows in a network
 - Define flow networks, source/sink nodes, and capacity constraints.
 - Analyze feasible flows and conservation of flow at intermediate nodes.
2. Explore the concept of cuts in flow networks
 - Define s-t cuts and their capacities.
 - Examine how cuts help bound the maximum flow in a network.
3. Learn and apply the Max-Flow Min-Cut Theorem
 - Understand the statement and intuition behind the theorem.
- Connect maximum flow values with the capacity of minimum cuts.
4. Understand the Ford-Fulkerson algorithm
 - Learn how augmenting paths are used to increase flow.
 - Analyze when the algorithm terminates and the role of integral capacities.
5. Study the Edmonds-Karp algorithm
 - See how BFS improves Ford-Fulkerson by guaranteeing polynomial time.
 - Analyze the time complexity and performance in practice.

Outline

1. Flows
2. Cuts
3. The Maxflow-Mincut Theorem
4. Ford and Fulkerson's algorithm
5. Edmonds and Karp's Algorithms

Problem

- In the mid-1950s, U. S. Air Force researcher Theodore E. Harris and retired U. S. Army general Frank S. Ross wrote a classified report studying the rail network that linked the Soviet Union to its satellite countries in Eastern Europe.
- The network was modeled as a graph with 44 vertices, representing geographic regions, and 105 edges, representing links between those regions in the rail network.
- Each edge was given a weight, representing the rate at which material could be shipped from one region to the next. The time
- Essentially by trial and error, they determined both the maximum amount of stuff that could be moved from Russia into Europe, as well as the cheapest way to disrupt the network by removing links (or in less abstract terms, blowing up train tracks), which they called "the bottleneck".
- Their report, which included the drawing of the network in the figure beside, was only declassified in 1999.



SECRET RM-1573 10-24-55 -55-

Fig. 7 — Traffic pattern: entire network available

Legend:

- International boundary
- ⊙ Railway operating division
- ← [12] → Capacity: 12 each way per day. Required flow of 9 per day toward destinations (in direction of arrow) with equivalent number of returning trains in opposite direction

All capacities in trains / 1000's of tons each way per day

Origins: Divisions 2, 3W, 3E, 2S, 13N, 13S, 12, 52 (USSR), and Roumania

Destinations: Divisions 3, 6, 9 (Poland); B (Czechoslovakia); and 2, 3 (Austria)

Alternative destinations: Germany or East Germany

Note: IIX of Division 9, Poland

Harris and Ross's map of the Warsaw Pact rail network.

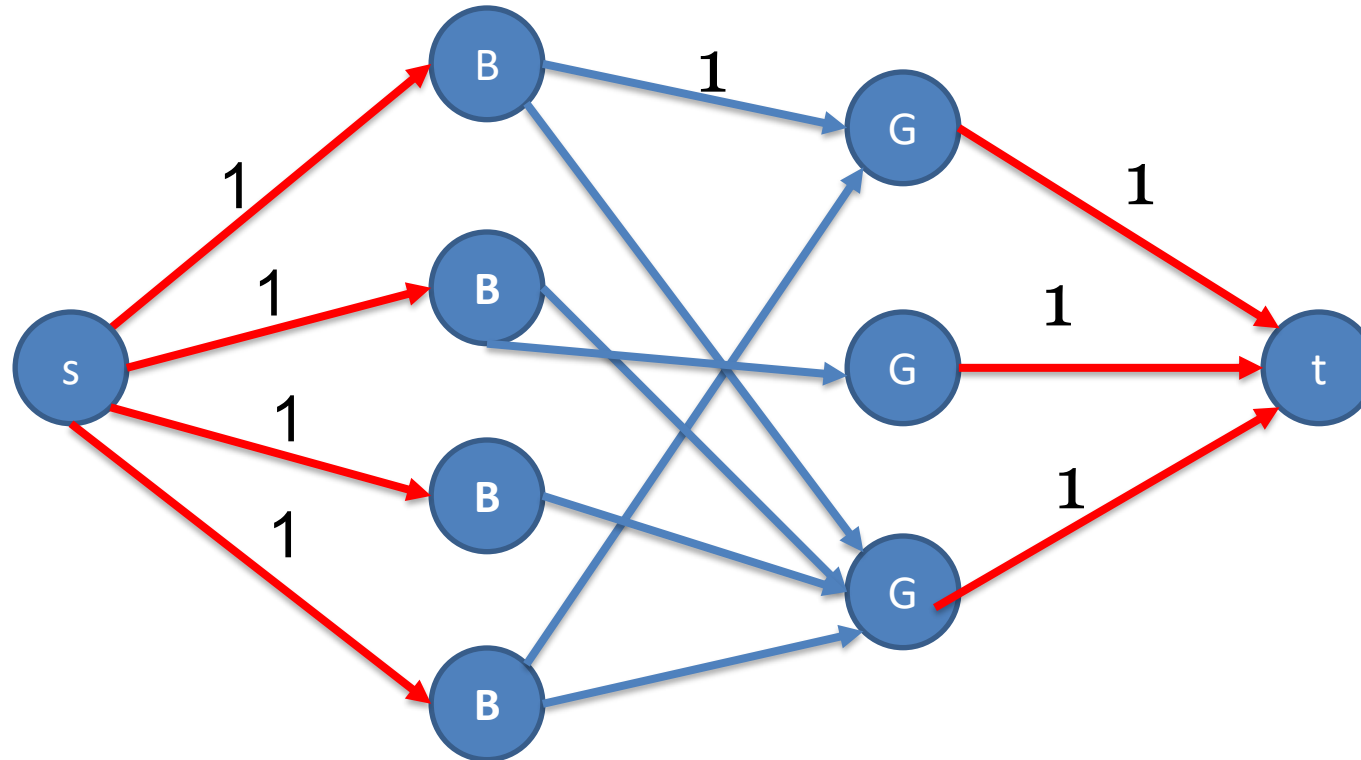
Harris, Theodore E., and F. S. Ross. *Fundamentals of a method for evaluating rail net capacities*. No. RM1573. 1955.

Oil distribution

- Consider the graph corresponding to an oil pipeline system. In this system, the pipes correspond to the edges, the source point is the oil tanker, the sink point is the storage tank, and the pipe junctions are the nodes of the graph.
- The capacity of each edge corresponds to the cross-section of each pipe.
- The task is to determine the maximum amount of oil that can be pumped from the tanker to the storage tank.

Matching problem

- Matching problem: there are m boys and n girls. Each boy likes a certain number of girls. Find a way to match pairs so that the number of matched pairs is maximized.



Outline

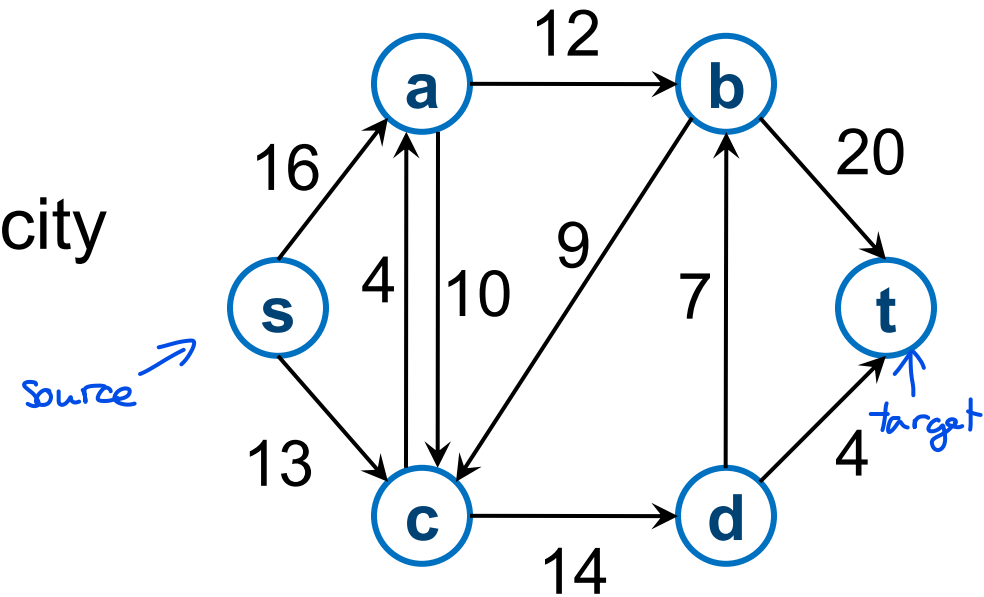
1. **Flows**
2. Cuts
3. The Maxflow-Mincut Theorem
4. Ford and Fulkerson's algorithm
5. Edmonds and Karp's Algorithms

Network *Mạng*

- A network is a directed graph $G = (V, E)$
- Edges represent pipes that carry flow
- Each edge $e = (u, v)$ has maximum capacity

$$c_e = c_{(u,v)}$$

- A source node s in which flow leaves
- A target node t out which flow arrives



Flows

total of input lines = total of output lines

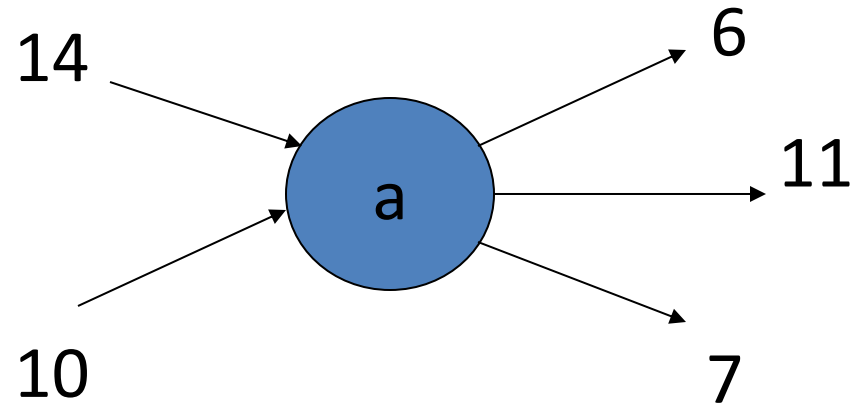
Definition 1. An (s, t) -flow is a function $f: E \rightarrow \mathbb{R}$ that satisfies the following **conservation constraint** at every vertex v except possibly s and t :

$$\sum_{u \in V} f(u \rightarrow v) = \sum_{w \in V} f(v \rightarrow w)$$

If there is no edge $u \rightarrow v$, then $f(u \rightarrow v) = 0$.

Flows

Example:



$$\sum_{u \in V} f(u \rightarrow a) = 14 + 10 = \sum_{v \in V} f(a \rightarrow v) = 6 + 11 + 7$$

Flows

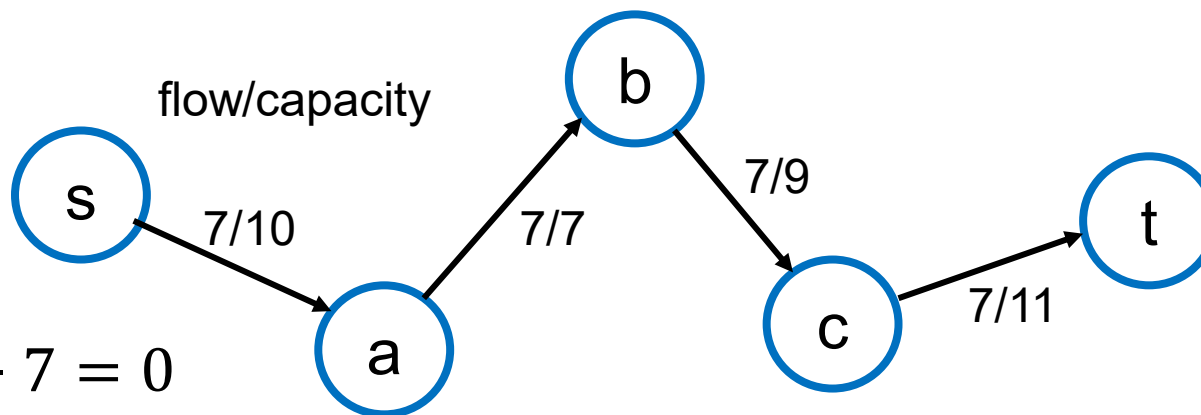
(s, t) -

Definition 2. The value of the flow f , denoted $|f|$, is the total net flow **out of** the source vertex s :

$$|f| := \sum_{w \in V} f(s \rightarrow w) - \sum_{u \in V} f(u \rightarrow s)$$

$$|s| = 7 - 0 = 7$$

$$|a| = |b| = |c| = 7 - 7 = 0$$



$$|t| = 0 - 7 = -7$$

Flows

Proposition 1. The value of the flow f is equal to the total net flow *into the* target vertex t .

You transfer n , you receive n

Proof. Let $\partial f(v)$ denote the total net flow out of any vertex v in the graph, we have:

$$\partial f(v) := \sum_{u \in V} f(u \rightarrow v) - \sum_{w \in V} f(v \rightarrow w)$$

The conservation constraint implies that $\partial f(v) = 0$ for every vertex v except s and t .

Flows

Proposition 1. The value of the flow f is equal to the total net flow *into the* target vertex t .

Proof (cont). So that

$$\sum_{v \in V} \partial f(v) = \partial f(s) + \partial f(t).$$

On the other hand, any flow that leaves one vertex must enter another vertex, so we must have $\sum_{v \in V} \partial f(v) = 0$. It follow that $|f| = \partial f(s) = -\partial f(t)$.

Flows

Definition 3. A flow f is *feasible* if

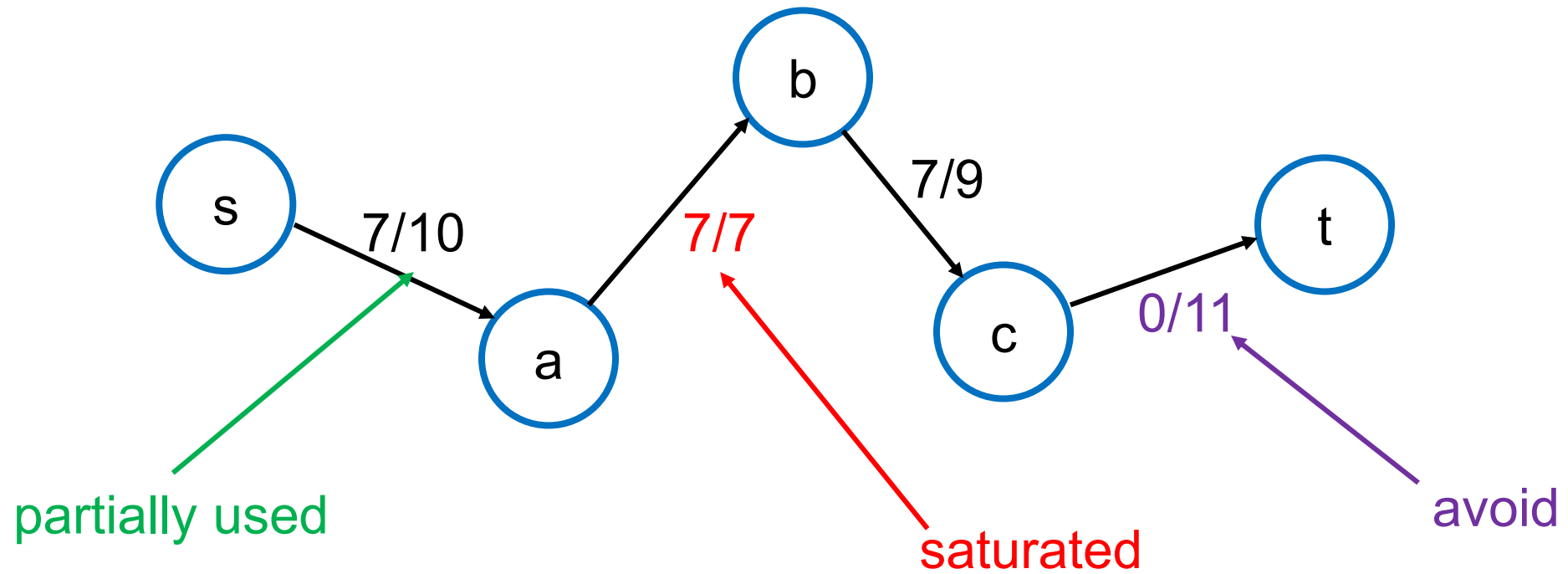
$$0 \leq f(e) \leq c(e), \forall e \in E$$

where a function $c: E \rightarrow \mathbb{R}_{\geq 0}$ that assigns a non-negative capacity $c(e)$ for every edge e .

We say that a flow ***saturates*** edge e if $f(e) = c(e)$, and ***avoids*** edge e if $f(e) = 0$.

Flows

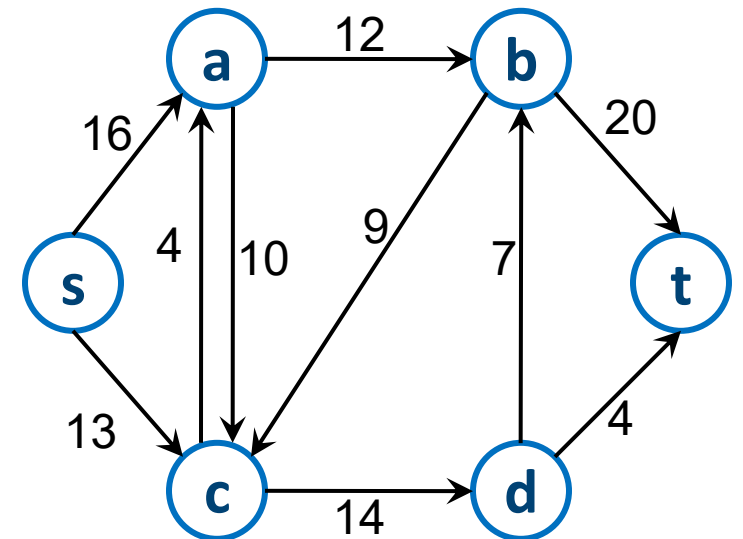
Example:



Maximum flow problem

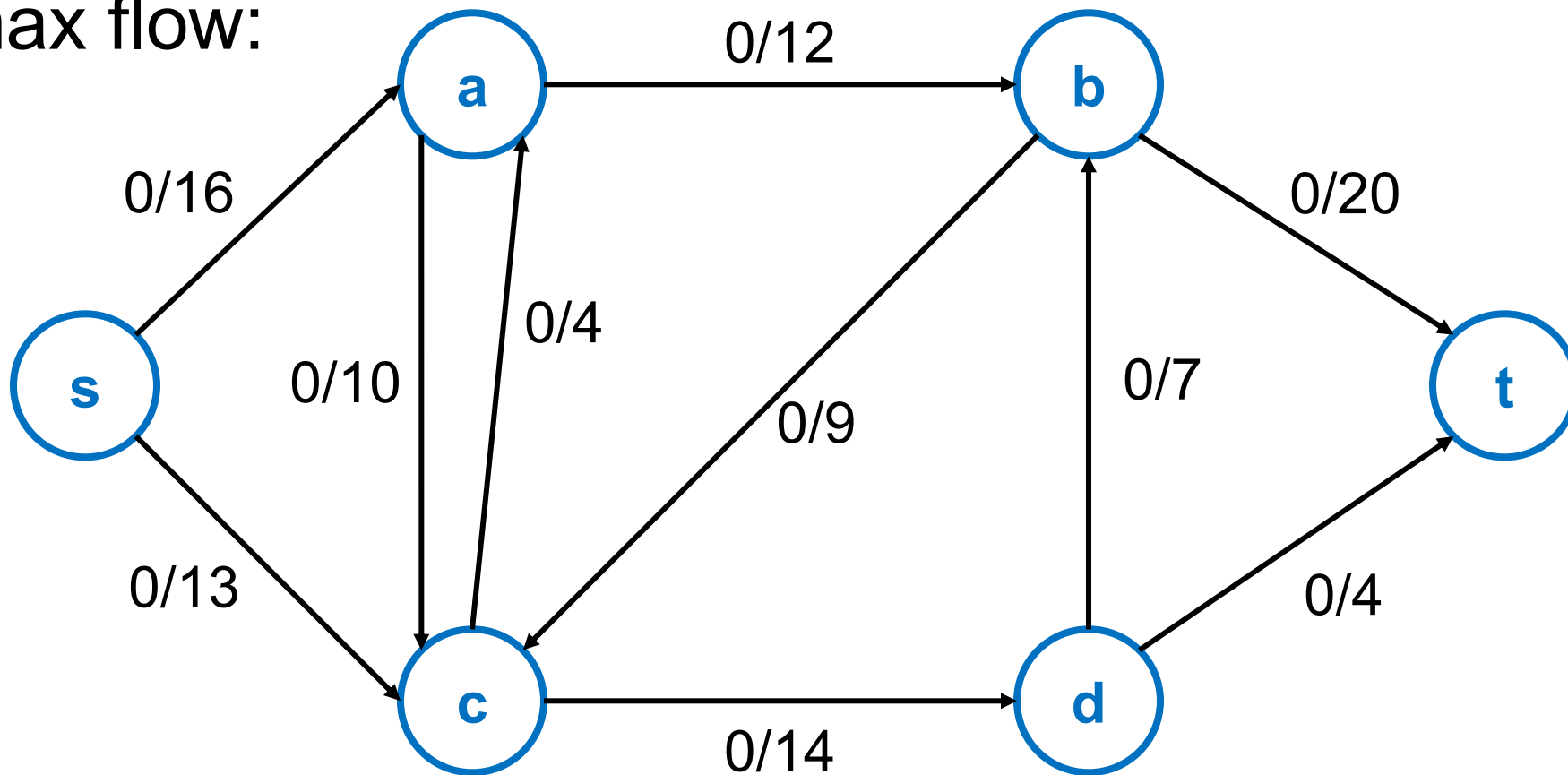
Find a possible flow that has the maximum capacities

- Given directed graph $G = (V, E)$,
- Two different vertices, source node s and target node t ,
- given capacity function c_e
- The maximum flow problem is to compute *a feasible (s, t) -flow* whose value is *maximum*.



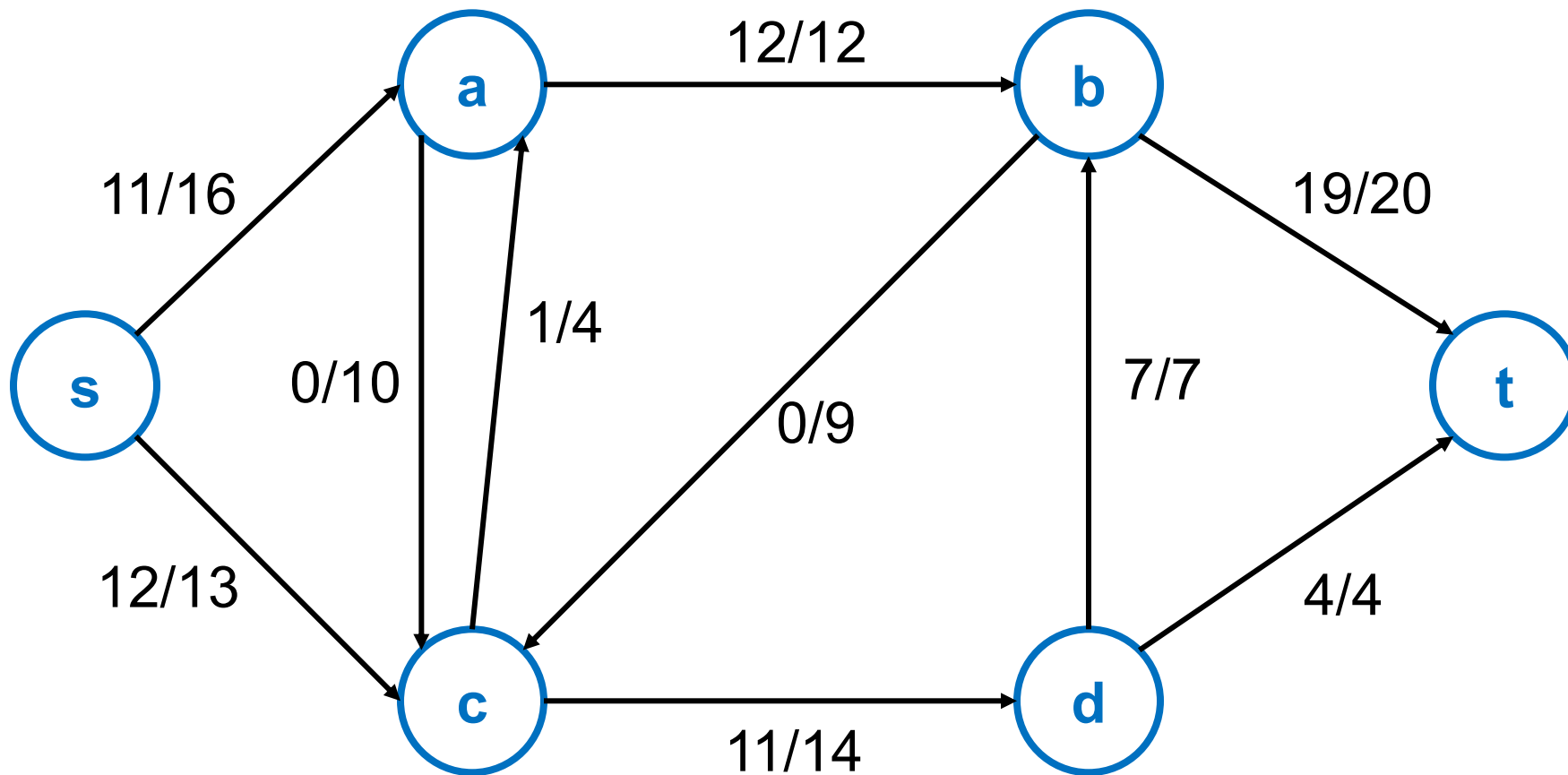
Maximum flow problem

Find max flow:



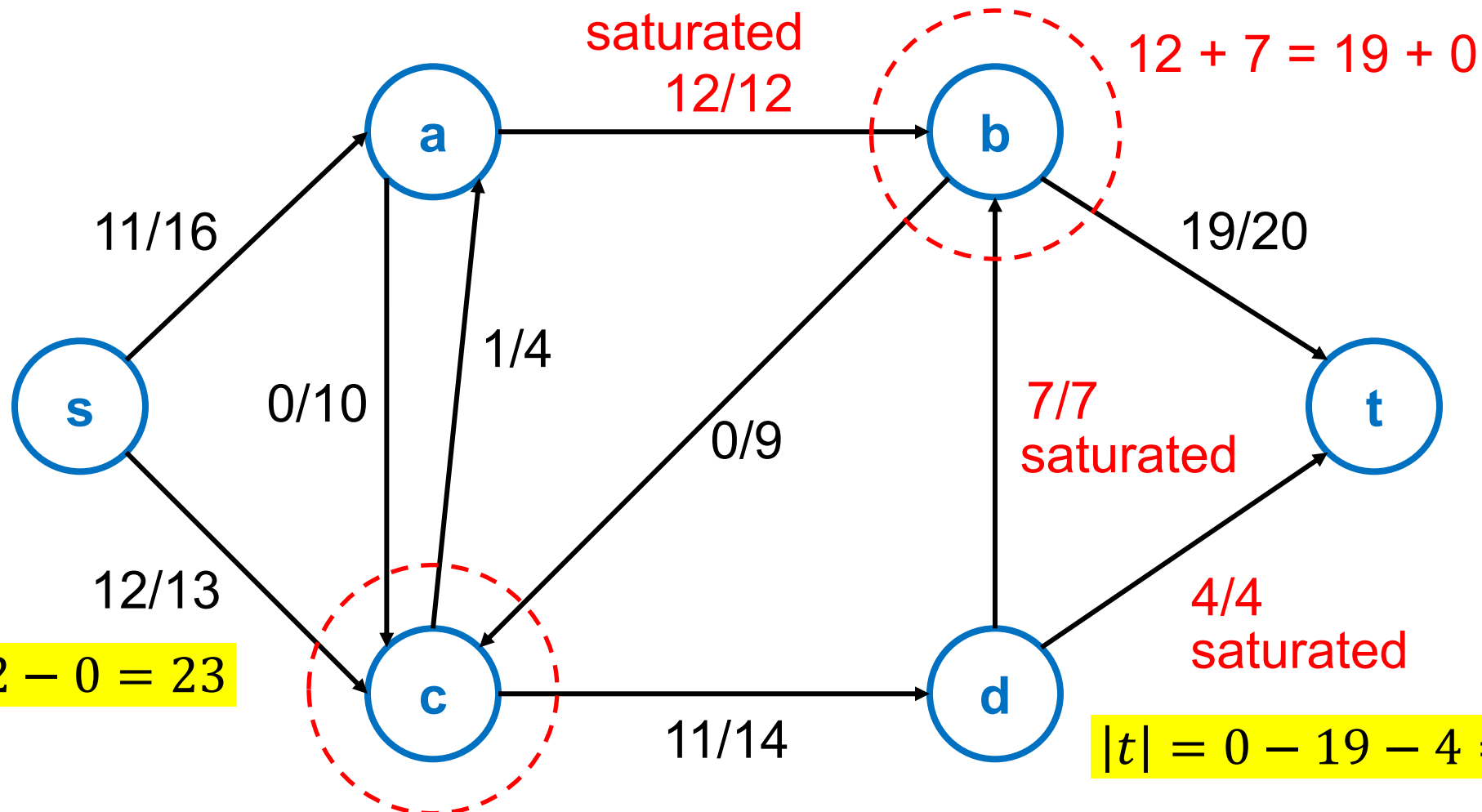
Maximum flow problem

A flow:



Maximum flow problem

A flow:



Outline

1. Flows
2. Cuts
3. The Maxflow-Mincut Theorem
4. Ford and Fulkerson's Algorithm
5. Edmonds and Karp's Algorithms

Cuts

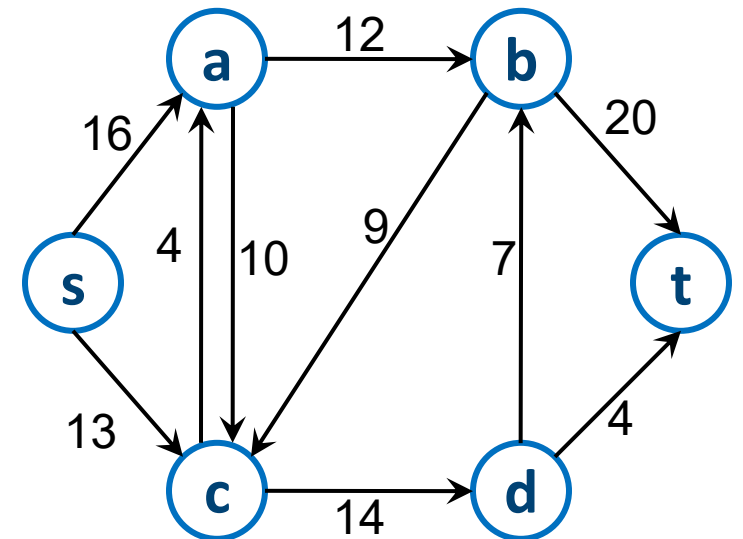
Definition 4. An (s, t) -cut is a partition of the vertices into disjoint subsets S and T – meaning $S \cup T = V$ and $S \cap T = \emptyset$ - **where $s \in S$ and $t \in T$** . The capacity of a cut is the sum of the capacities of the edges that start in S and end in T (***asymmetric: $S \rightarrow T$***):

$$\|S, T\| := \sum_{v \in S} \sum_{w \in T} c(v \rightarrow w).$$

If $v \rightarrow w$ is not an edge, then $c(v \rightarrow w) = 0$.

Minimum cut problem

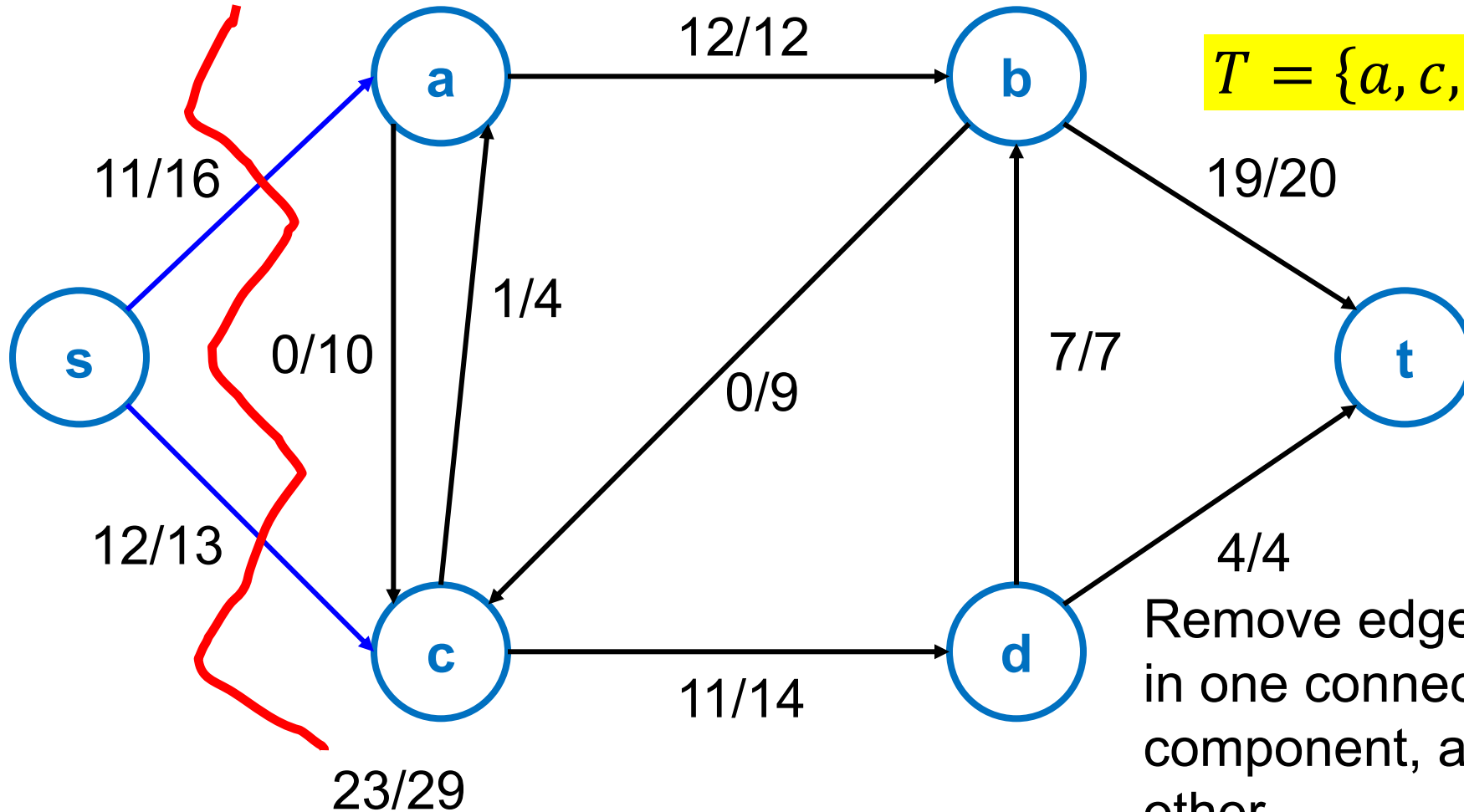
- Given directed graph $G = (V, E)$,
- Two different vertices, source node s and target node t ,
- given capacity function c_e
- The minimum cut problem is to compute *an* (s, t) -cut whose capacity is *minimum*.



Minimum cut problem

$$\|S, T\| := c(s \rightarrow a) + c(s \rightarrow c) = 16 + 13 = 29$$

A cut:



$$S = \{s\}$$

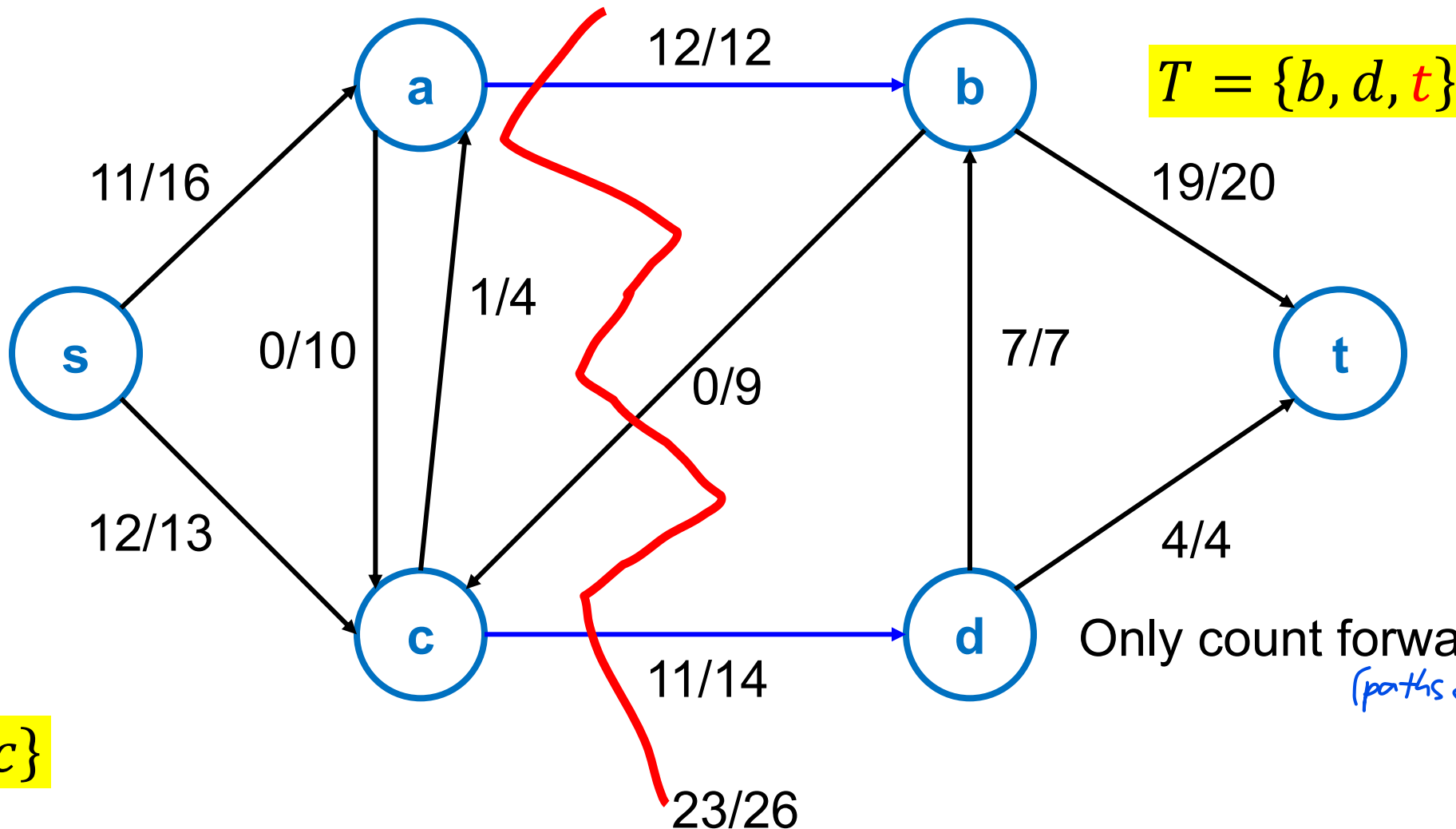
$$T = \{a, c, b, d, t\}$$

Remove edges until S is in one connected component, and T in the other

Minimum cut problem

$$\|S, T\| := c(a \rightarrow b) + c(c \rightarrow d) = 12 + 14 = 26$$

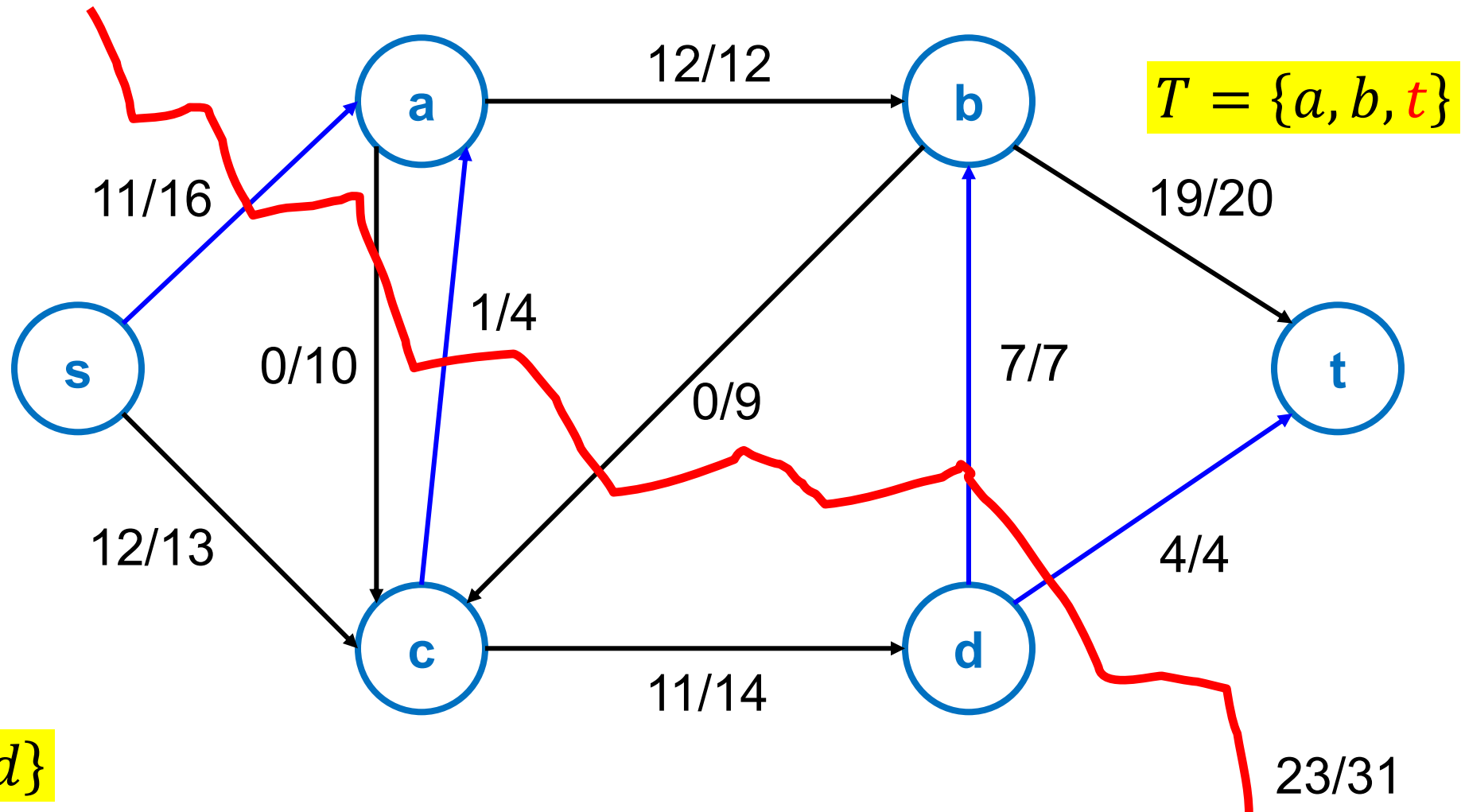
A cut:



Minimum cut problem

$$\|S, T\| := c(s \rightarrow a) + c(c \rightarrow a) + c(d \rightarrow b) + c(d \rightarrow t) = 16 + 4 + 7 + 4 = 31$$

A cut:



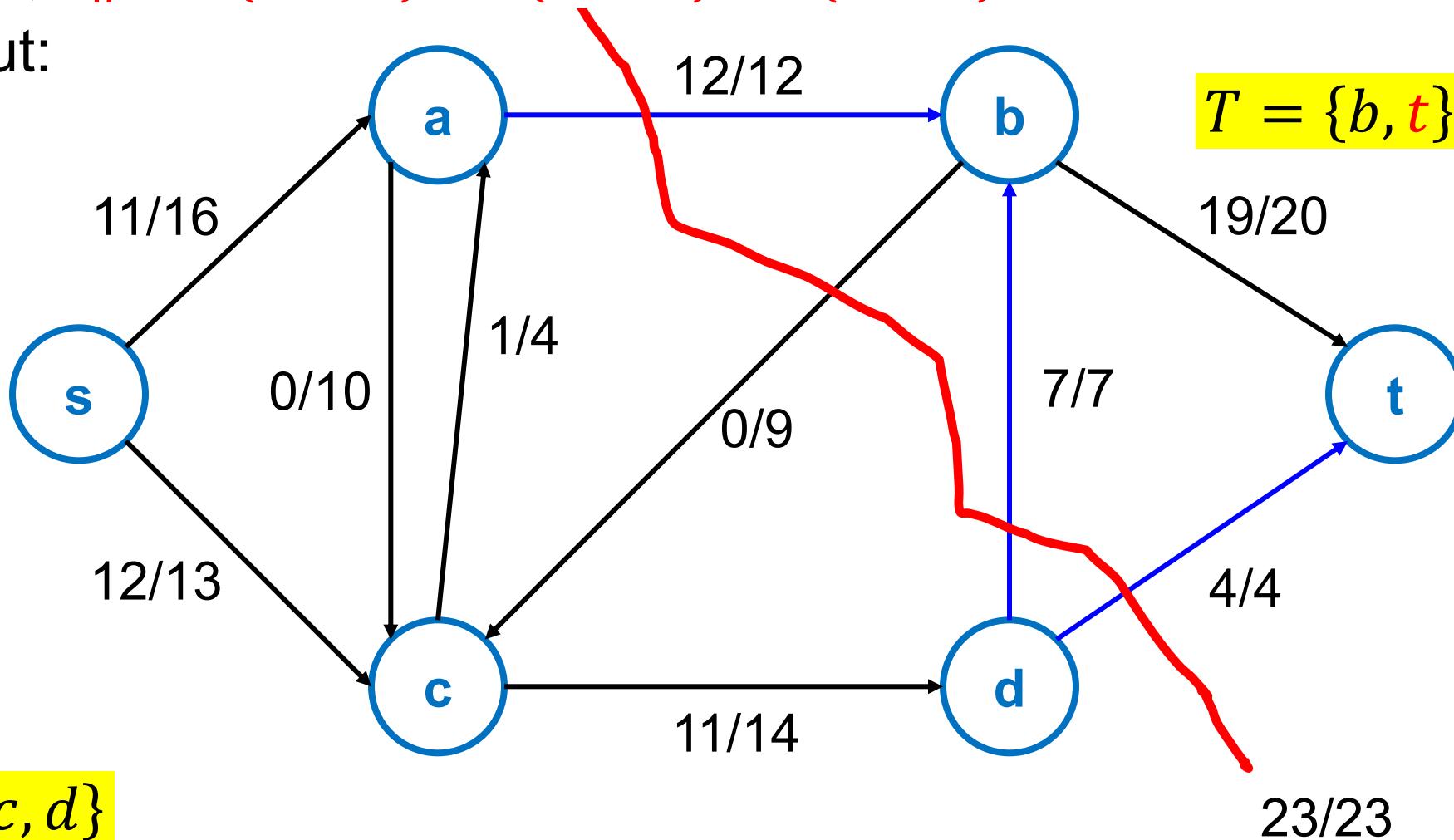
$$S = \{s, c, d\}$$

$$T = \{a, b, t\}$$

Minimum cut problem

$$\|S, T\| := c(a \rightarrow b) + c(d \rightarrow b) + c(d \rightarrow t) = 12 + 7 + 4 = 23$$

A min cut:



$$S = \{s, a, c, d\}$$

$$T = \{b, t\}$$

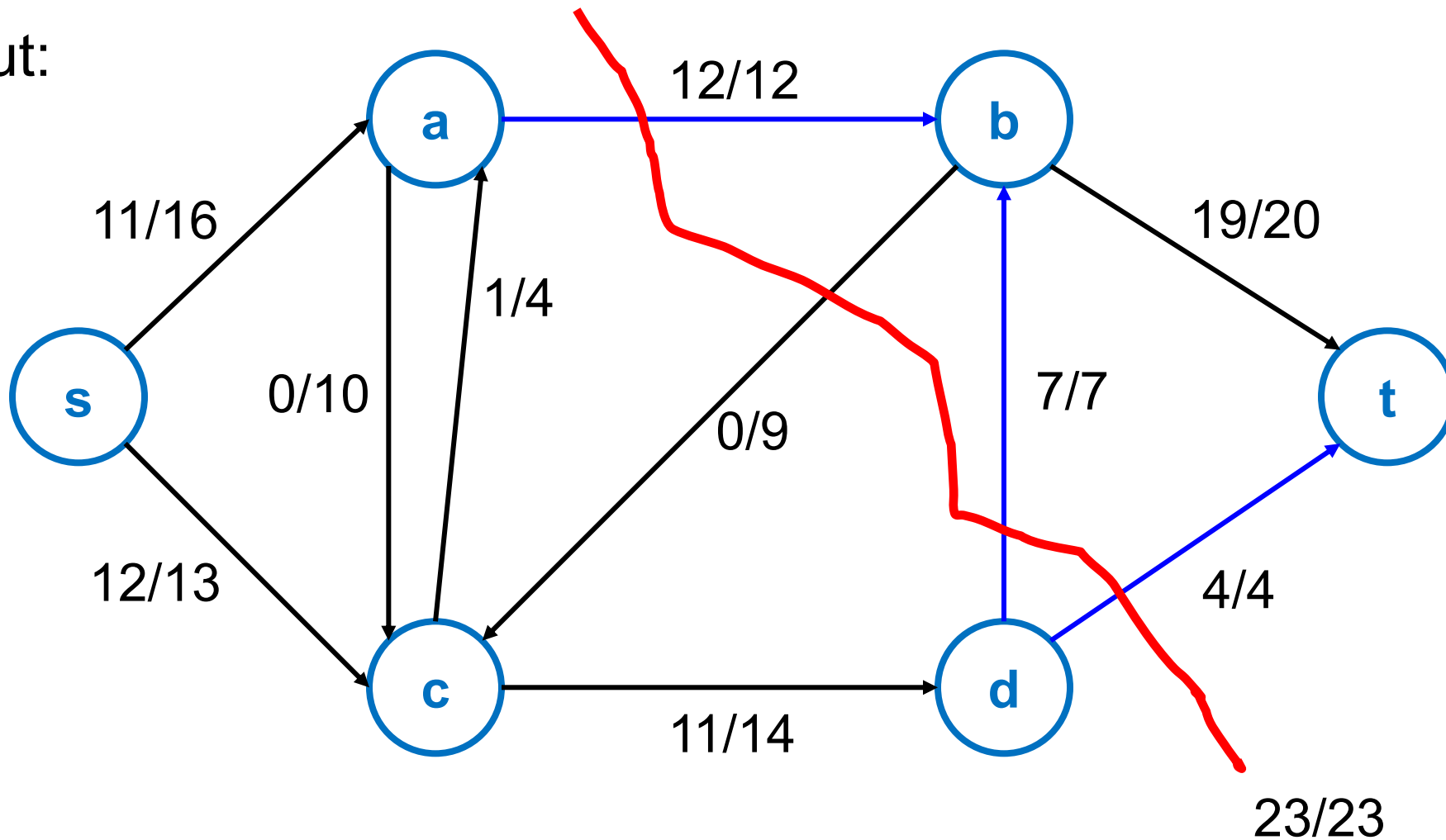
The relationship between flows and cuts

Lemma 1. Let f be any feasible (s, t) -flow, and let (S, T) be any (s, t) -cut. The value of f is at most the capacity of (S, T) . Moreover, $|f| = ||S, T||$ iff f saturates every edge from S to T and avoids every edge from T to S .

In other word, if $|f| = ||S, T||$ then f must be a maximum flow and (S, T) must be a minimum cut.

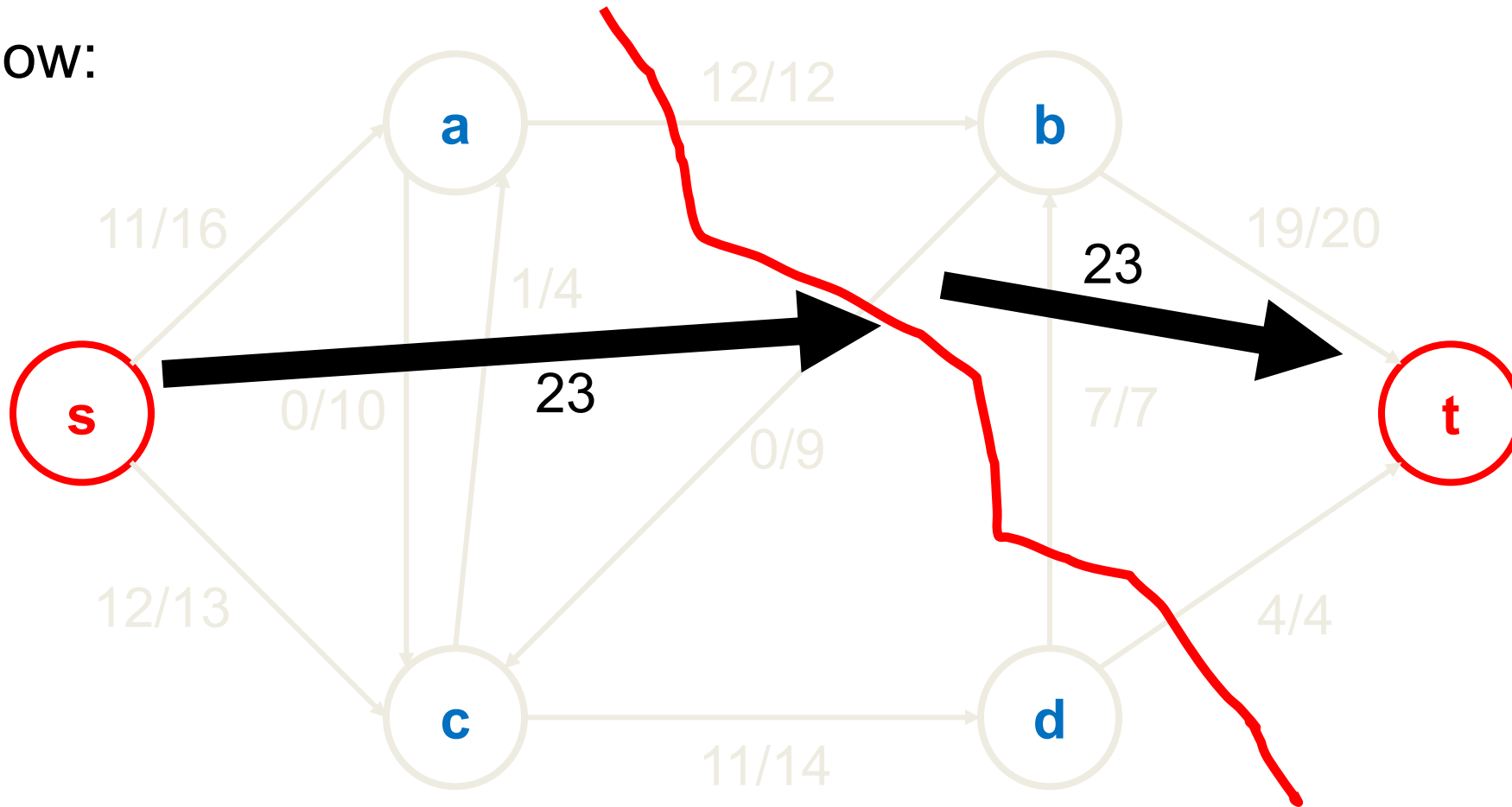
The relationship between flows and cuts

A min cut:



The relationship between flows and cuts

A max flow:



No more capacity along cut to take more flow

The relationship between flows and cuts

Proof. Choose any flow f and cut (S, T) , and then following the bounding inequalities:

$$|f| = \partial f(s) \quad (\text{by definition})$$

$$= \sum_{v \in S} \partial f(v) \partial f(v) := -(\sum_{w \in V} f(v \rightarrow w) - \sum_{w \in V} f(u \rightarrow v)) \quad (\text{conservation constraint})$$

$$= \sum_{v \in S} \sum_{w \in V} f(v \rightarrow w) - \sum_{v \in S} \sum_{u \in V} f(u \rightarrow v) \quad (\text{math, definition of } \partial)$$

$$= \sum_{v \in S} \sum_{w \notin S} f(v \rightarrow w) - \sum_{v \in S} \sum_{u \notin S} f(u \rightarrow v) \quad (\text{removing edges from } S \text{ to } S)$$

The relationship between flows and cuts

Proof. Choose any flow f and cut (S, T) , and then following the bounding inequalities:

$$= \sum_{v \in S} \sum_{w \in T} f(v \rightarrow w) - \sum_{v \in S} \sum_{u \in T} f(u \rightarrow v) \quad (\text{definition of cut})$$

$$\leq \sum_{v \in S} \sum_{w \in T} f(v \rightarrow w) \quad (\text{because } f(u \rightarrow v) \geq 0)$$

$$\leq \sum_{v \in S} \sum_{w \in T} c(v \rightarrow w) \quad (\text{because } f(v \rightarrow w) \leq c(v \rightarrow w))$$

$$\leq \|S, T\| \quad (\text{by definition})$$

Outline

1. Flows
2. Cuts
3. The Maxflow-Mincut Theorem
4. Ford and Fulkerson's Algorithm
5. Edmonds and Karp's Algorithms

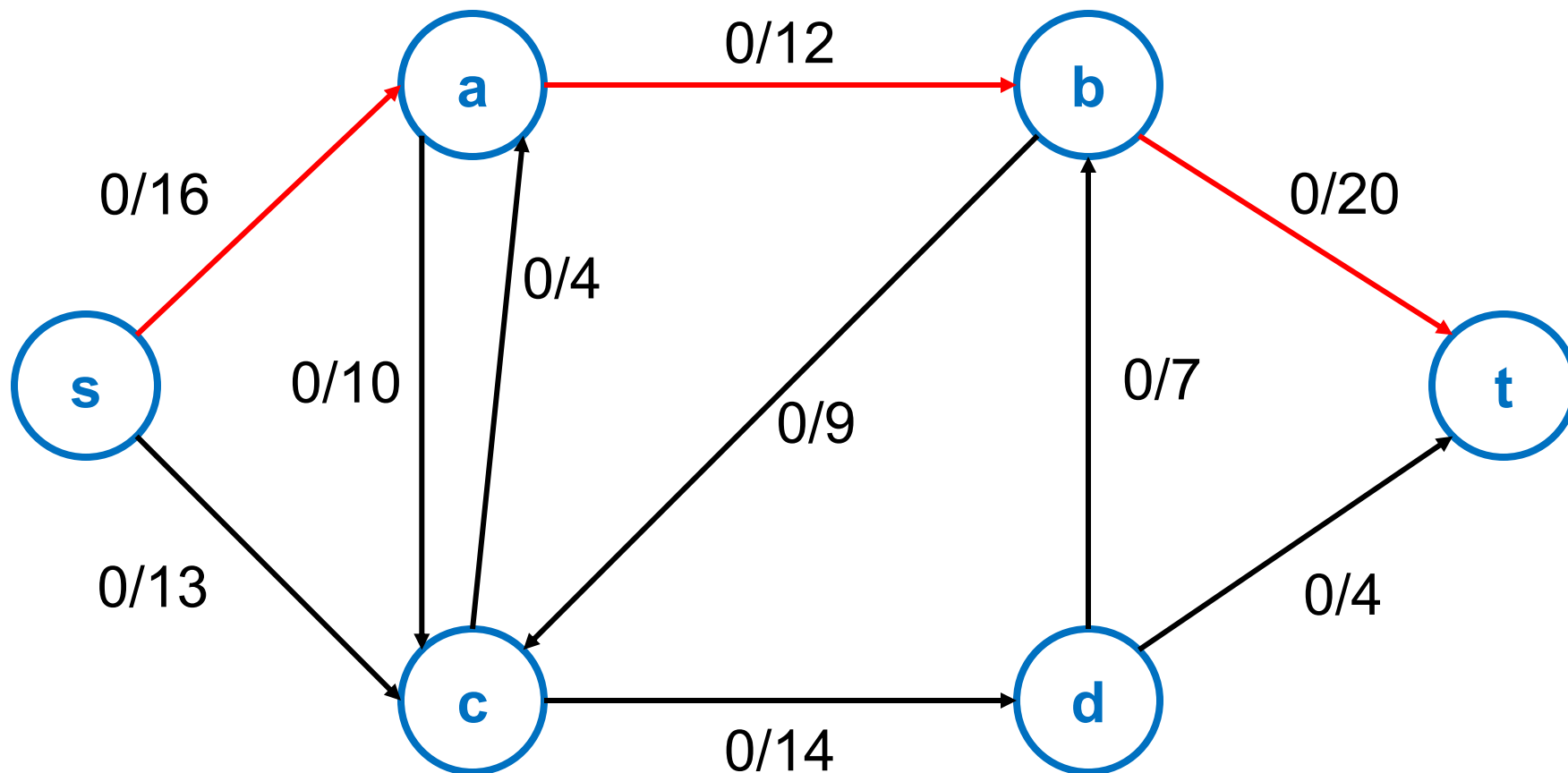
Maxflow-Mincut Theorem

Theorem 1. In every flow network with source s and target t , the value of maximum (s, t) -flow is equal to the capacity of the minimum (s, t) -cut.

Before prove this theorem, we will consider some examples and construct some useful definitions.

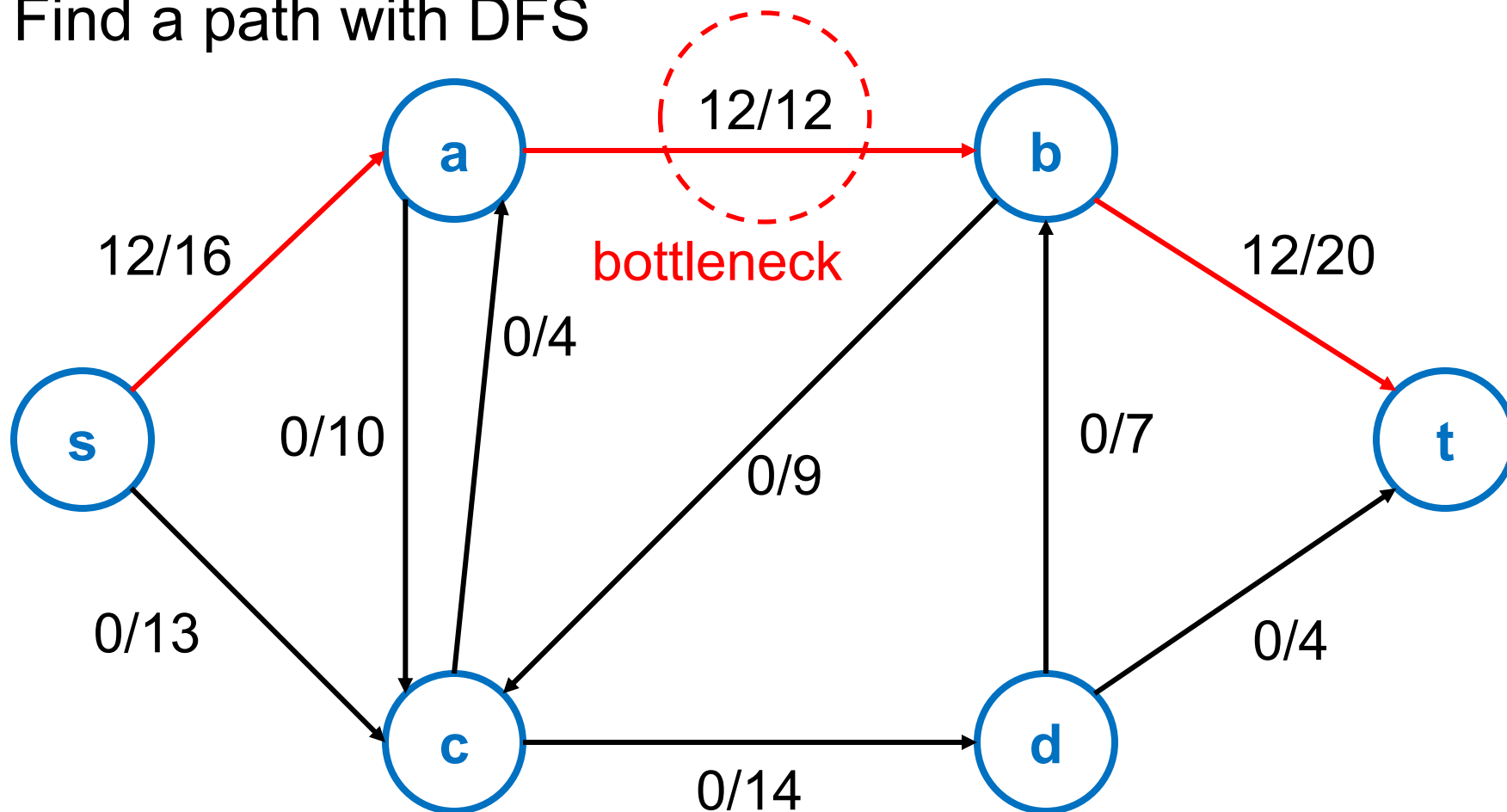
Residual capacity

Question: How much can be sent along the path?



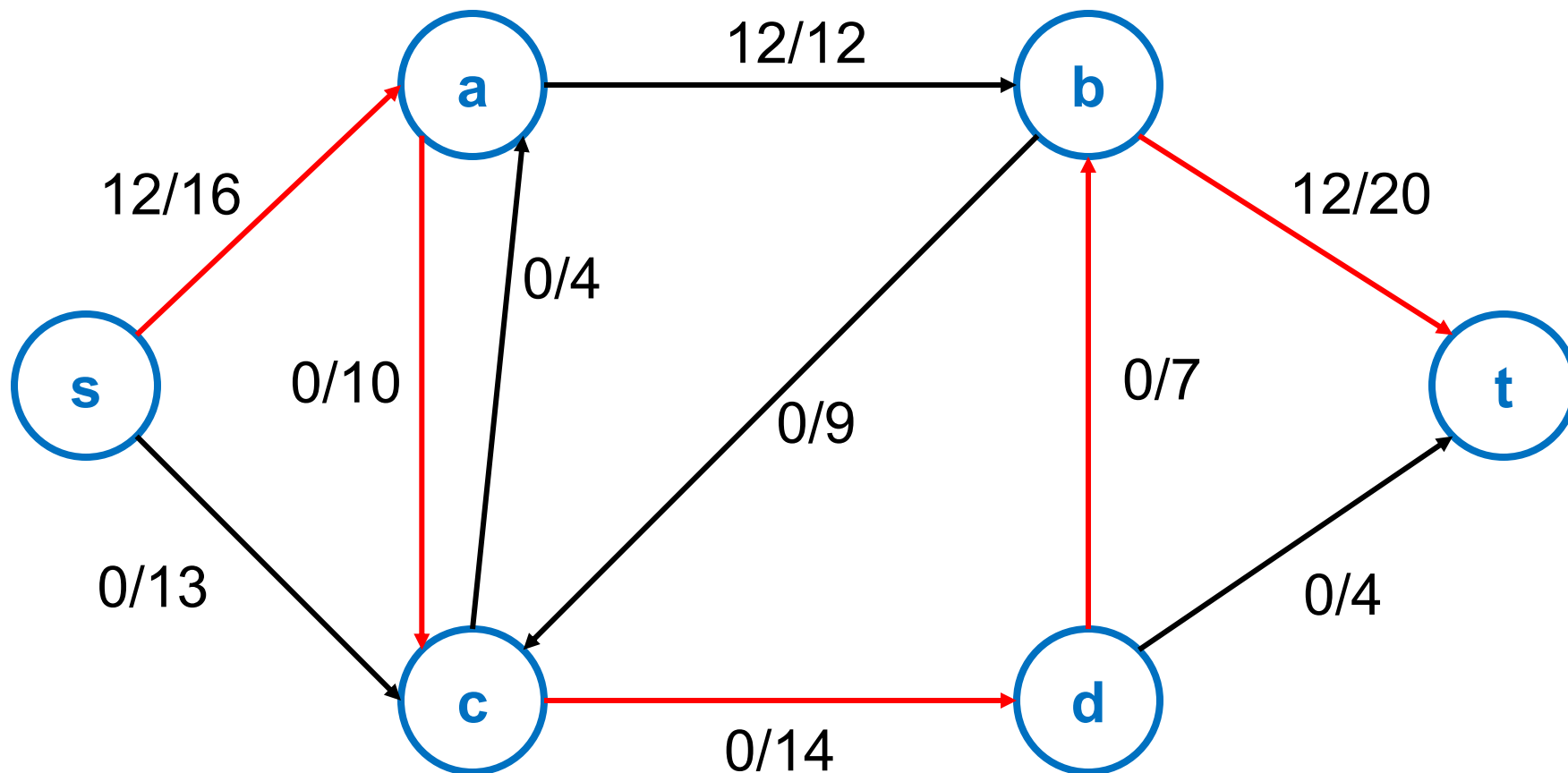
Residual capacity

Solution: Find a path with DFS or BFS.



Residual capacity

Question: How much can be sent along the path?



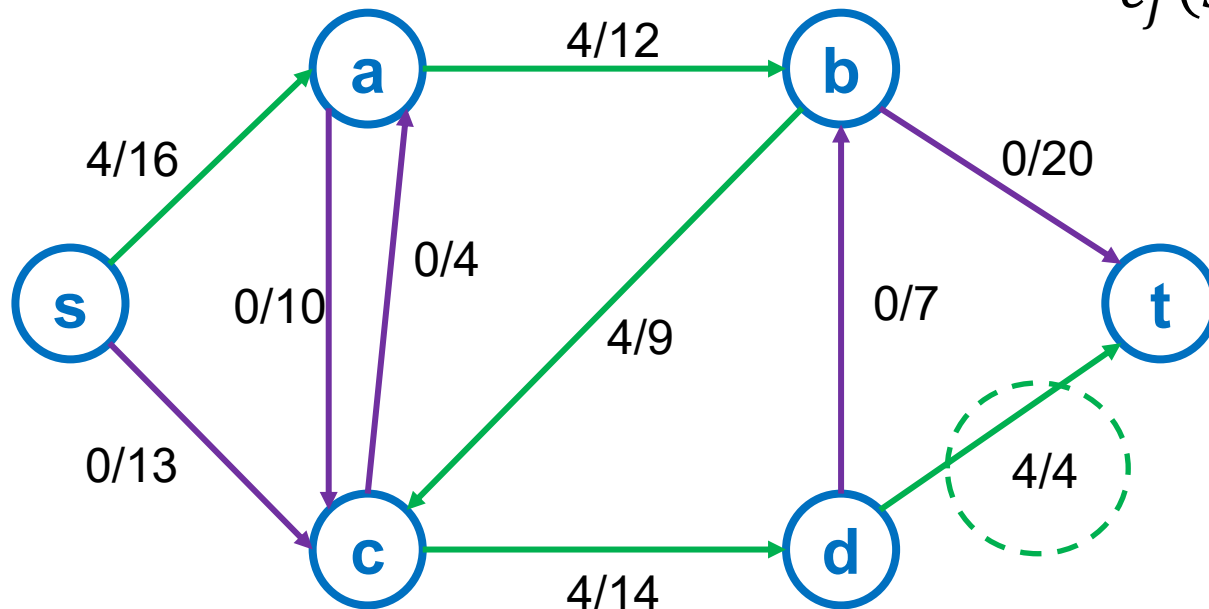
Residual capacity

Definition 5. Let f be an arbitrary feasible (s, t) -flow in G . A function $c_f: V \times V \rightarrow \mathbb{R}$ called the **residual capacity**, as follows:

$$c_f(u \rightarrow v) = \begin{cases} c(u \rightarrow v) - f(u \rightarrow v) & \text{if } u \rightarrow v \in E \\ f(v \rightarrow u) & \text{if } v \rightarrow u \in E \\ 0 & \text{otherwise} \end{cases}$$

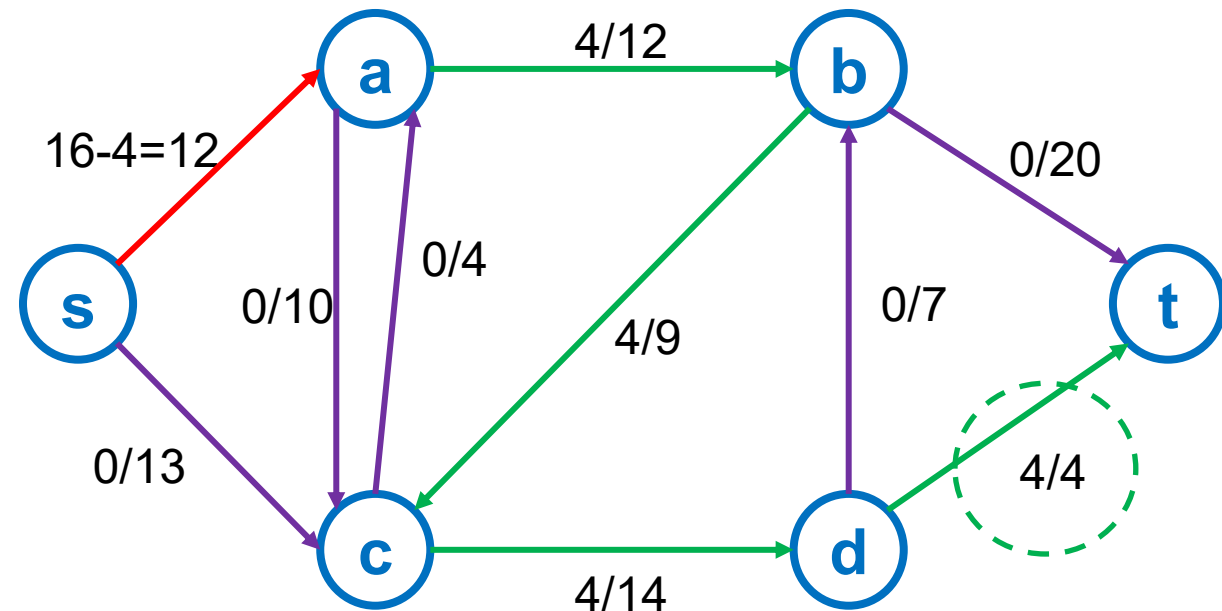
Residual capacity (1/18)

Example:



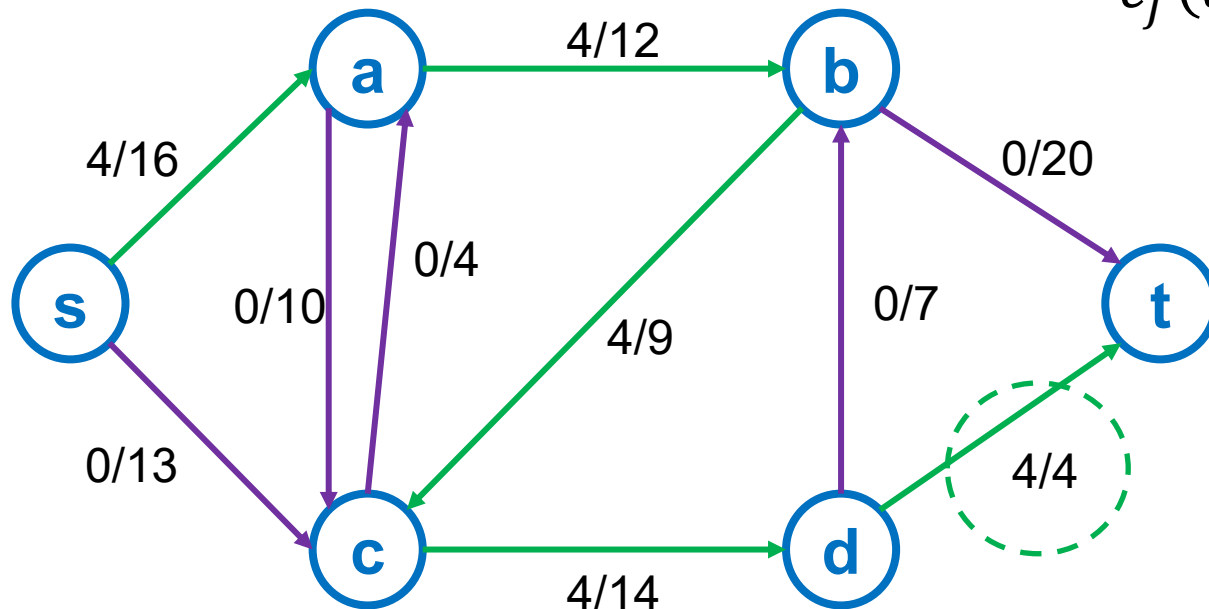
$$c_f(s \rightarrow a) = \begin{cases} c(s \rightarrow a) - f(s \rightarrow a) & \text{if } s \rightarrow a \in E \\ f(a \rightarrow s) & \text{if } a \rightarrow s \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(s \rightarrow a) = c(s \rightarrow a) - f(s \rightarrow a) = 16 - 4 = 12$$



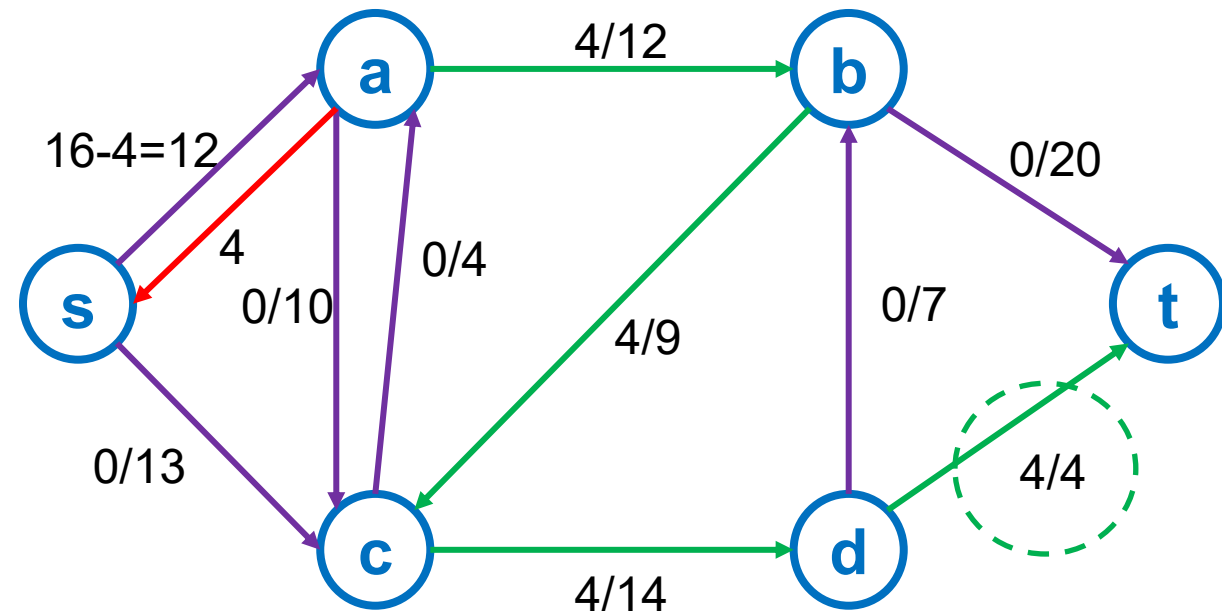
Residual capacity (2/18)

Example:



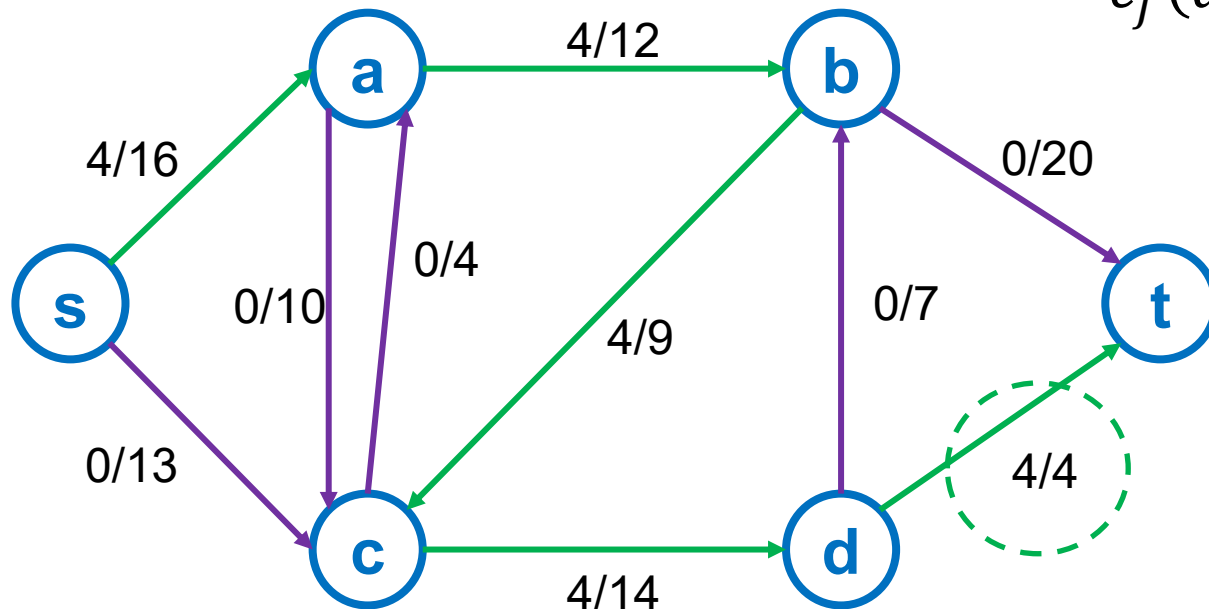
$$c_f(a \rightarrow s) = \begin{cases} c(a \rightarrow s) - f(a \rightarrow s) & \text{if } a \rightarrow s \in E \\ f(s \rightarrow a) & \text{if } s \rightarrow a \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(a \rightarrow s) = f(s \rightarrow a) = 4$$



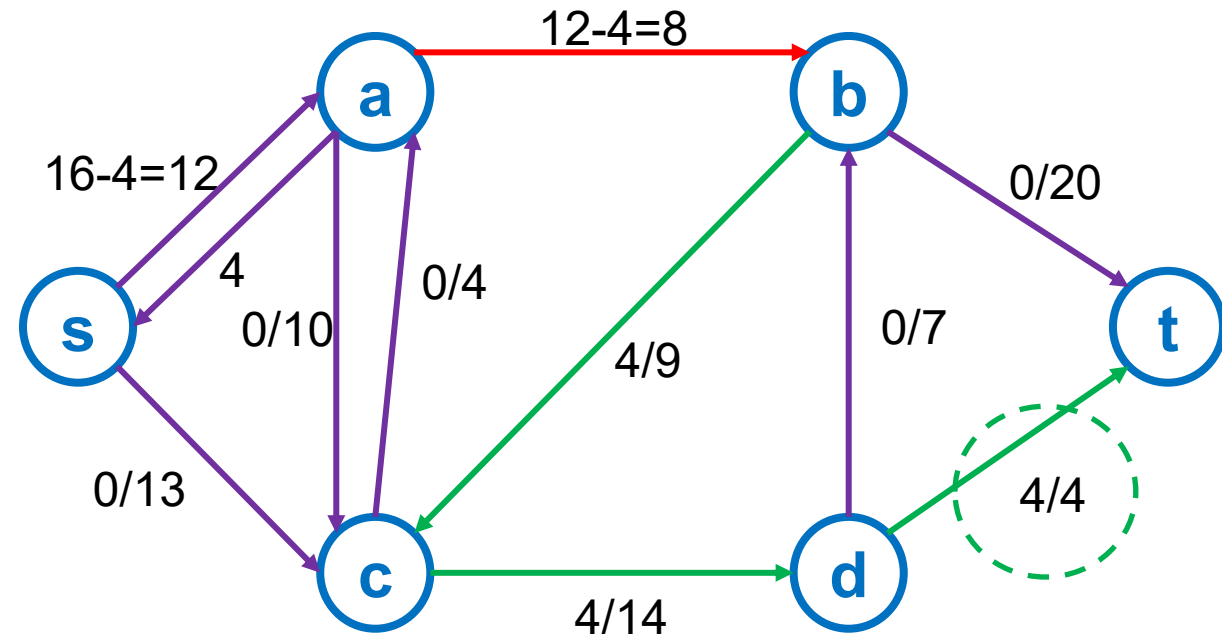
Residual capacity (3/18)

Example:



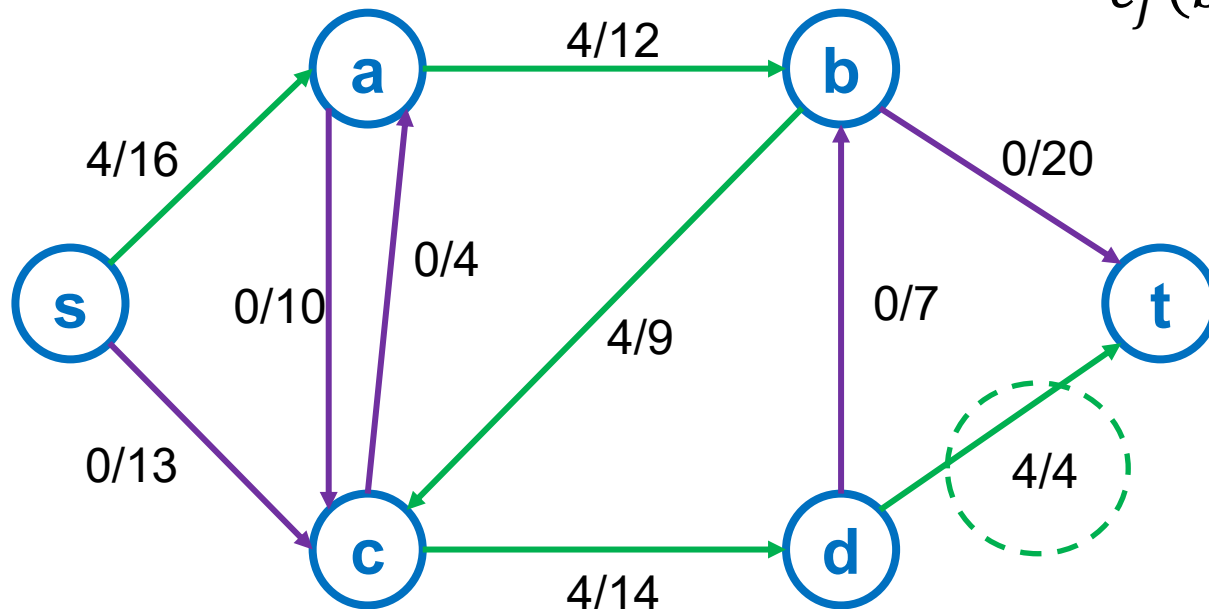
$$c_f(a \rightarrow b) = \begin{cases} c(a \rightarrow b) - f(a \rightarrow b) & \text{if } a \rightarrow b \in E \\ f(b \rightarrow a) & \text{if } b \rightarrow a \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(a \rightarrow b) = c(a \rightarrow b) - f(a \rightarrow b) = 12 - 4 = 8$$



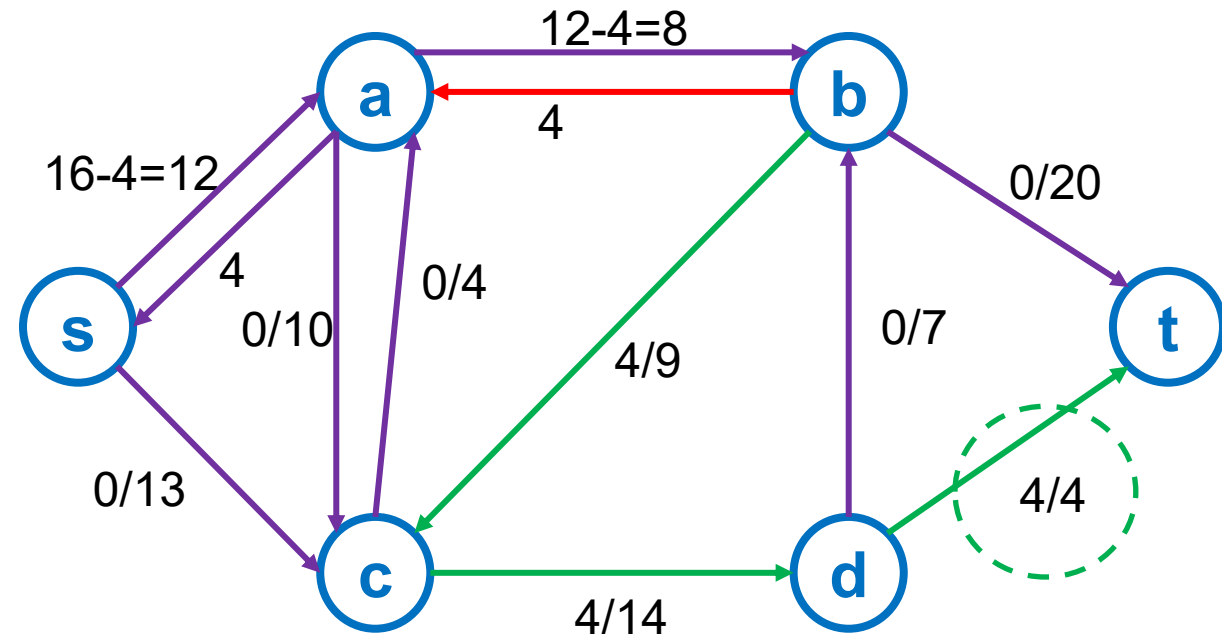
Residual capacity (4/18)

Example:



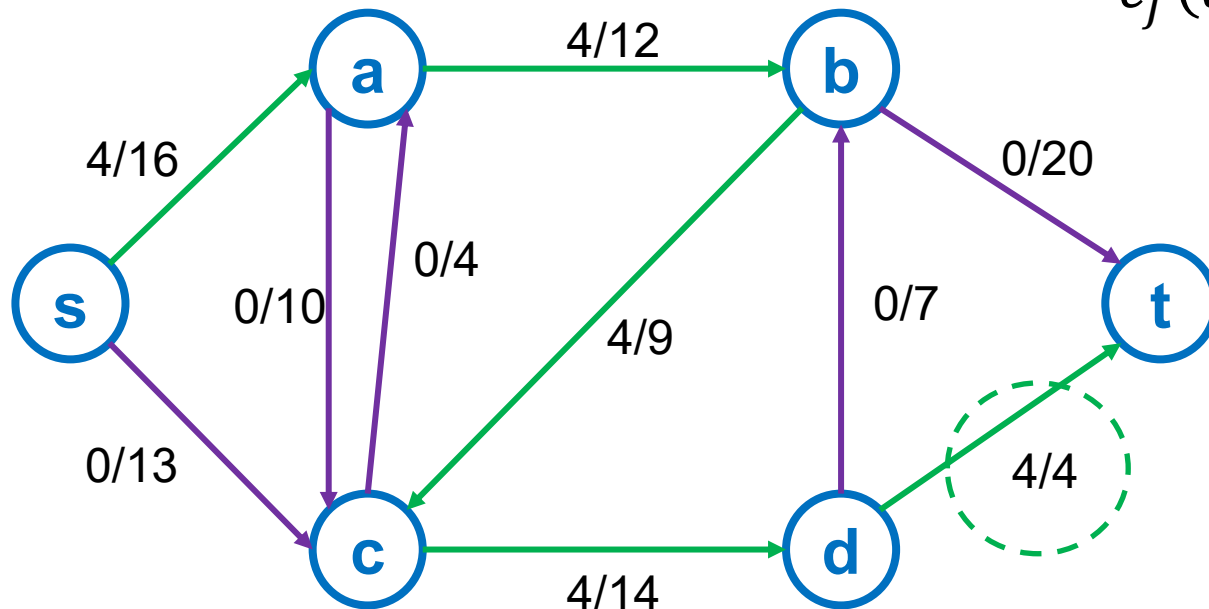
$$c_f(b \rightarrow a) = \begin{cases} c(b \rightarrow a) - f(b \rightarrow a) & \text{if } b \rightarrow a \in E \\ f(a \rightarrow b) & \text{if } a \rightarrow b \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(b \rightarrow a) = f(a \rightarrow b) = 4$$



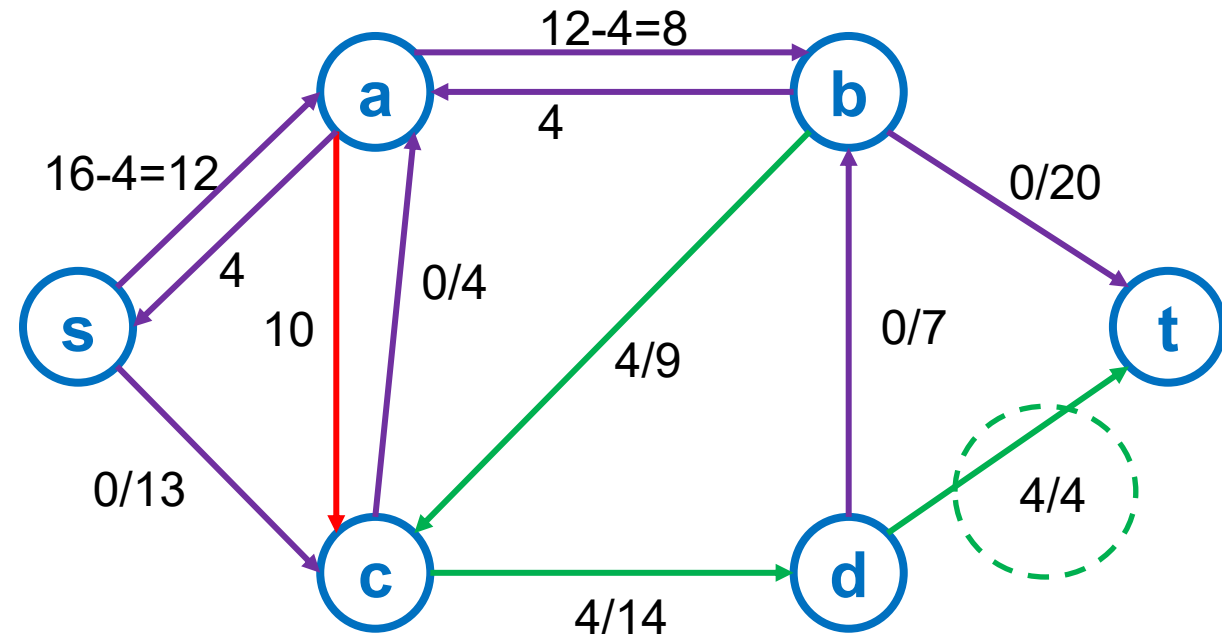
Residual capacity (5/18)

Example:



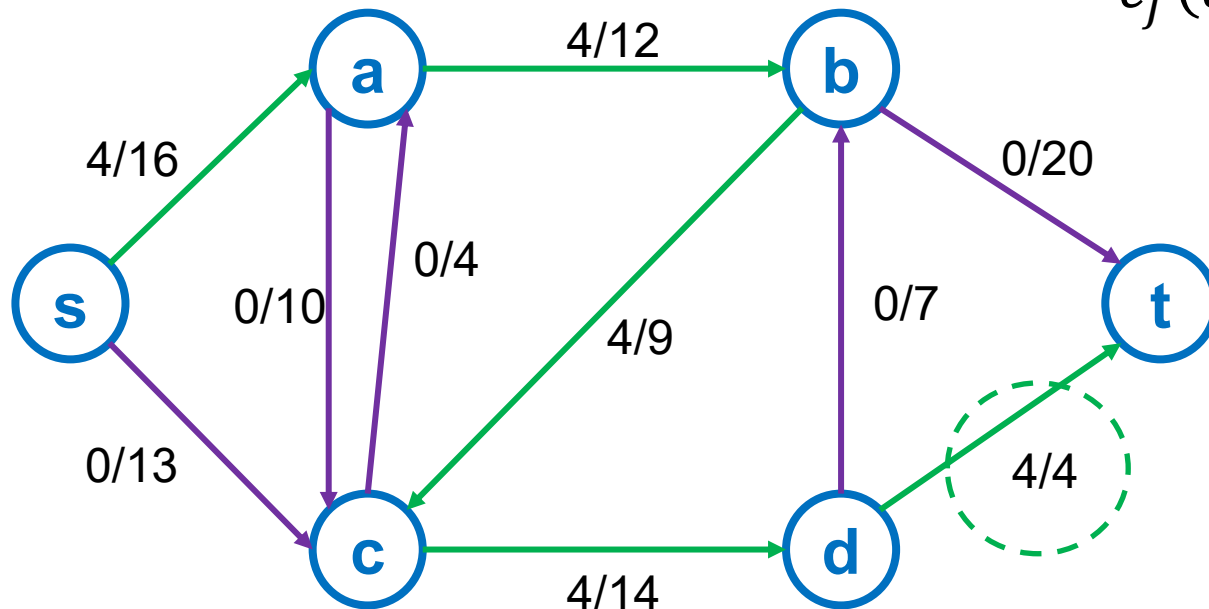
$$c_f(a \rightarrow c) = \begin{cases} c(a \rightarrow c) - f(a \rightarrow c) & \text{if } a \rightarrow c \in E \\ f(c \rightarrow a) & \text{if } c \rightarrow a \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(a \rightarrow c) = c(a \rightarrow c) - f(a \rightarrow c) = 10 - 0 = 10$$



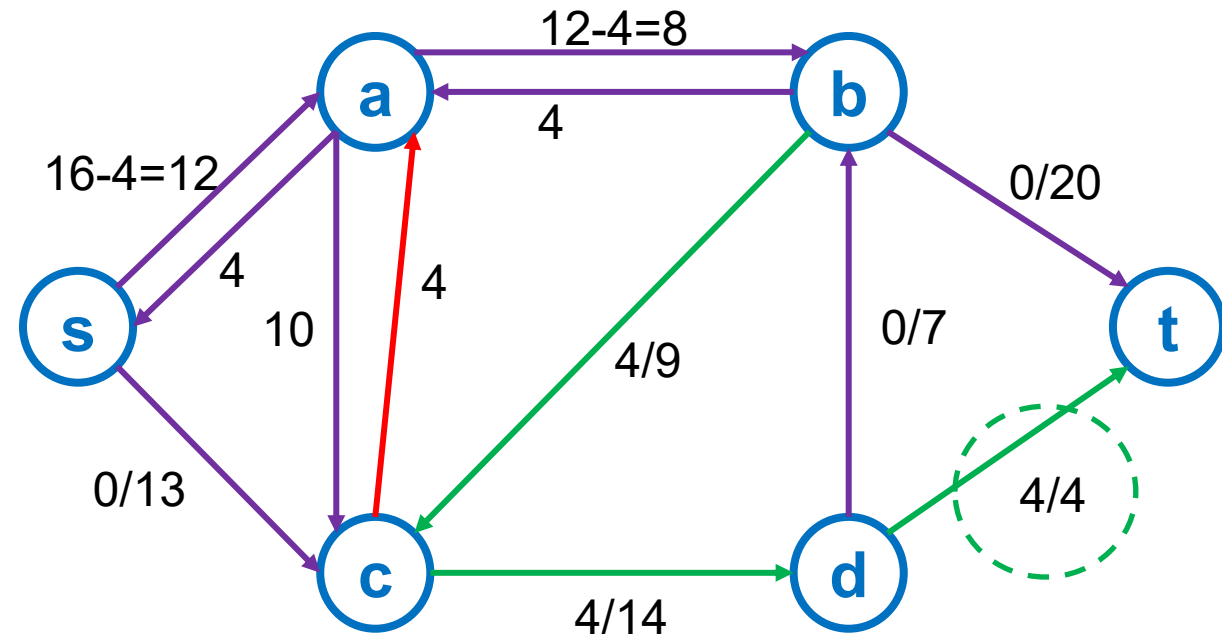
Residual capacity (6/18)

Example:



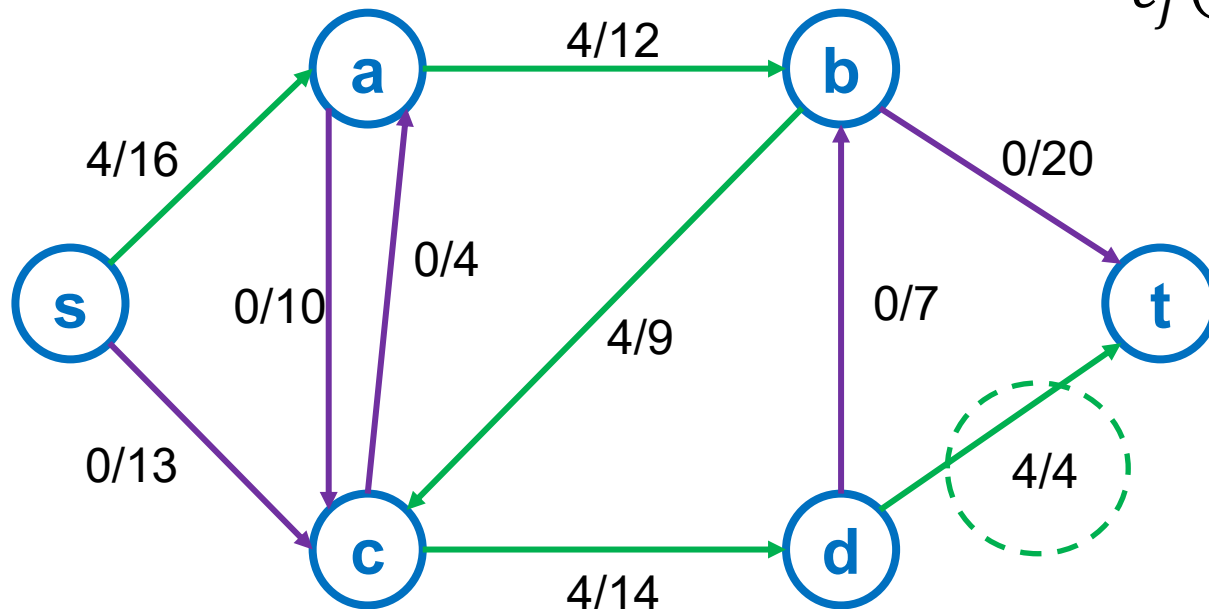
$$c_f(c \rightarrow a) = \begin{cases} c(c \rightarrow a) - f(c \rightarrow a) & \text{if } c \rightarrow a \in E \\ f(a \rightarrow c) & \text{if } a \rightarrow c \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(c \rightarrow a) = c(c \rightarrow a) - f(c \rightarrow a) = 4 - 0 = 4$$



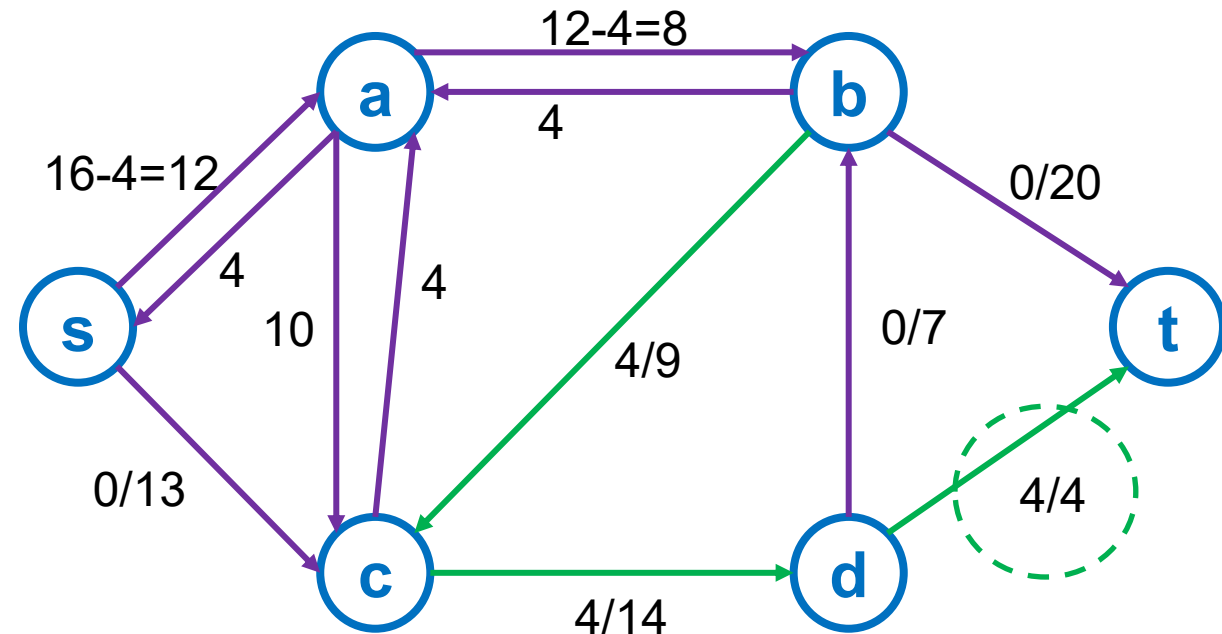
Residual capacity (7/18)

Example:



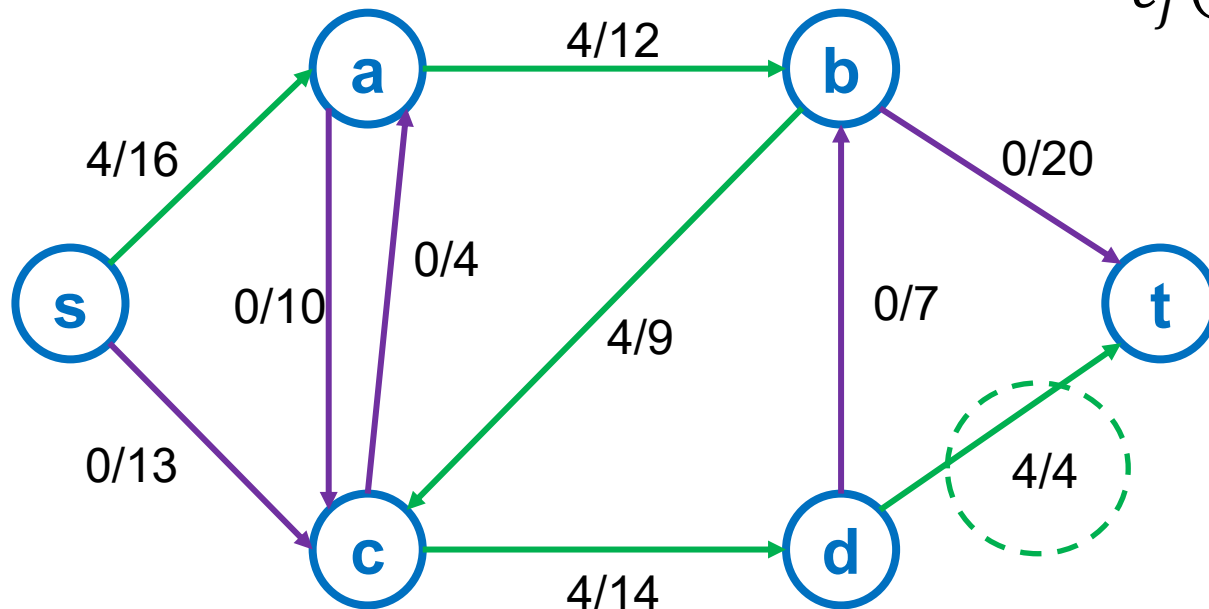
$$c_f(c \rightarrow s) = \begin{cases} c(c \rightarrow s) - f(c \rightarrow s) & \text{if } c \rightarrow s \in E \\ f(s \rightarrow c) & \text{if } s \rightarrow c \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(c \rightarrow s) = f(s \rightarrow c) = 0$$



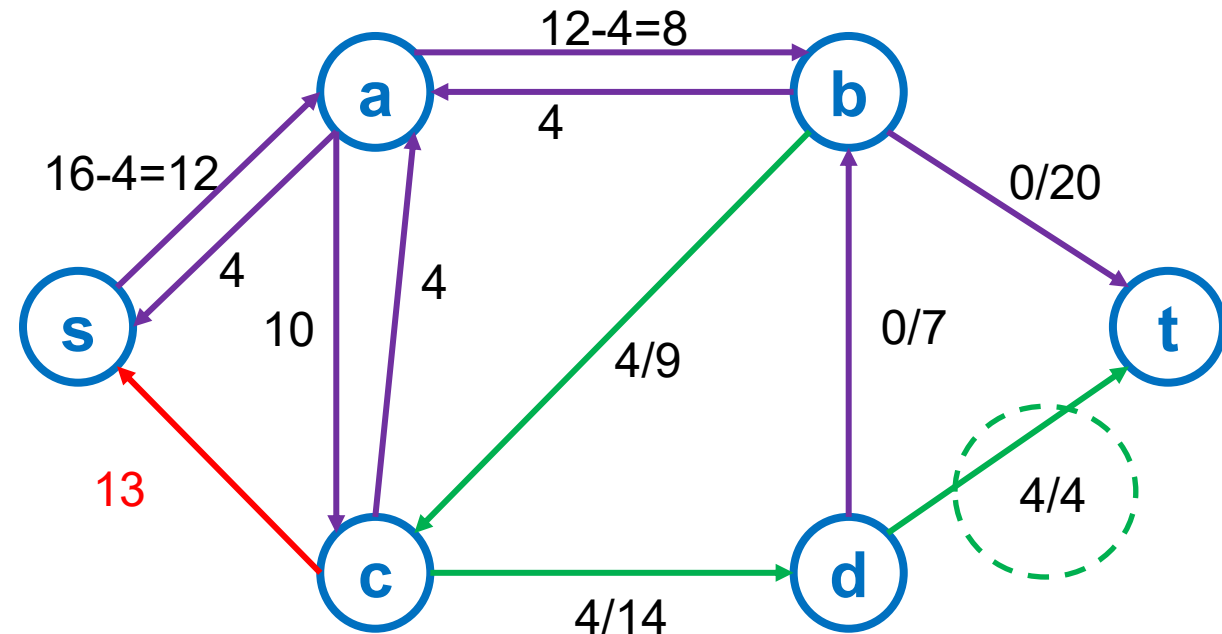
Residual capacity (8/18)

Example:



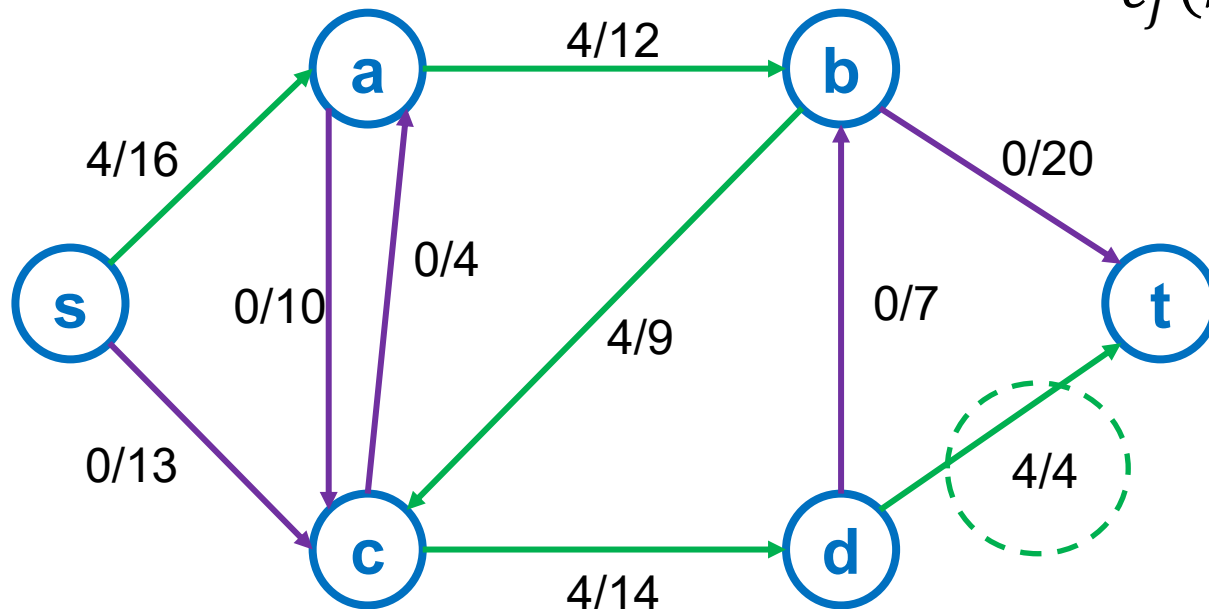
$$c_f(s \rightarrow c) = \begin{cases} c(s \rightarrow c) - f(s \rightarrow c) & \text{if } s \rightarrow c \in E \\ f(c \rightarrow s) & \text{if } c \rightarrow s \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(s \rightarrow c) = c(s \rightarrow c) - f(s \rightarrow c) = 13 - 0 = 13$$



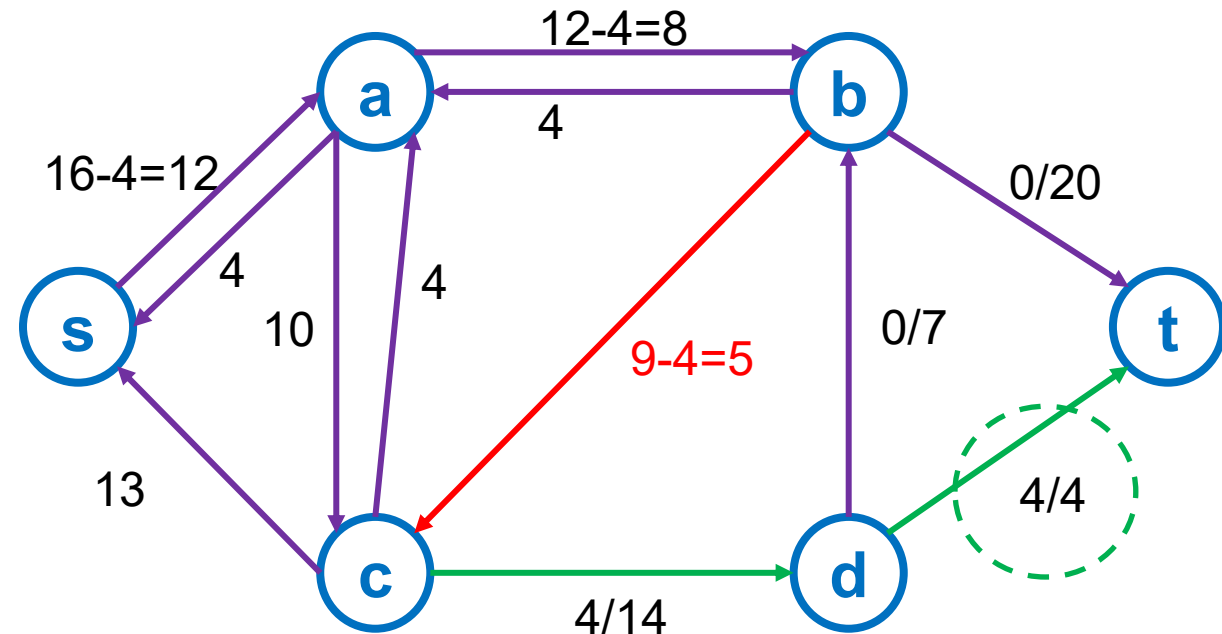
Residual capacity (9/18)

Example:



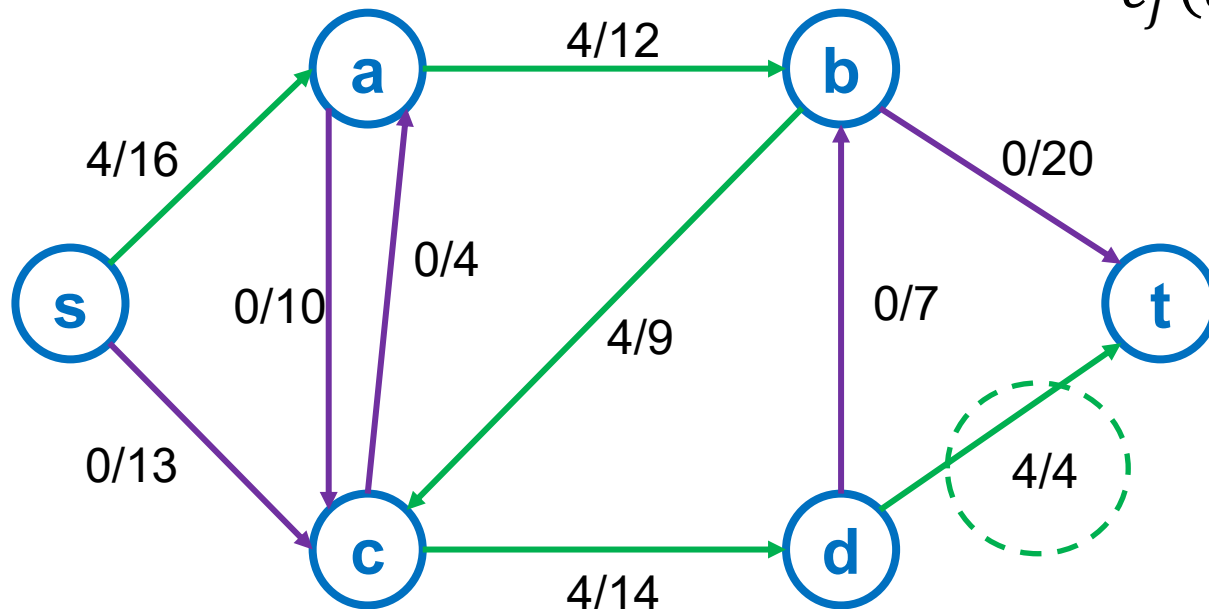
$$c_f(b \rightarrow c) = \begin{cases} c(b \rightarrow c) - f(b \rightarrow c) & \text{if } b \rightarrow c \in E \\ f(c \rightarrow b) & \text{if } c \rightarrow b \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(b \rightarrow c) = c(b \rightarrow c) - f(b \rightarrow c) = 9 - 4 = 5$$



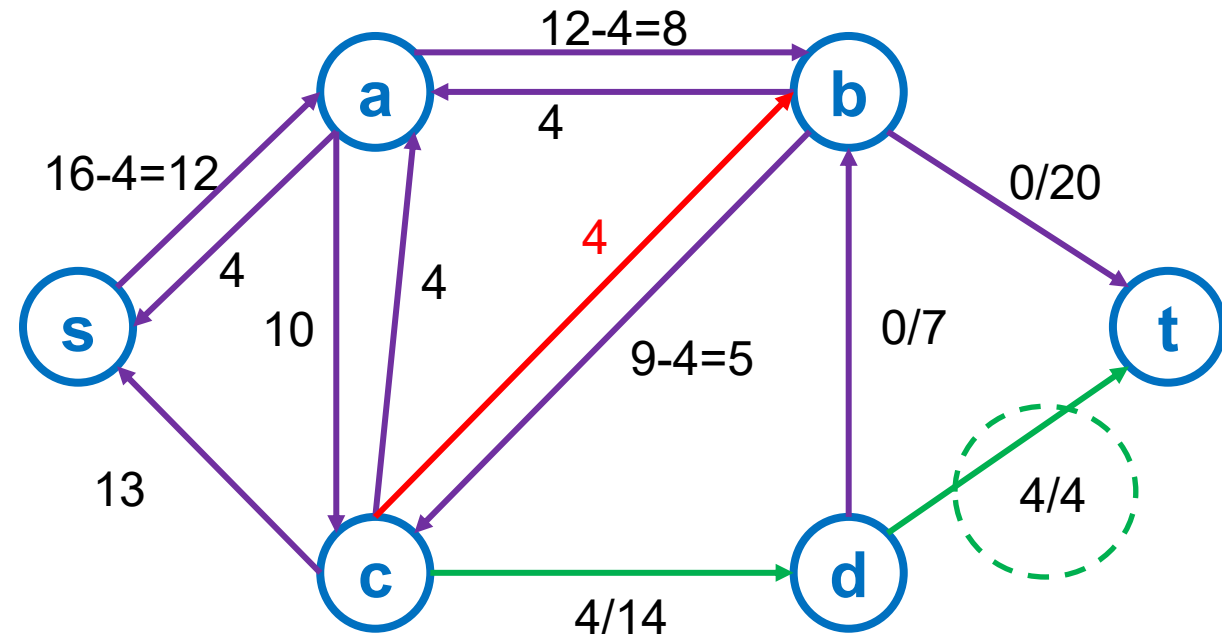
Residual capacity (10/18)

Example:



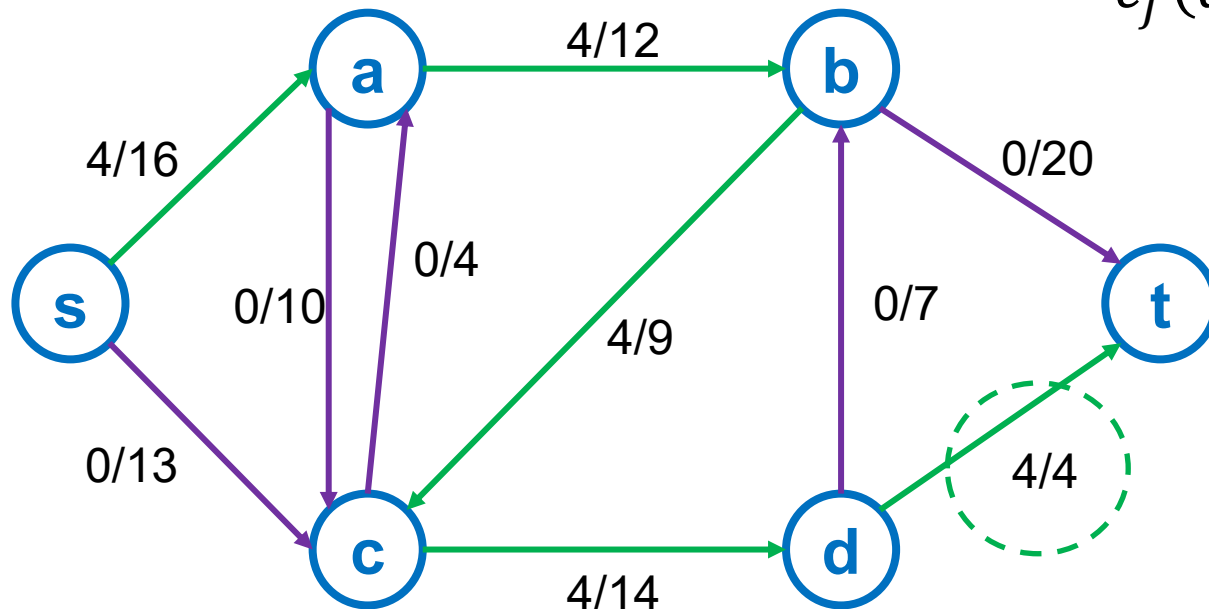
$$c_f(c \rightarrow b) = \begin{cases} c(c \rightarrow b) - f(c \rightarrow b) & \text{if } c \rightarrow b \in E \\ f(b \rightarrow c) & \text{if } b \rightarrow c \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(c \rightarrow b) = f(b \rightarrow c) = 4$$



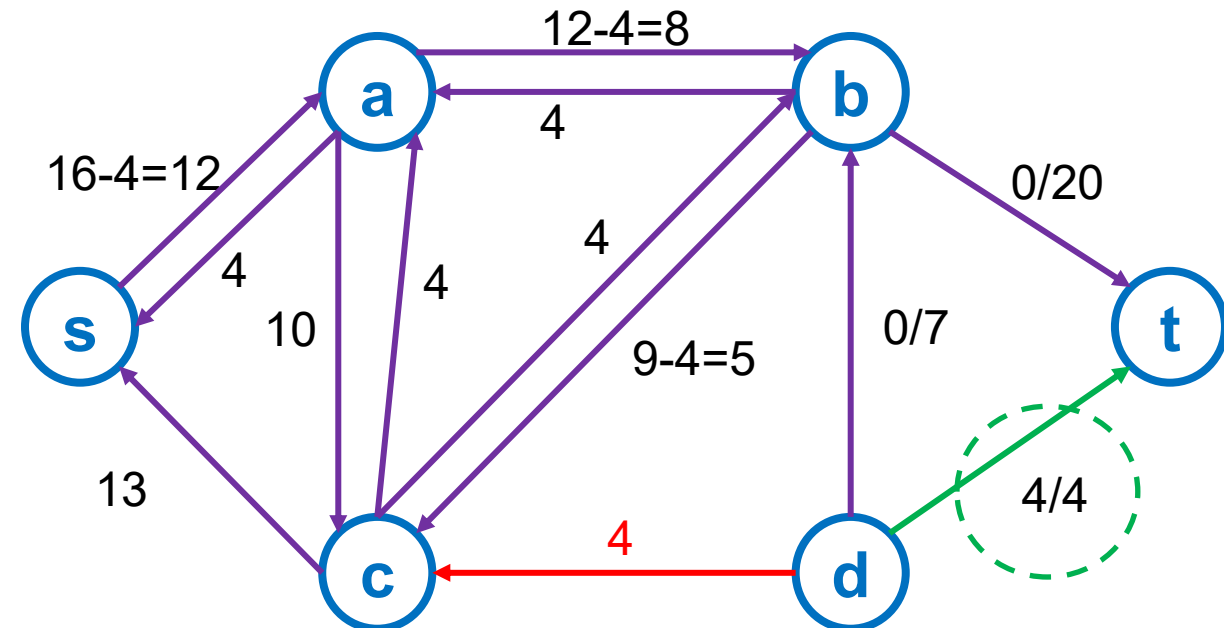
Residual capacity (11/18)

Example:



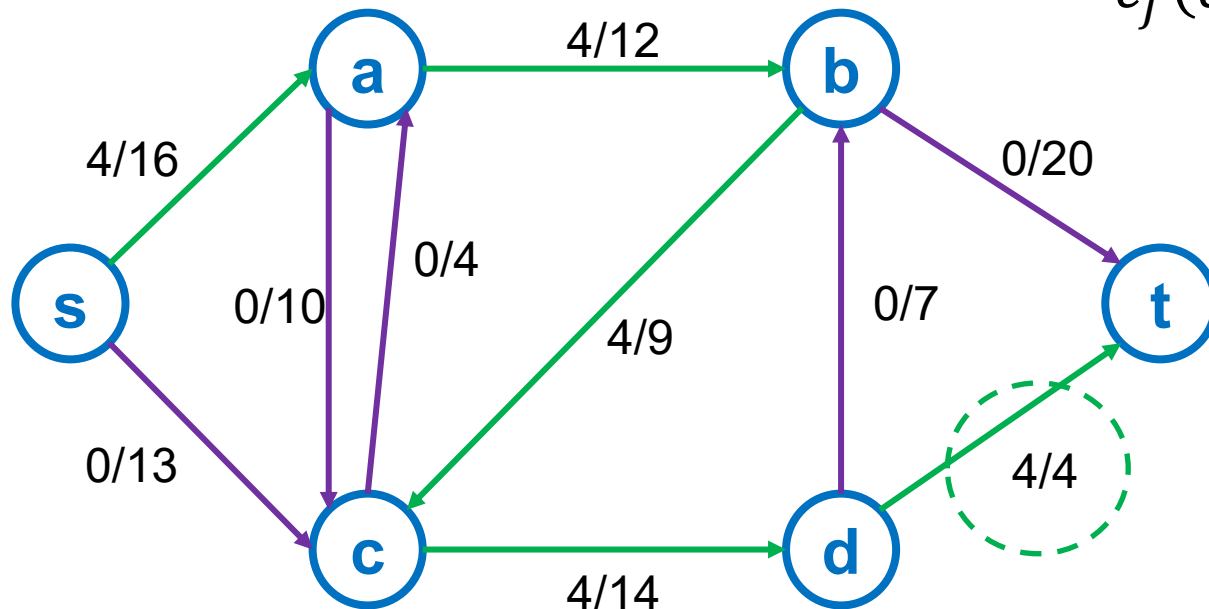
$$c_f(d \rightarrow c) = \begin{cases} c(d \rightarrow c) - f(d \rightarrow c) & \text{if } d \rightarrow c \in E \\ f(c \rightarrow d) & \text{if } c \rightarrow d \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(d \rightarrow c) = f(c \rightarrow d) = 4$$



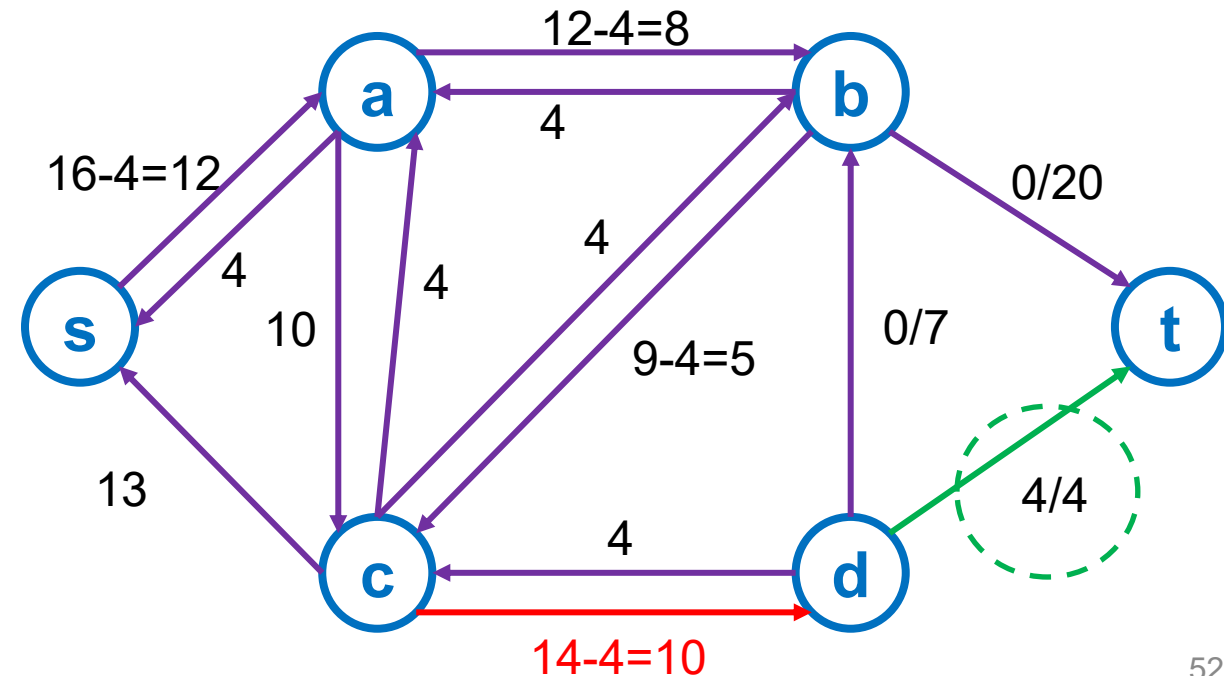
Residual capacity (12/18)

Example:



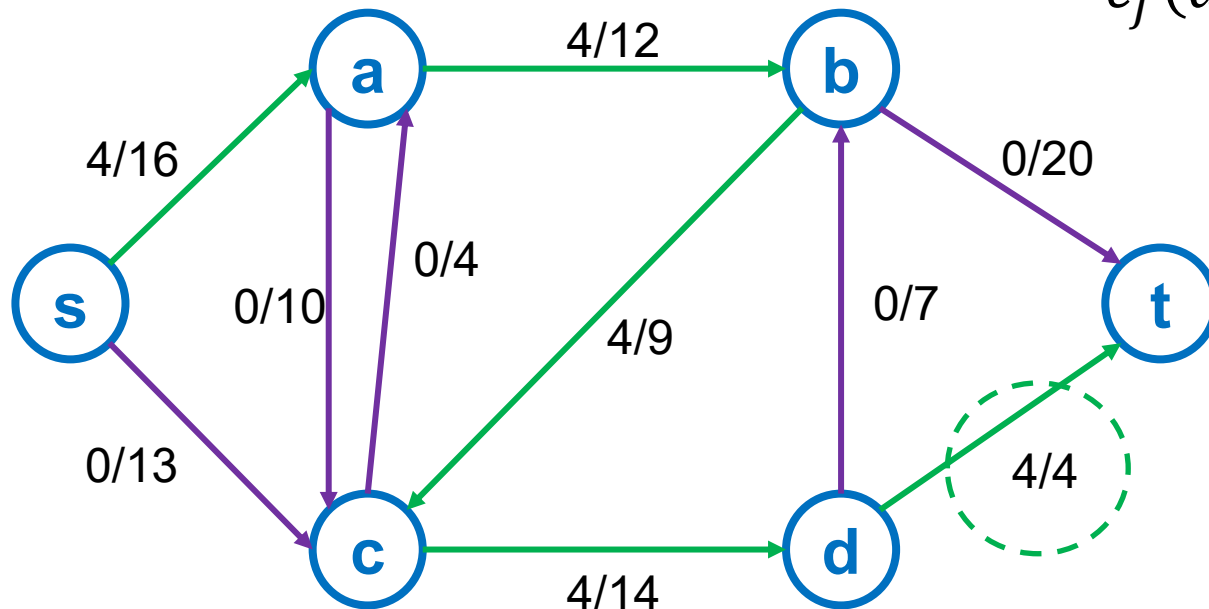
$$c_f(c \rightarrow d) = \begin{cases} c(c \rightarrow d) - f(c \rightarrow d) & \text{if } c \rightarrow d \in E \\ f(d \rightarrow c) & \text{if } d \rightarrow c \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(c \rightarrow d) = c(c \rightarrow d) - f(c \rightarrow d) = 14 - 4 = 10$$



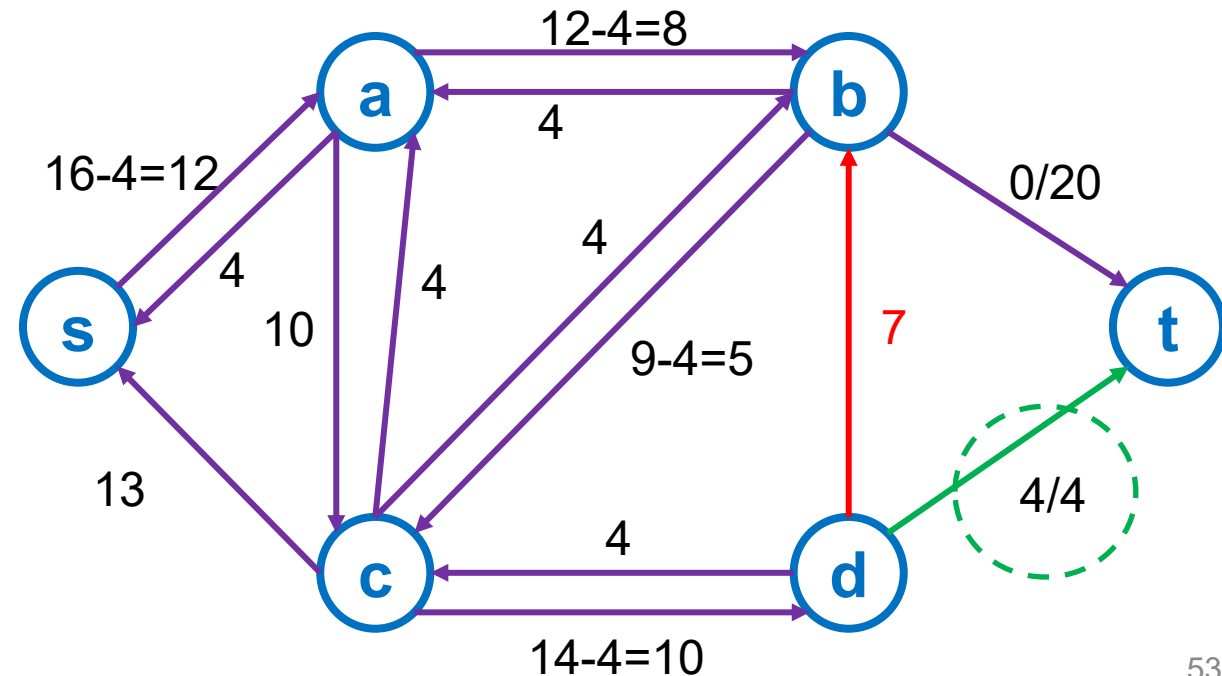
Residual capacity (13/18)

Example:



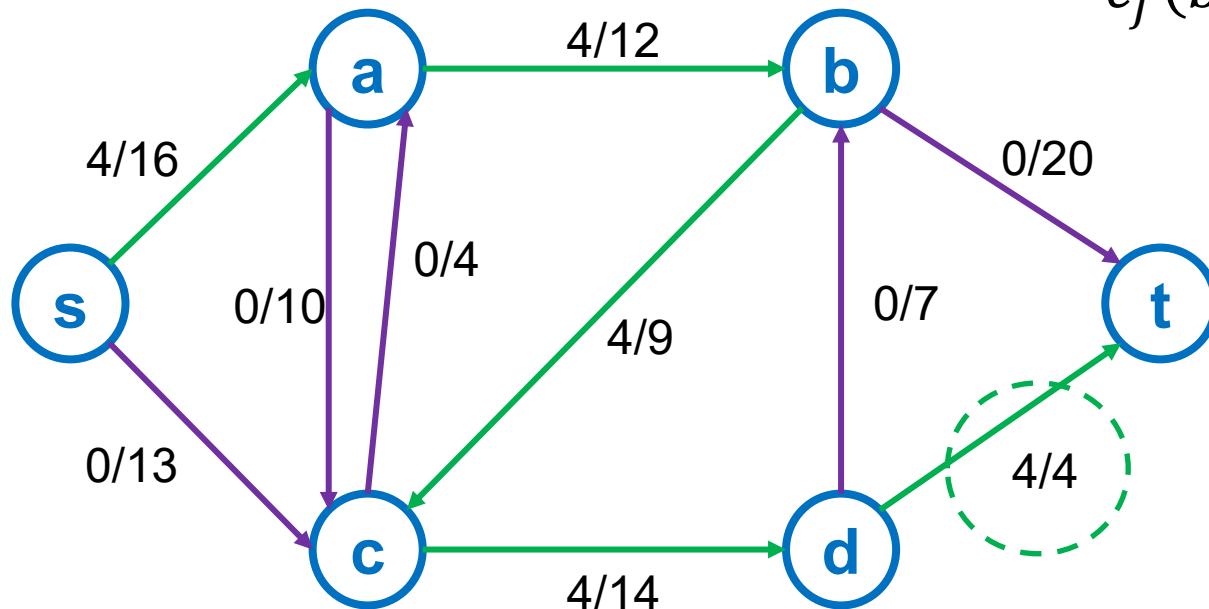
$$c_f(d \rightarrow b) = \begin{cases} c(d \rightarrow b) - f(d \rightarrow b) & \text{if } d \rightarrow b \in E \\ f(b \rightarrow d) & \text{if } b \rightarrow d \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(d \rightarrow b) = c(d \rightarrow b) - f(d \rightarrow b) = 7 - 0 = 7$$



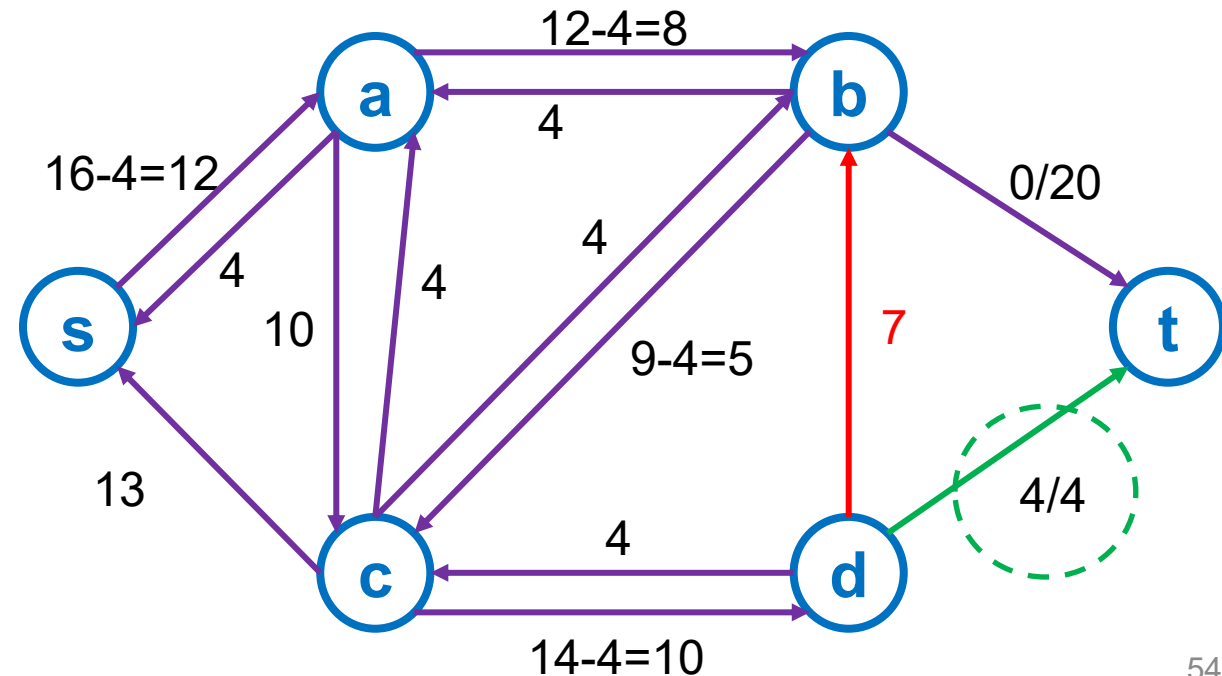
Residual capacity (14/18)

Example:



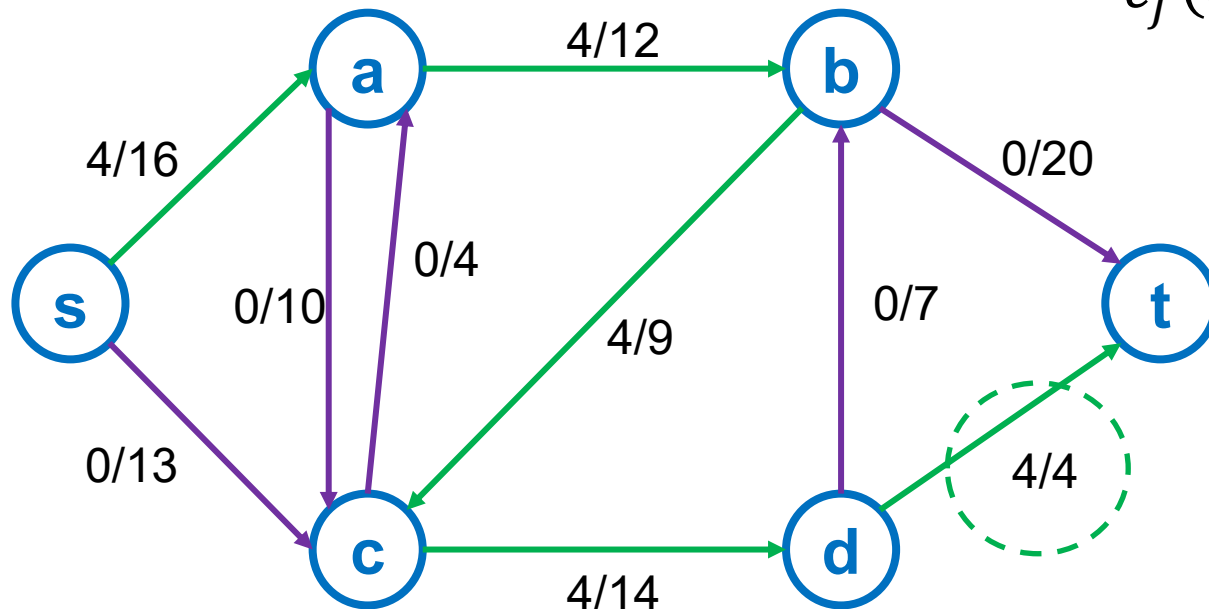
$$c_f(b \rightarrow d) = \begin{cases} c(b \rightarrow d) - f(b \rightarrow d) & \text{if } b \rightarrow d \in E \\ f(d \rightarrow b) & \text{if } d \rightarrow b \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(b \rightarrow d) = f(d \rightarrow b) = 0$$



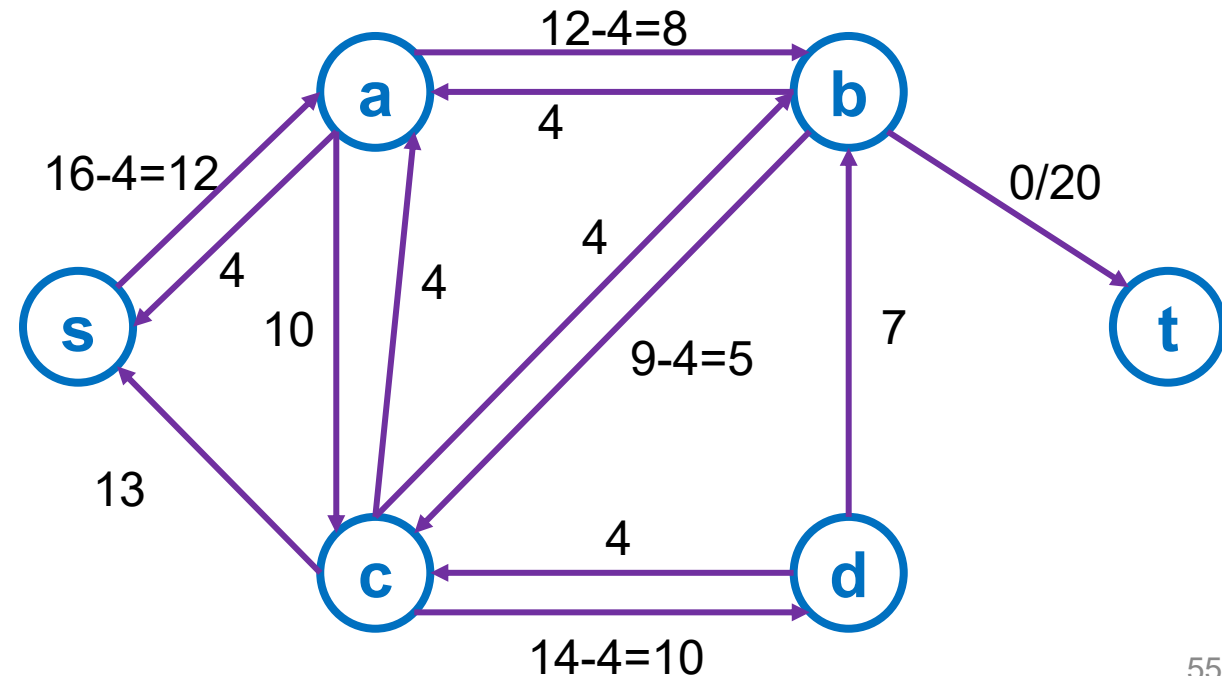
Residual capacity (15/18)

Example:



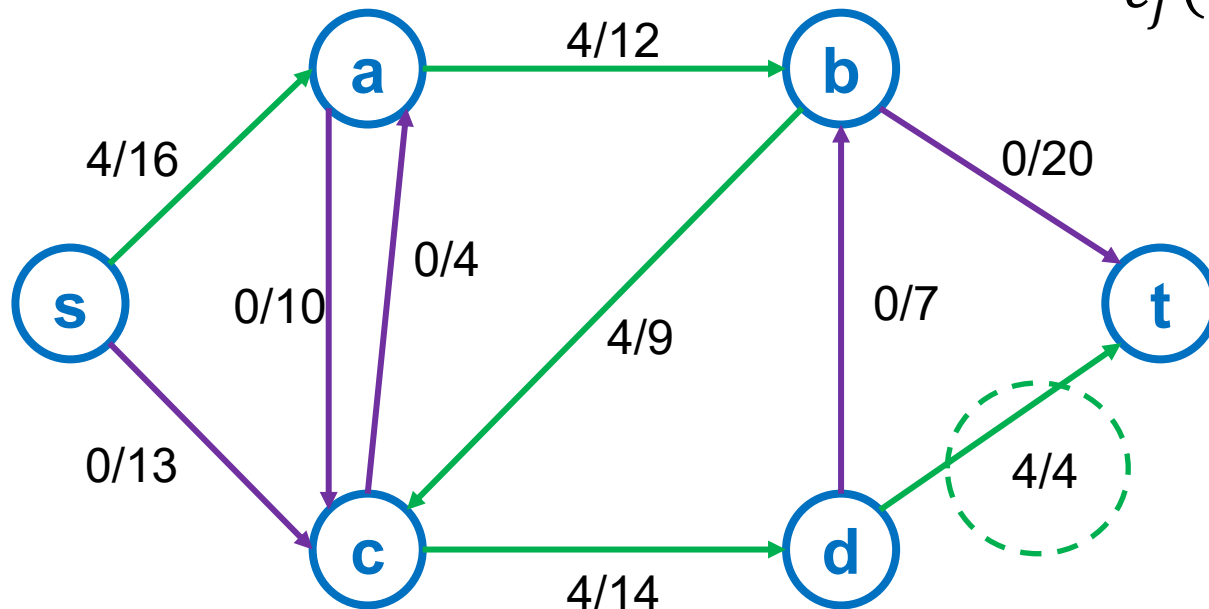
$$c_f(d \rightarrow t) = \begin{cases} c(d \rightarrow t) - f(d \rightarrow t) & \text{if } d \rightarrow t \in E \\ f(t \rightarrow d) & \text{if } t \rightarrow d \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(d \rightarrow t) = c(d \rightarrow t) - f(d \rightarrow t) = 4 - 4 = 0$$



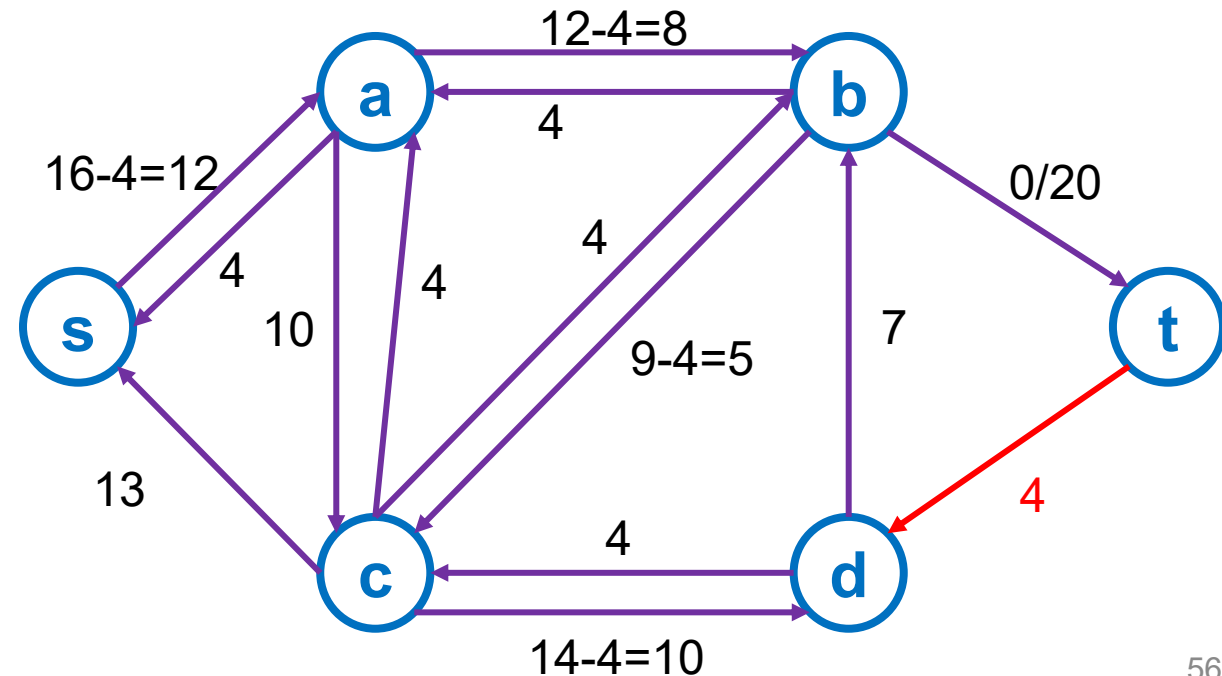
Residual capacity (16/18)

Example:



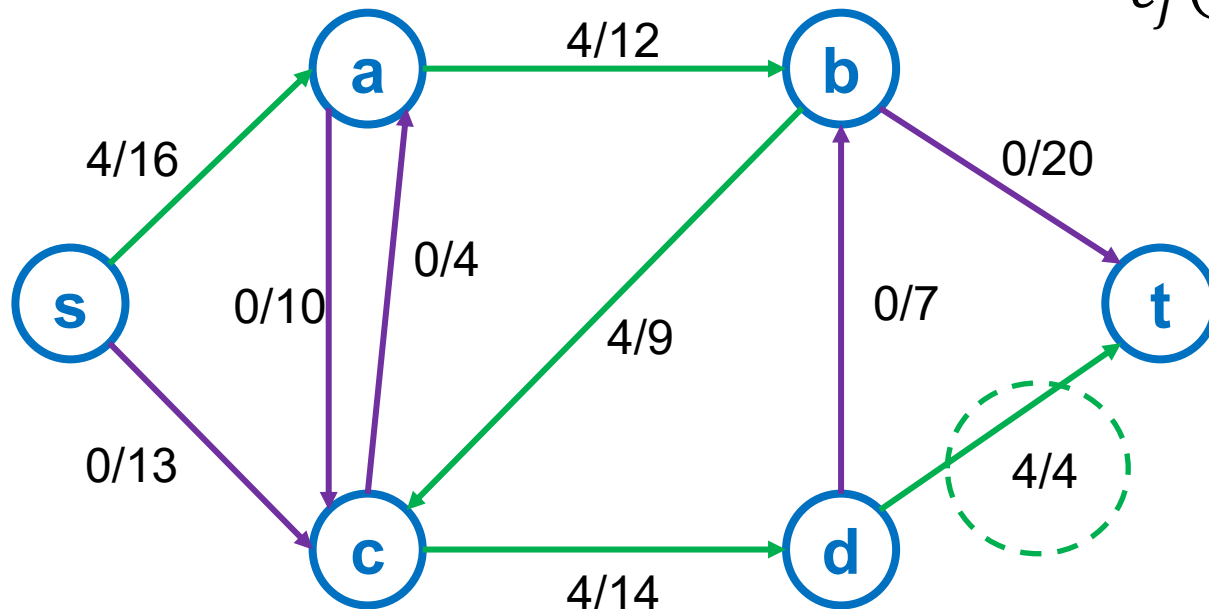
$$c_f(t \rightarrow d) = \begin{cases} c(t \rightarrow d) - f(t \rightarrow d) & \text{if } t \rightarrow d \in E \\ f(d \rightarrow t) & \text{if } d \rightarrow t \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(t \rightarrow d) = f(d \rightarrow t) = 4$$



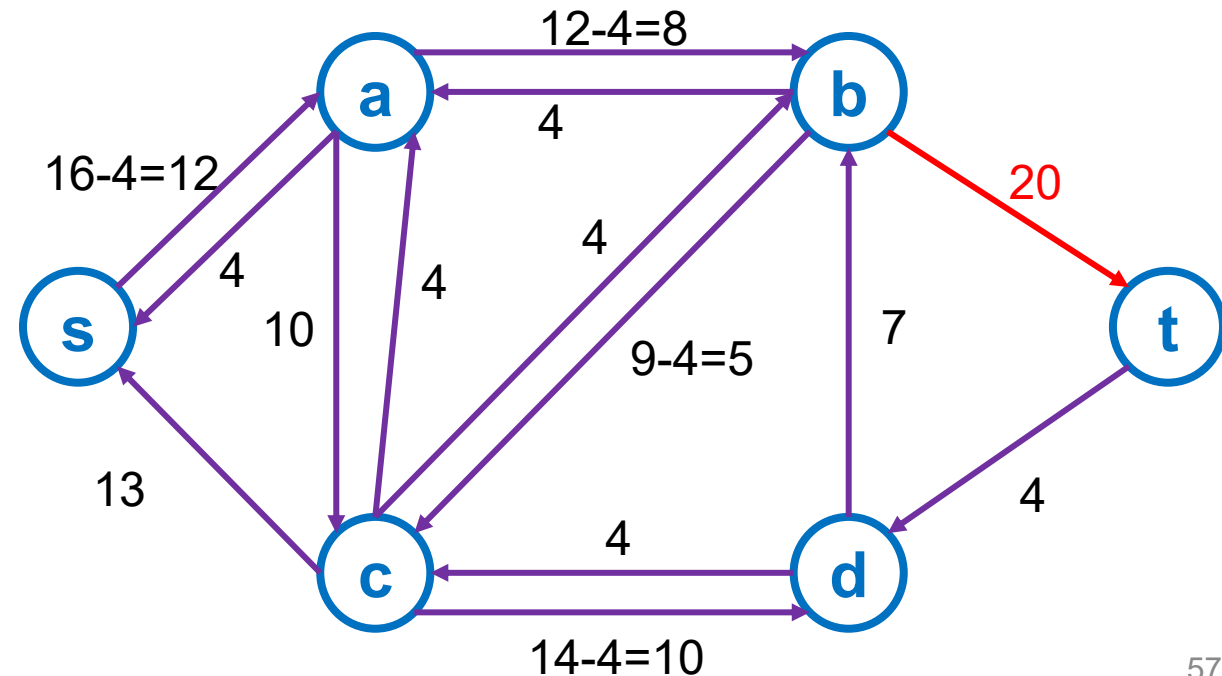
Residual capacity (17/18)

Example:



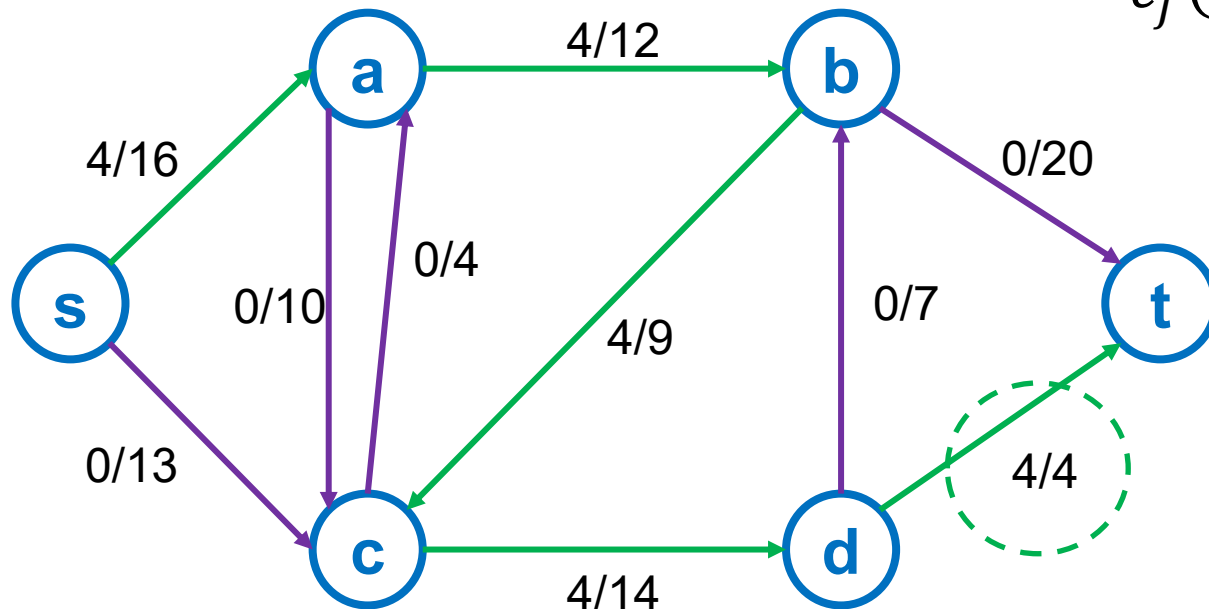
$$c_f(b \rightarrow t) = \begin{cases} c(b \rightarrow t) - f(b \rightarrow t) & \text{if } b \rightarrow t \in E \\ f(t \rightarrow b) & \text{if } t \rightarrow b \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(b \rightarrow t) = c(b \rightarrow t) - f(b \rightarrow t) = 20 - 0 = 20$$



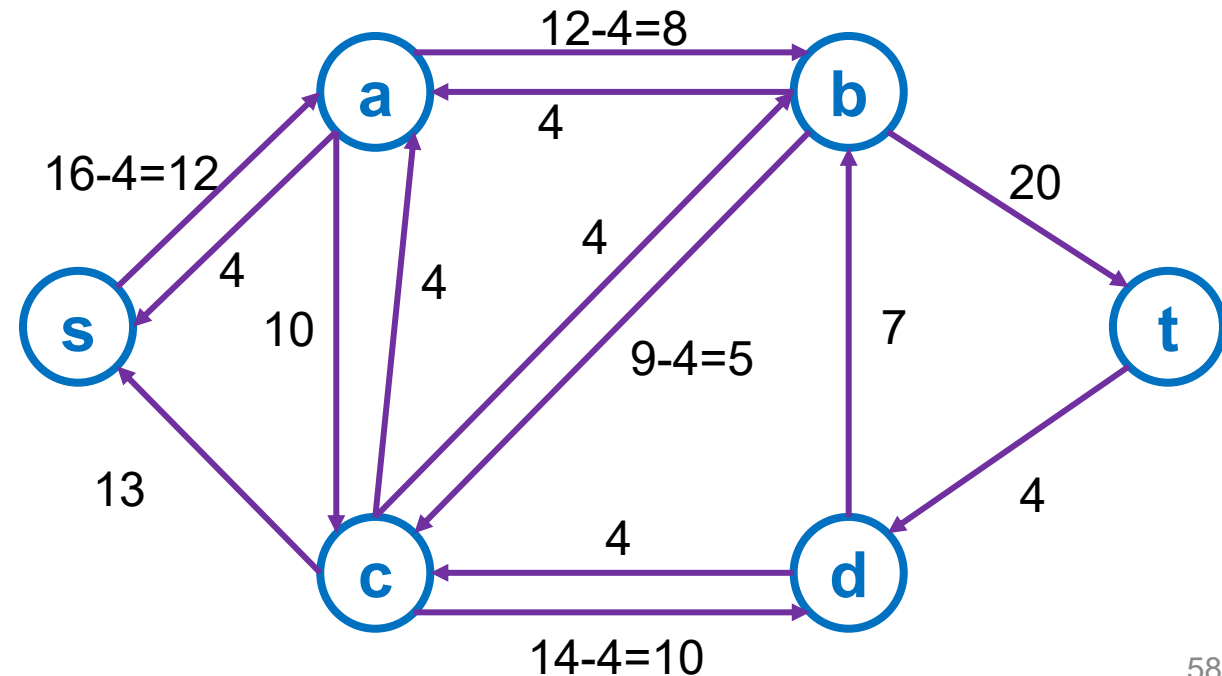
Residual capacity (18/18)

Example:



$$c_f(t \rightarrow b) = \begin{cases} c(t \rightarrow b) - f(t \rightarrow b) & \text{if } t \rightarrow b \in E \\ f(b \rightarrow t) & \text{if } b \rightarrow t \in E \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(t \rightarrow b) = f(b \rightarrow t) = 0$$

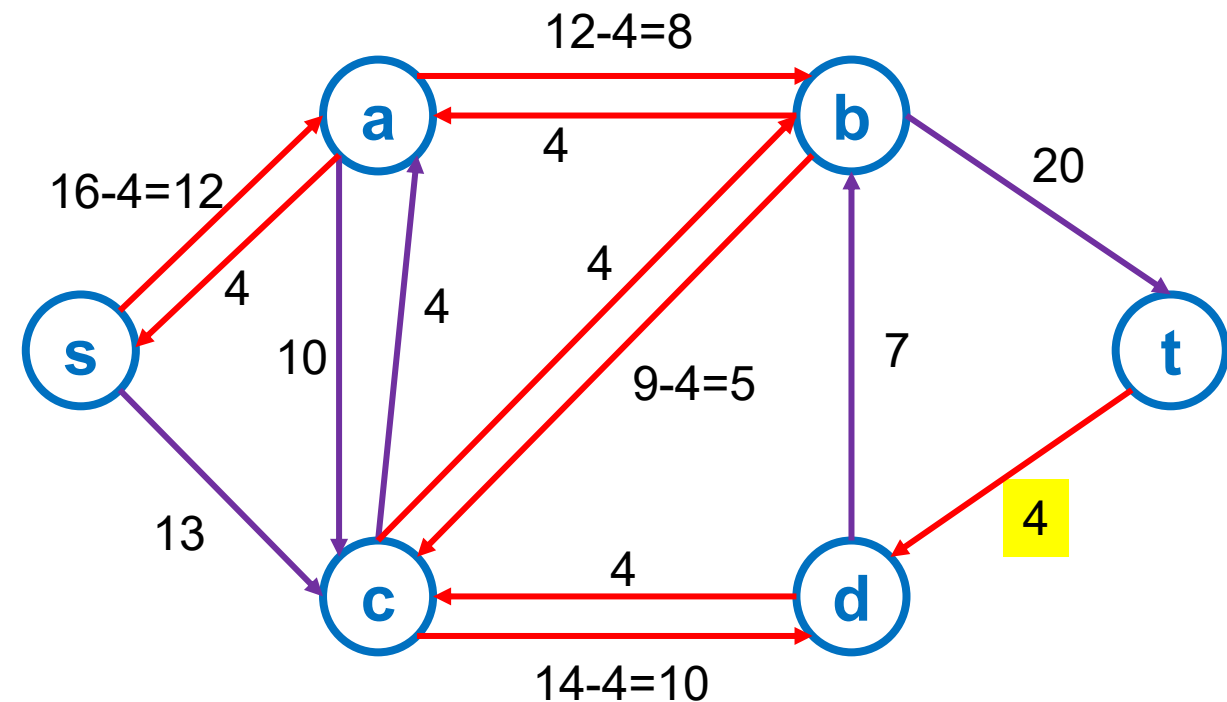
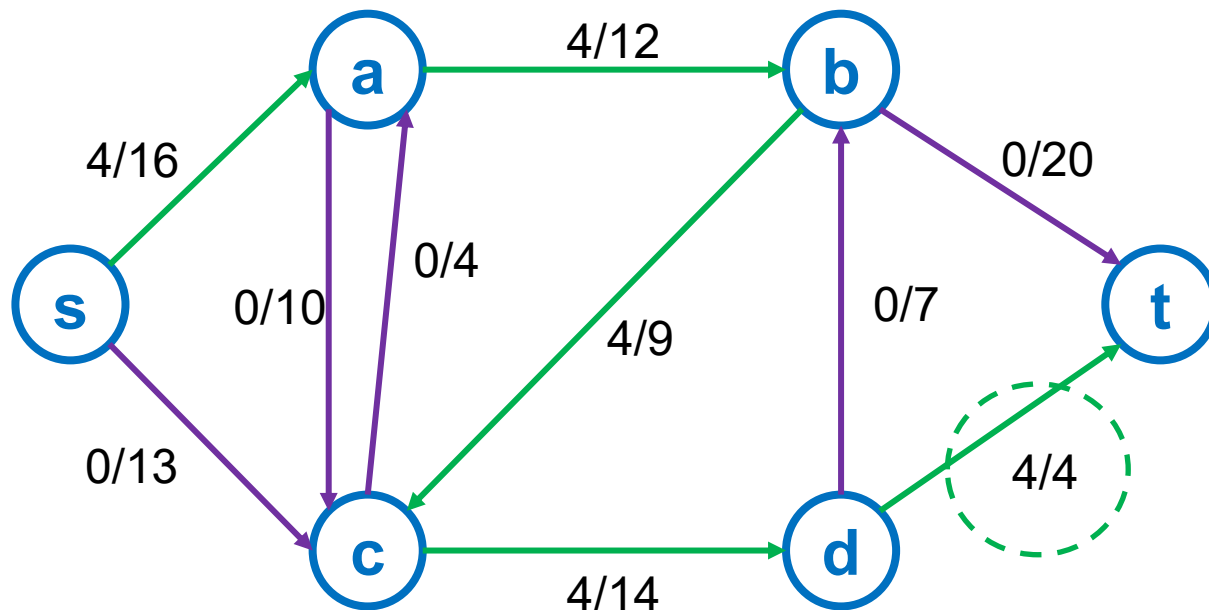


Maxflow-Mincut Theorem Proof

- The residual graph $G_f = (V, E_f)$ is defined where E_f is the set of edges whose residual capacity is positive.
- It is possible to have $c_f(u \rightarrow v)$ even if $u \rightarrow v$ is not an edge in the original graph G .

Residual capacity

Example:



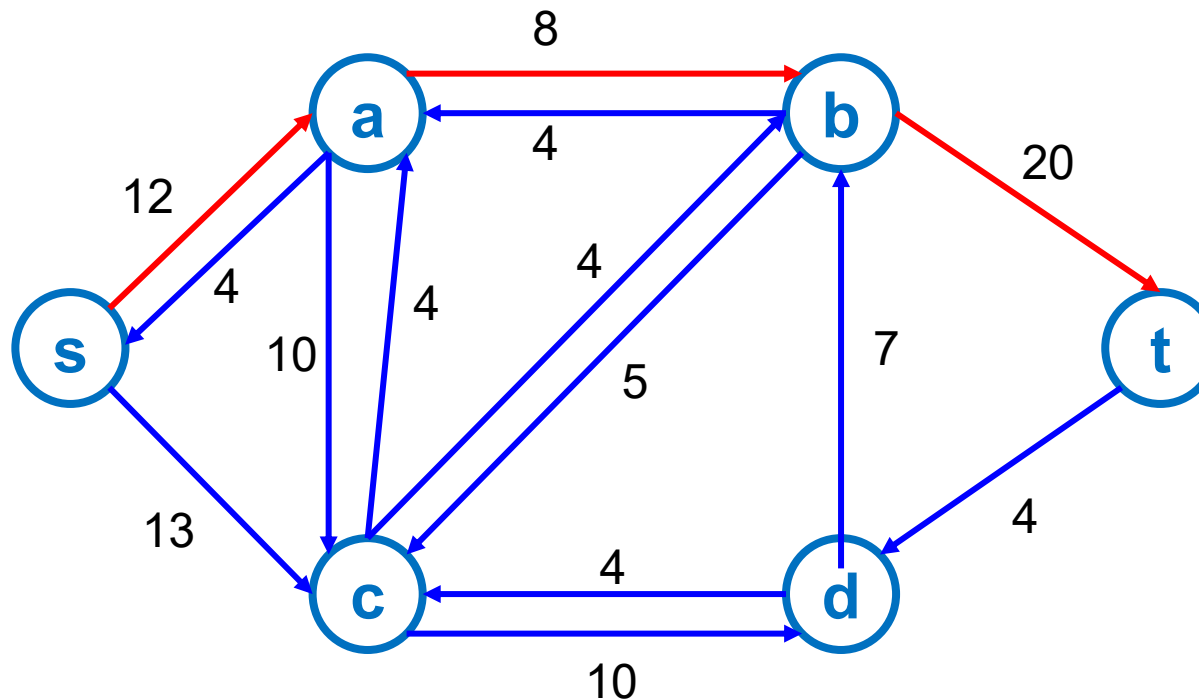
Maxflow-Mincut Theorem Proof

Consider two cases:

- **Case 1:** There is a directed path from source vertex s to the target vertex t in the residual graph G_f
- **Case 2:** There isn't.

Augmenting Path

Definition 6. Suppose G_f contains a directed path P from source node s to target node t . We call P an **augmenting path**.



Augmenting Path

Definition 7. Let $F = \min_{u \rightarrow v \in P} c_f(u \rightarrow v)$ denote the maximum amount of flow that can push through P . A new flow $f': E \rightarrow \mathbb{R}$ in the **original graph** is defined as follows:

This one updates the base flow, not the augmented one.

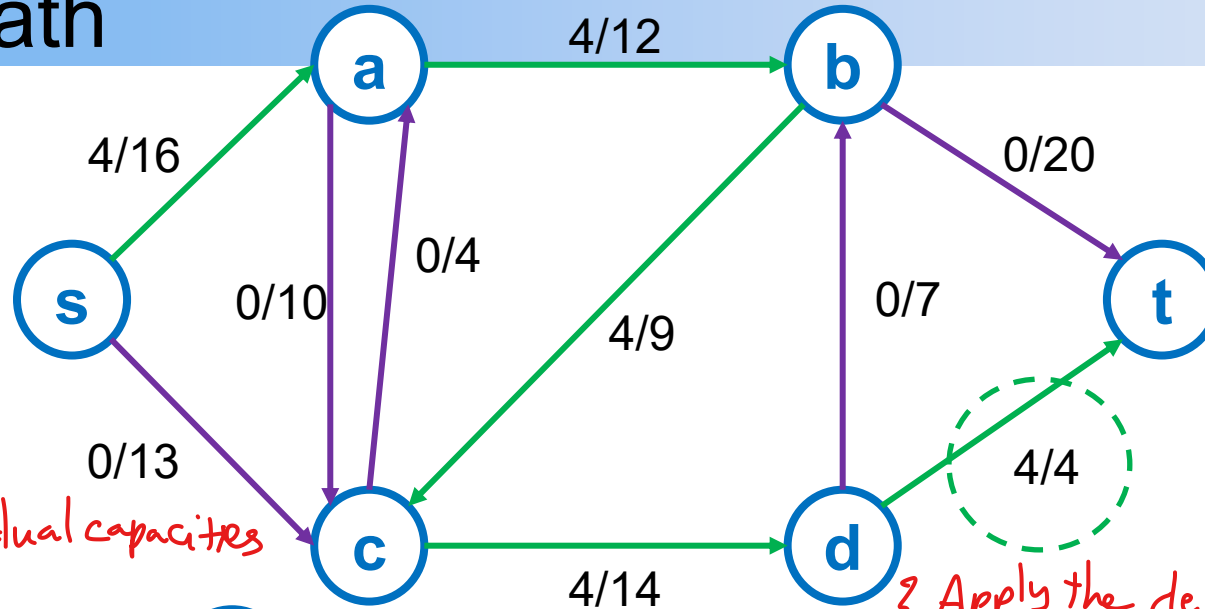
$$f'(u \rightarrow v) = \begin{cases} f(u \rightarrow v) + F & \text{if } u \rightarrow v \in P \\ f(u \rightarrow v) - F & \text{if } v \rightarrow u \in P \\ f(u \rightarrow v) & \text{otherwise} \end{cases}$$

Flow augmenting

Flow reclamation

Augmenting Path

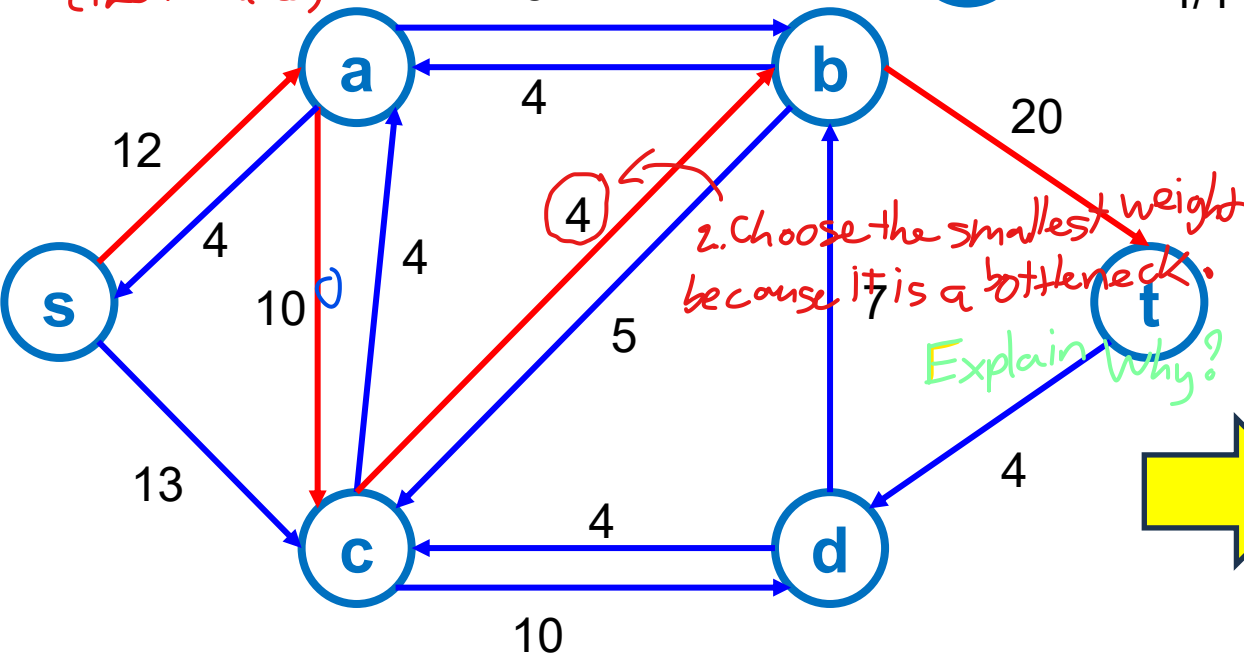
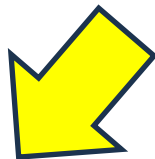
Example: $F = 4$



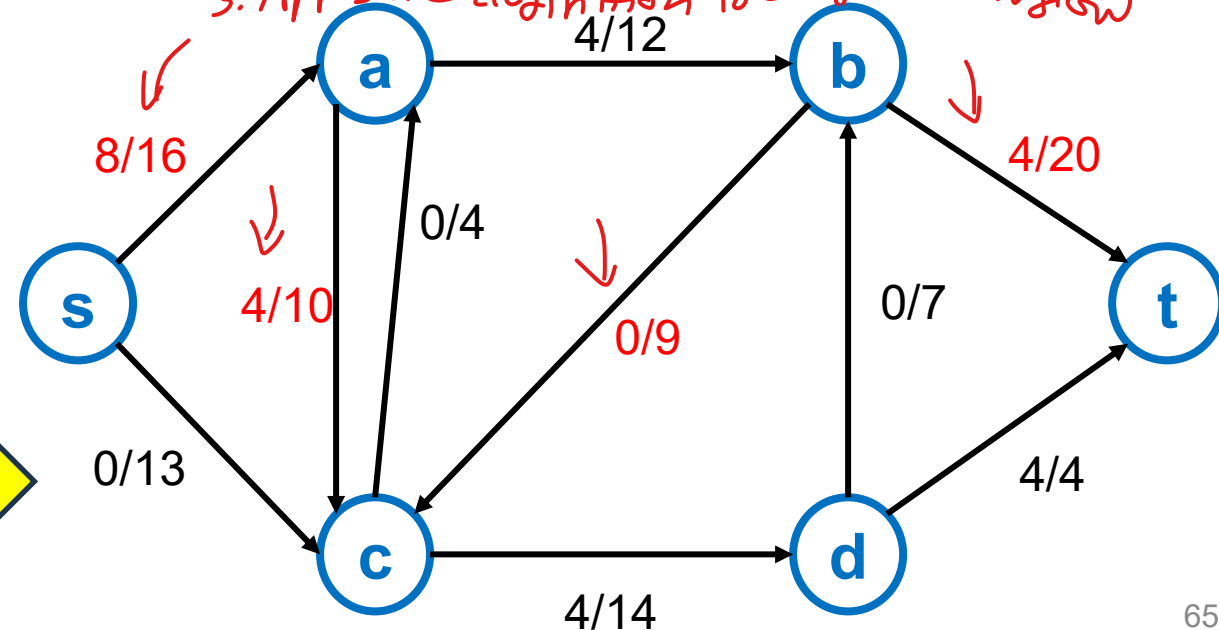
Definition
 $f'(u \rightarrow v)$

$$= \begin{cases} f(u \rightarrow v) + F & \text{if } u \rightarrow v \in P \\ f(u \rightarrow v) - F & \text{if } v \rightarrow u \in P \\ f(u \rightarrow v) & \text{otherwise} \end{cases}$$

1. Choose a path from residual capacities (The red line)



3. Apply the definition to create a new flow



Augmenting Path

Claim 1. This new flow f' is feasible w.r.t the original capacities c meaning $f' \geq 0$ and $f' \leq c$ everywhere.

Max flow algorithm

1. Update the base flow using residual capacities
2. Repeat until there are no available augmenting paths (Using path finding algorithm to find paths)

1. Choose a path from residual capacities
2. Find the smallest weight in that path
3. Update the base flow by adding the smallest weight to the path
4. Check if the inputs == outputs
If wrong, recheck the steps!!!

Maxflow-Mincut Theorem Proof

Consider an edge $u \rightarrow v$ in the original graph G . There are three cases to consider:

- *Case 1:* If the augmenting path P contains $u \rightarrow v$;
- *Case 2:* If the augmenting path P contains the reversed edge $v \rightarrow u$;
- *Case 3:* If neither $u \rightarrow v$ nor $v \rightarrow u$ is in the augmenting path.

Maxflow-Mincut Theorem Proof

Case 1: If the augmenting path P contains $u \rightarrow v$ then

$$f'(u \rightarrow v) = f(u \rightarrow v) + F > f(u \rightarrow v) \geq 0,$$

because f is feasible, and

$$f'(u \rightarrow v) = f(u \rightarrow v) + F \text{ (by def of } f')$$

$$\leq f(u \rightarrow v) + c_f(u \rightarrow v) \text{ (by def of } F)$$

$$= f(u \rightarrow v) + c(u \rightarrow v) - f(u \rightarrow v) \text{ (by def of } c_f)$$

$$= c(u \rightarrow v)$$

Maxflow-Mincut Theorem Proof

Case 2: If the augmenting path P contains the reversed edge $v \rightarrow u$, then

$$f'(u \rightarrow v) = f(u \rightarrow v) - F < f(u \rightarrow v) \leq c(u \rightarrow v),$$

due to because f is feasible, and

$$\begin{aligned} f'(u \rightarrow v) &= f(u \rightarrow v) - F \text{ (by def of } f') \\ &\geq f(u \rightarrow v) - c_f(v \rightarrow u) \text{ (by def of } F) \\ &= f(u \rightarrow v) - f(u \rightarrow v) \text{ (by def of } c_f) \\ &= 0 \end{aligned}$$

Maxflow-Mincut Theorem Proof

Case 3: If neither $u \rightarrow v$ nor $v \rightarrow u$ is in the augmenting path, then

$$f'(u \rightarrow v) = f(u \rightarrow v).$$

Therefore, we have

$$0 \leq f'(u \rightarrow v) \leq c(u \rightarrow v)$$

due to f is feasible.

So f' is indeed feasible.

Maxflow-Mincut Theorem Proof

Finally, only the first edge in the augmenting path leaves s , which implies

$$|f'| = |f| + F > |f|$$

Thus, f' is a feasible flow with larger value than f .

So, if there is a path from s to t in the residual graph G_f , then f is **not maximum flow**.

Maxflow-Mincut Theorem Proof

Suppose the residual graph G_f does not contain a directed path from s to t . Let S be the set of vertices that are reachable from s in G_f and let $T = V \setminus S$. The partition (S, T) is clearly an (s, t) -cut.

For every vertex $u \in S$ and $v \in T$, we have

$$c_f(u \rightarrow v) = [c(u \rightarrow v) - f(u \rightarrow v)] + f(v \rightarrow u) = 0.$$

Maxflow-Mincut Theorem Proof

The feasibility of f implies that

$$c(u \rightarrow v) - f(u \rightarrow v) \geq 0 \text{ and } f(v \rightarrow u) \geq 0,$$

so that we must have:

$$\begin{cases} c(u \rightarrow v) - f(u \rightarrow v) = 0 \\ f(v \rightarrow u) = 0 \end{cases}$$

In other words, our flow f saturates every edge from S to T and avoids every edge from T to S . By the **Lemma 1**, $|f| = \|S, T\|$.

Outline

1. Flows
2. Cuts
3. The Maxflow-Mincut Theorem
4. Ford and Fulkerson's Algorithm
5. Edmonds and Karp's Algorithms

History

Lester Randolph Ford Jr.
(1927–2017)



Delbert Ray Fulkerson
(1924–1976)

- The goal of the algorithm/method is to compute the maximum flow in a flow network.
- It was published in 1956 by L. R. Ford Jr. and D. R. Fulkerson [1]
- It is sometimes called a “method” instead of an “algorithm” as the approach to finding augmenting paths in a residual graph is not fully specified.

[1] Ford, L. R.; Fulkerson, D. R. (1956). "Maximal flow through a network". Canadian Journal of Mathematics.

Ford and Fulkerson's Algorithm

- The Ford-Fulkerson method is iterative,
- Starts with $f(u, v)$ for $u, v \in V$, initial flow of value 0.
- The method is based on the augmenting path which is defined as a path from s to t along which we can push more flow and then augment flow along this path.

Ford and Fulkerson's Algorithm

Corollary 1. If all capacities in a flow network are integers, then there is a maximum flow such that the flow through every edge is an integer.

Ford and Fulkerson's Algorithm

Corollary 1. If all capacities in a flow network are integers, then there is a maximum flow such that the flow through every edge is an integer.

Proof. We argue by induction that after each iteration of the augmenting path algorithm, all flow values and all residual capacities are integers.

Ford and Fulkerson's Algorithm

Proof (cont).

- Before the first iteration, all flow values are 0 (which is an integer), and all residual capacities are the original capacities, which are integers by definition.
- In each later iteration, the induction hypothesis implies that the capacity F of the augmenting path is an integer, so augmenting changes the flow on each edge, and therefore the residual capacity of each edge, by an integer.

Ford and Fulkerson's Algorithm

Proof (cont).

In particular, each iteration of the augmenting path algorithm increases the value of the flow by a positive integer.

It follows that the algorithm eventually halts and returns a maximum flow.

Ford and Fulkerson's Algorithm

Algorithm **FordFulkerson**(Graph G , Source s , Target t):

Initialize flow $f(u, v) = 0$ for all edges (u, v) in G

While there exists an augmenting path P from s to t in residual graph G_f :

Find **any** augmenting path P using **any search method**

Find the minimum residual capacity c_f along path P

Augment flow f along P by c_f

Update residual graph G_f

Return f

Ford and Fulkerson's Algorithm

Ford-Fulkerson(G, s, t) ; $G = (V, E)$

for each edge (u, v) in E

$$f(u, v) = f(v, u) = 0$$

while **exists** path P from s to t in residual network G_f

do $c_f(p) = \min\{c_f(u, v) : (u, v) \text{ is in } P\}$

for each edge (u, v) on P

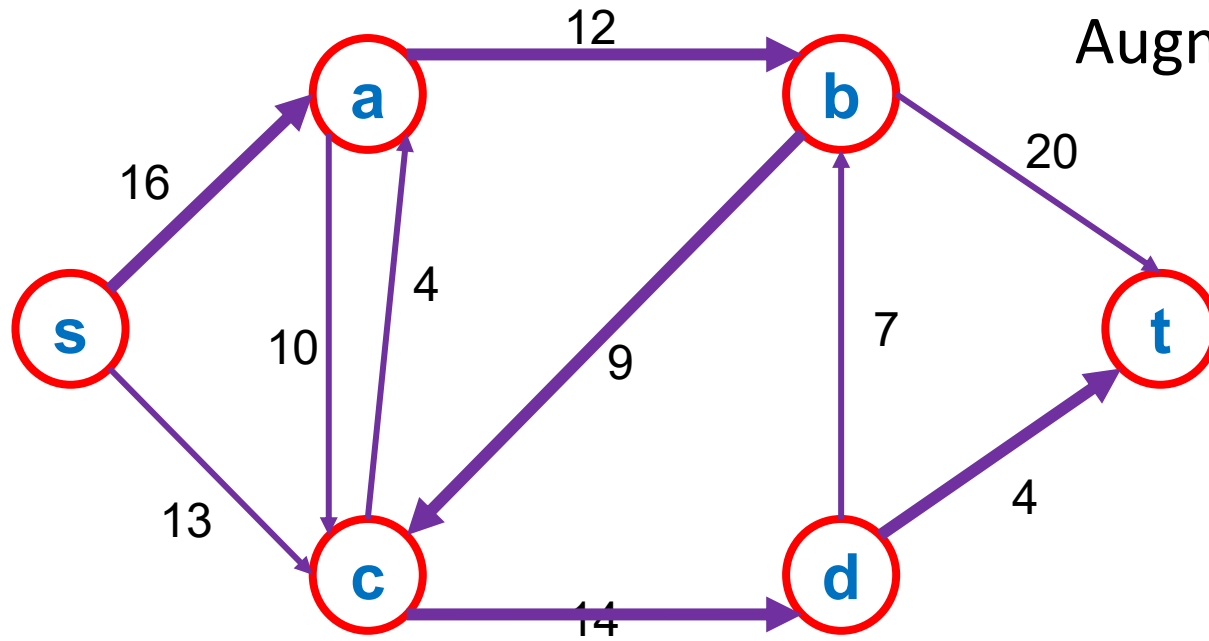
if $(u, v) \in E$

$f(u, v) = f(u, v) + c_f(p)$ (Send flow along the path)

else

$f(v, u) = f(v, u) + c_f(p)$ (The flow might be "returned" later)

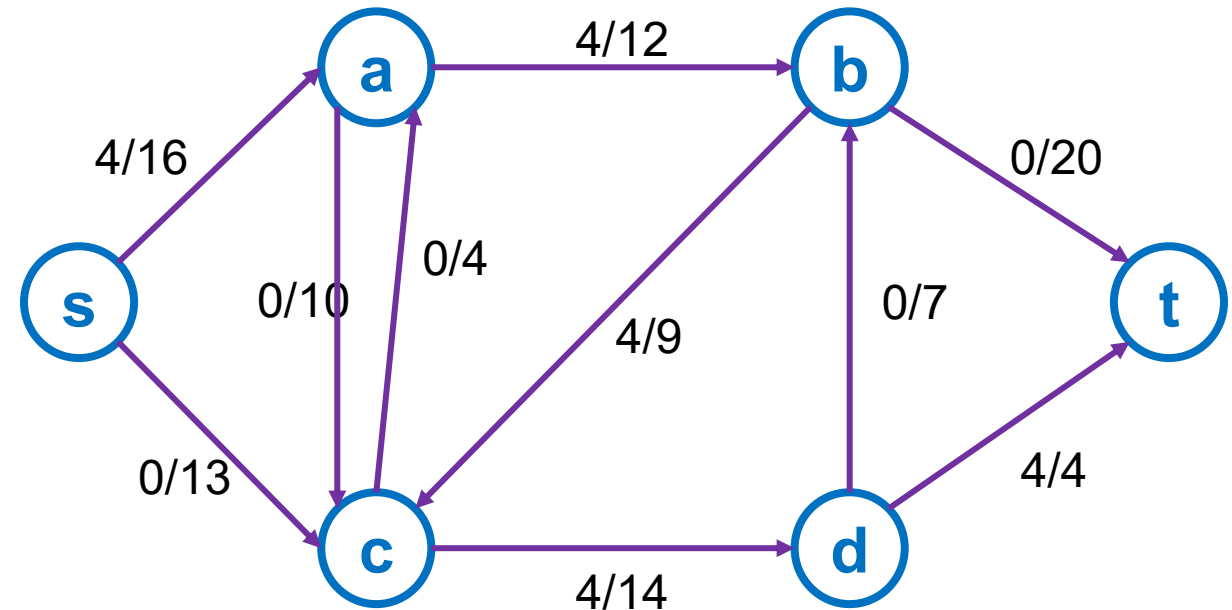
Ford and Fulkerson's Algorithm (1/7)



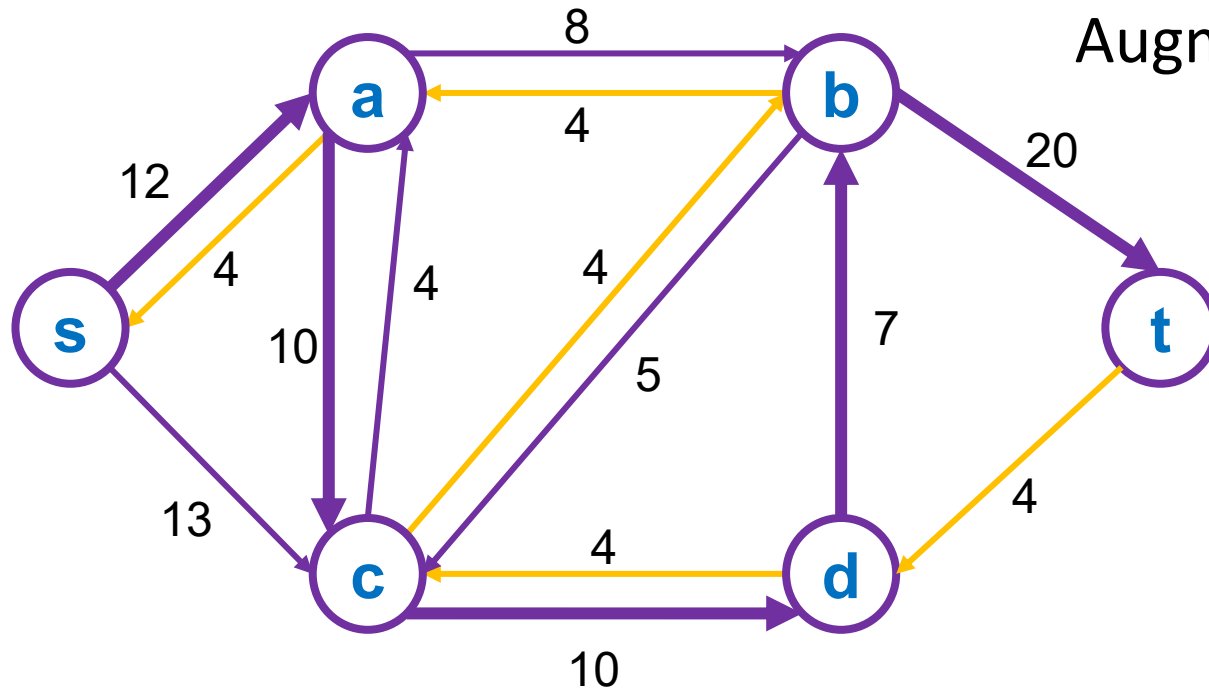
Augmenting Path: $s \rightarrow a \rightarrow b \rightarrow c \rightarrow d \rightarrow t$

$$F = 4$$

$$f'(u \rightarrow v) = \begin{cases} f(u \rightarrow v) + F & \text{if } u \rightarrow v \in P \\ f(u \rightarrow v) - F & \text{if } v \rightarrow u \in P \\ f(u \rightarrow v) & \text{otherwise} \end{cases}$$



Ford and Fulkerson's Algorithm (2/7)

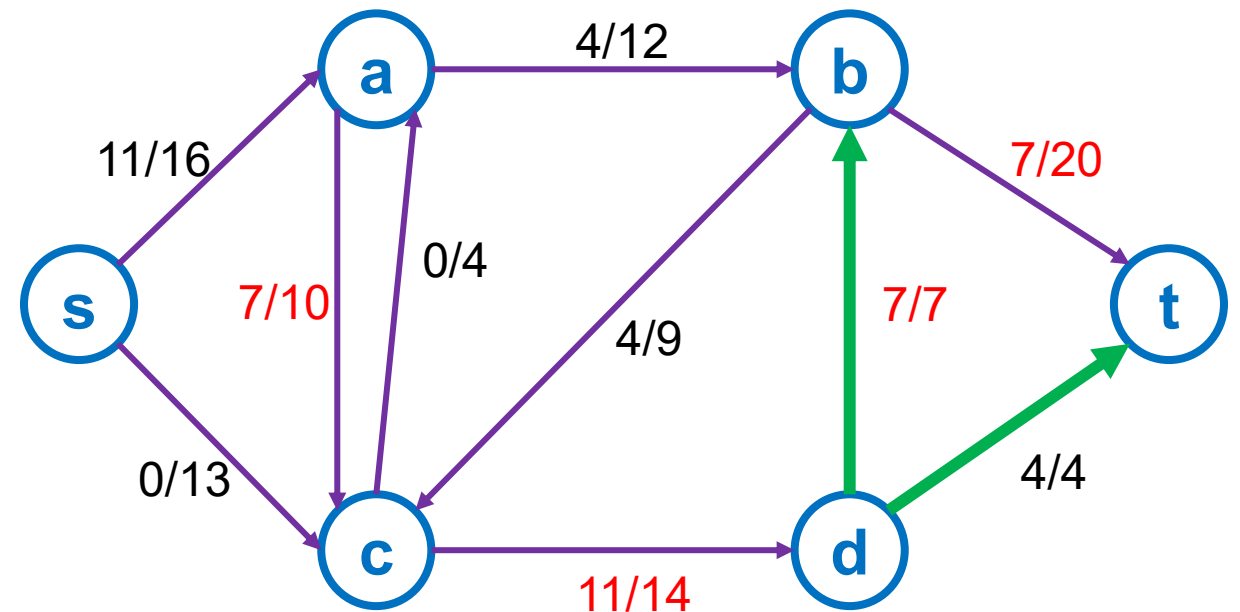


residual graph

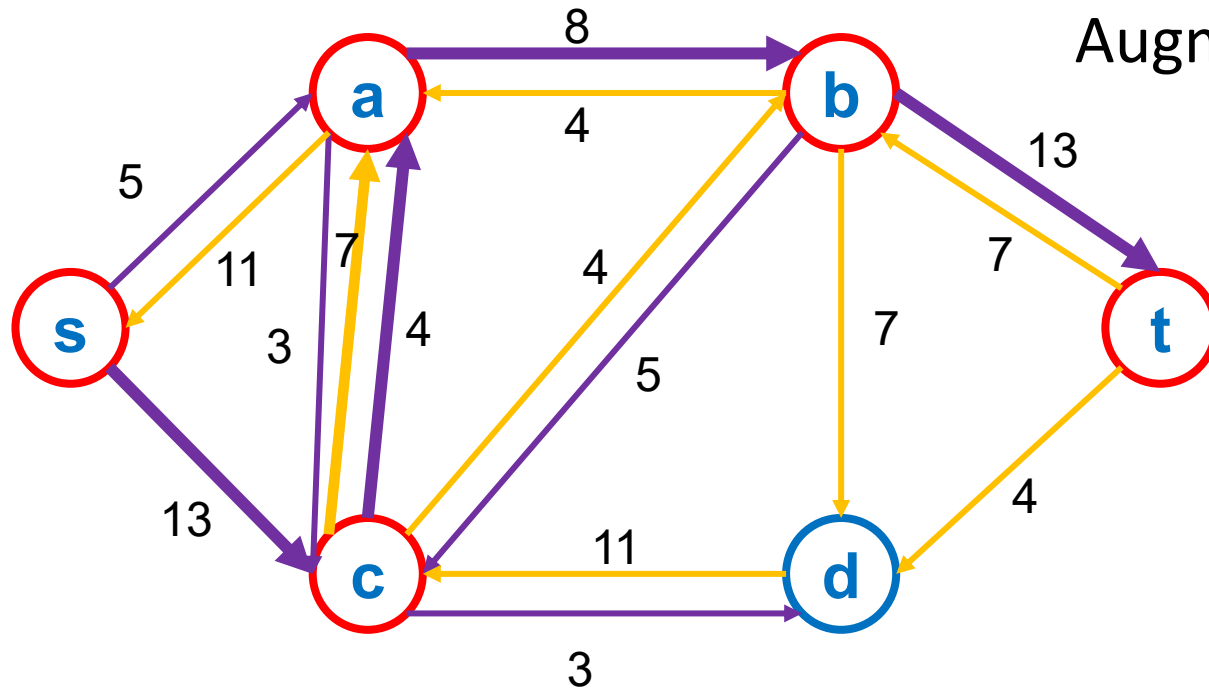
Augmenting Path: $s \rightarrow a \rightarrow c \rightarrow d \rightarrow b \rightarrow t$

$F = 7$

$$f'(u \rightarrow v) = \begin{cases} f(u \rightarrow v) + F & \text{if } u \rightarrow v \in P \\ f(u \rightarrow v) - F & \text{if } v \rightarrow u \in P \\ f(u \rightarrow v) & \text{otherwise} \end{cases}$$



Ford and Fulkerson's Algorithm (3/7)



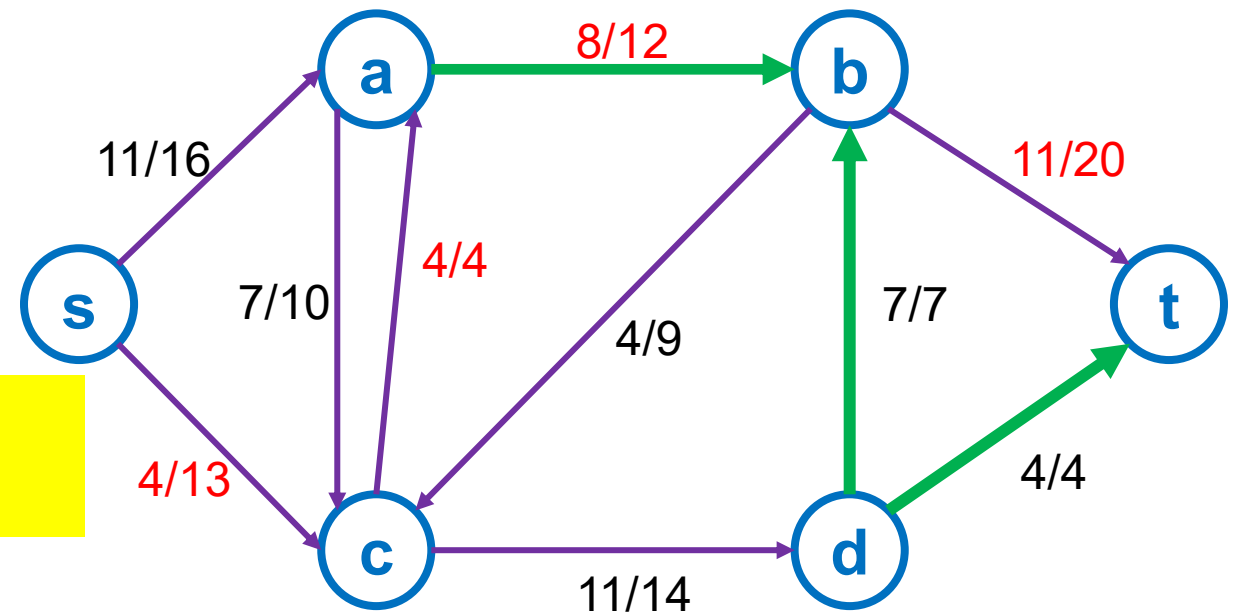
residual graph

Augmenting Path: $s \rightarrow c \rightarrow a \rightarrow b \rightarrow t$

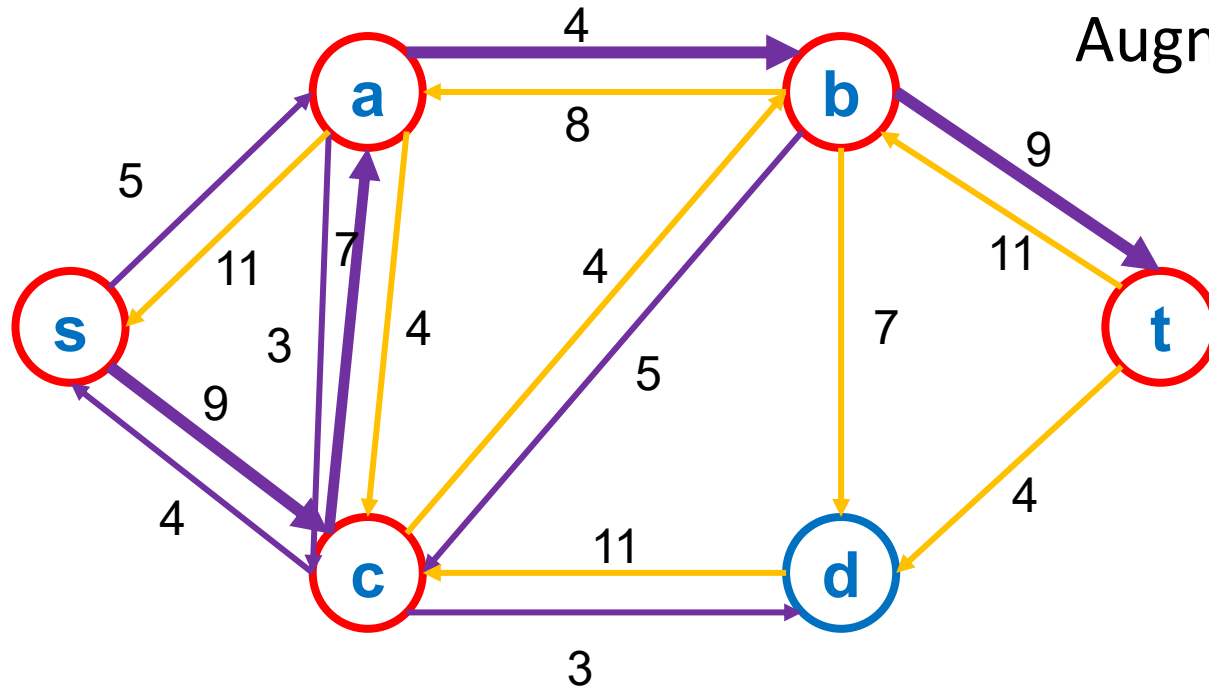
$$F = 4$$

$$f'(u \rightarrow v) = \begin{cases} f(u \rightarrow v) + F & \text{if } u \rightarrow v \in P \\ f(u \rightarrow v) - F & \text{if } v \rightarrow u \in P \\ f(u \rightarrow v) & \text{otherwise} \end{cases}$$

Separate them and do an augmentation step by step



Ford and Fulkerson's Algorithm (4/7)



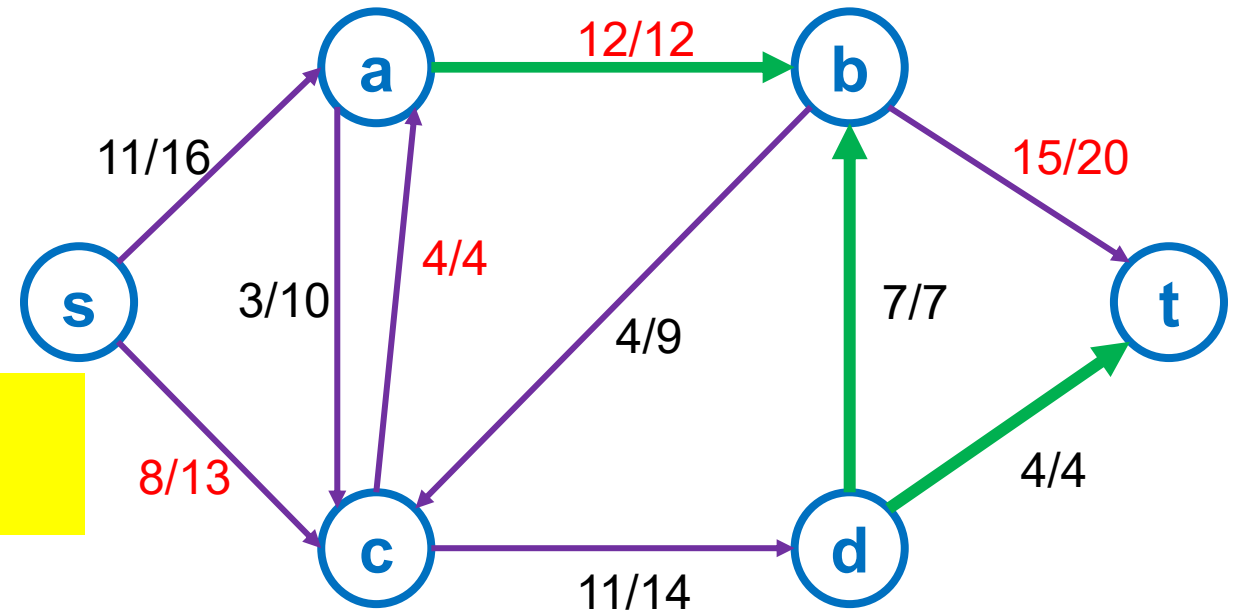
residual graph

Augmenting Path: $s \rightarrow c \rightarrow a \rightarrow b \rightarrow t$

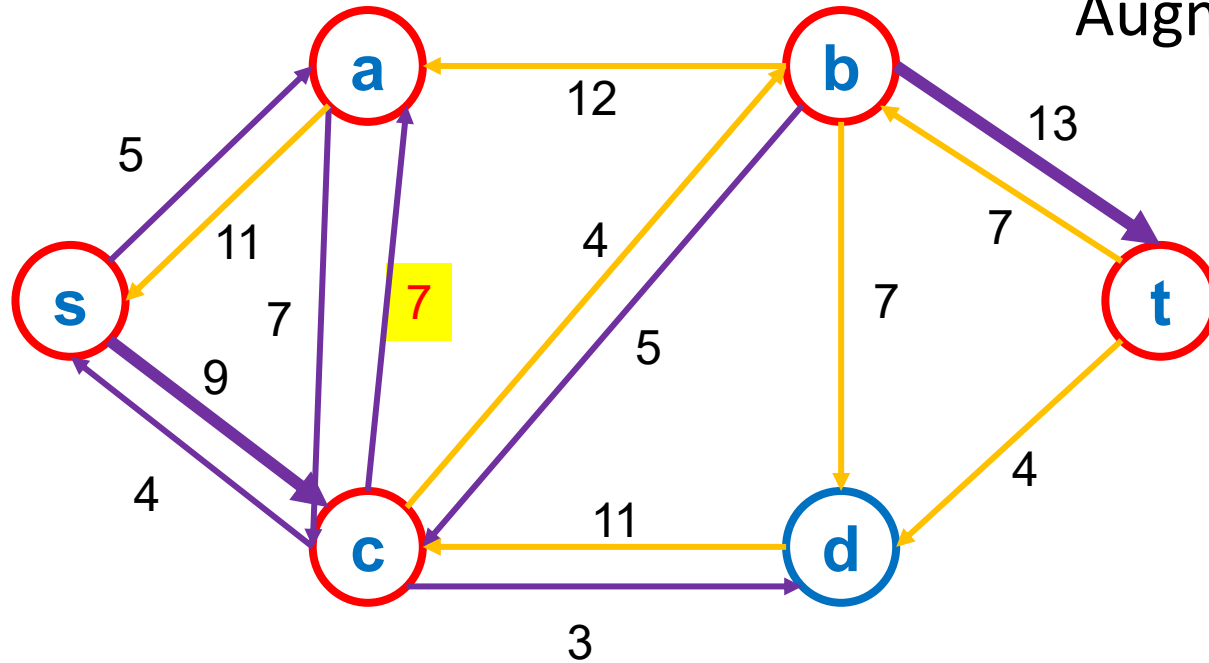
$$F = 4$$

$$f'(u \rightarrow v) = \begin{cases} f(u \rightarrow v) + F & \text{if } u \rightarrow v \in P \\ f(u \rightarrow v) - F & \text{if } v \rightarrow u \in P \\ f(u \rightarrow v) & \text{otherwise} \end{cases}$$

Separate them and do an augmentation step by step



Ford and Fulkerson's Algorithm (4*/7): **ALTERNATIVE** approach



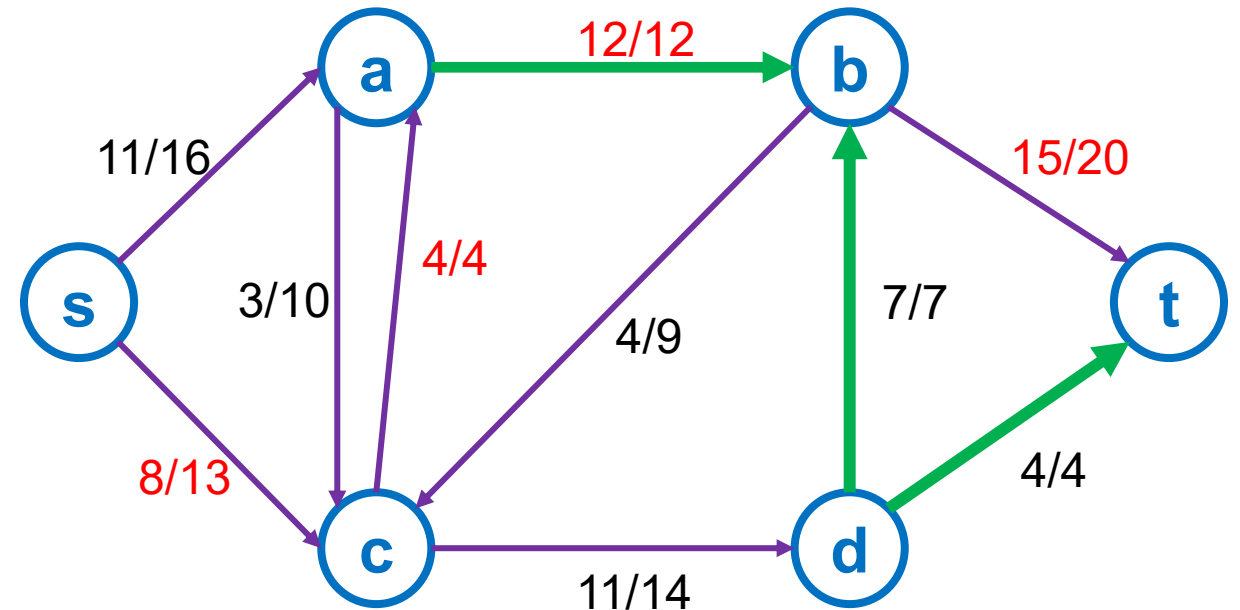
residual graph

Augmenting Path: $s \rightarrow a \rightarrow b \rightarrow c \rightarrow d \rightarrow t$

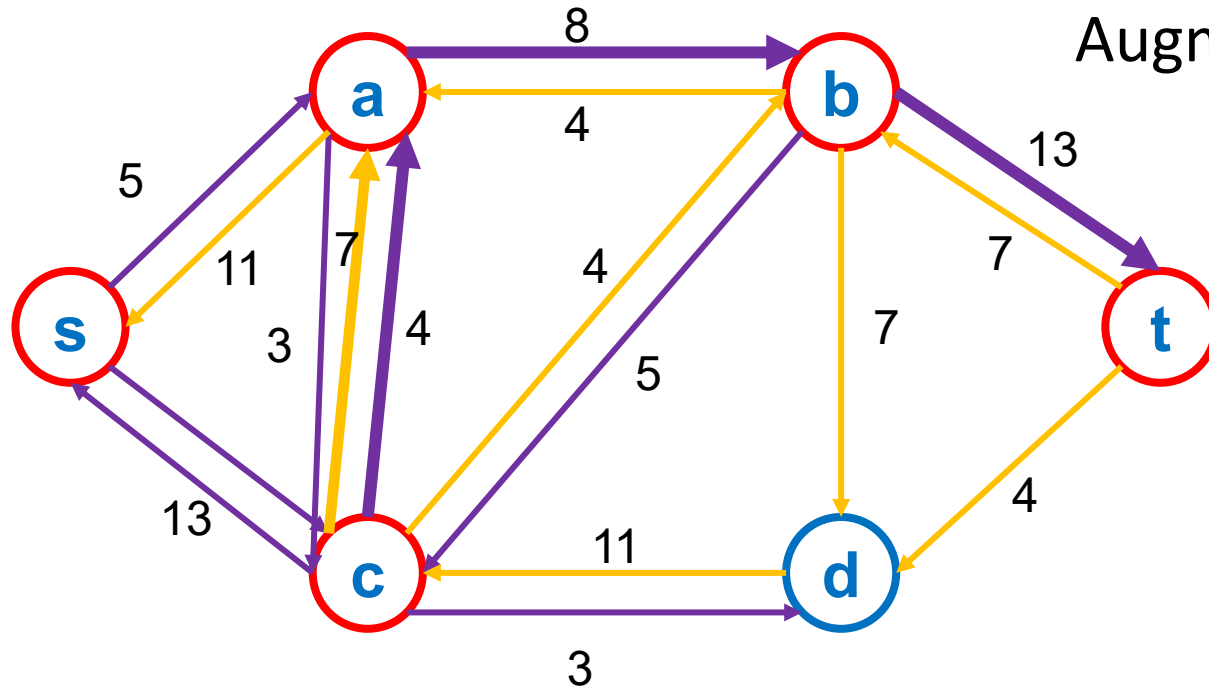
$$F = 8$$

$$f'(u \rightarrow v) = \begin{cases} f(u \rightarrow v) + F & \text{if } u \rightarrow v \in P \\ f(u \rightarrow v) - F & \text{if } v \rightarrow u \in P \\ f(u \rightarrow v) & \text{otherwise} \end{cases}$$

We can also MERGE two flows: one from “flow reclamation” and one from the actual augmenting flow to 11 as in $c \rightarrow a$.



Ford and Fulkerson's Algorithm (4*/7): **ALTERNATIVE** approach

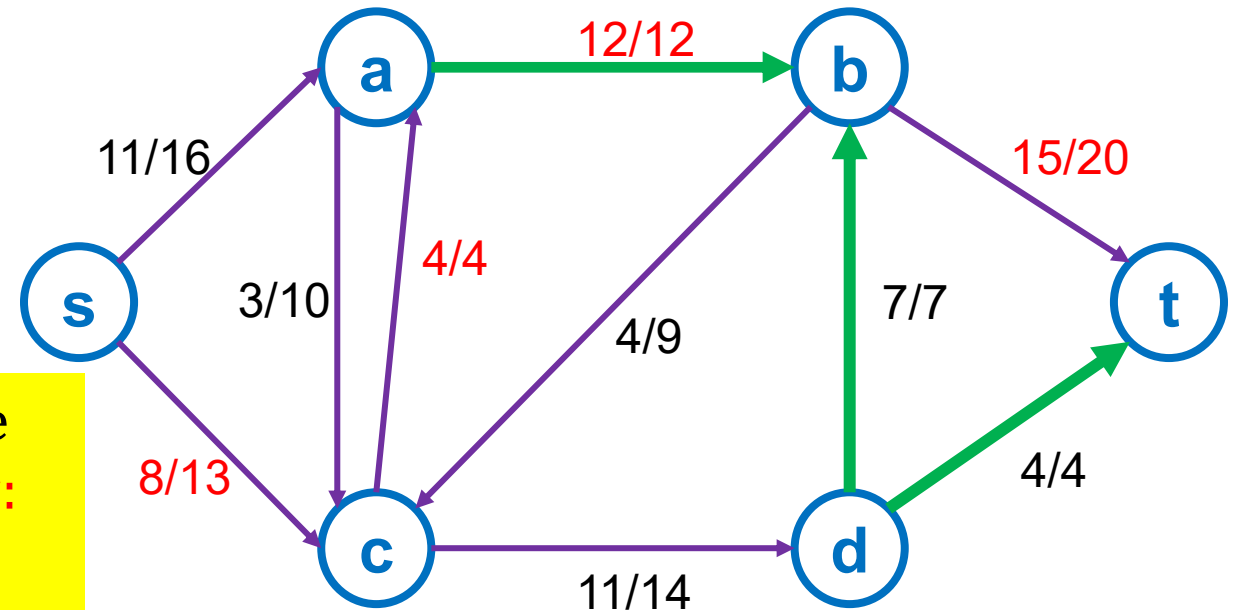


residual graph

Augmenting Path: $s \rightarrow a \rightarrow b \rightarrow c \rightarrow d \rightarrow t$

$$F = 8$$

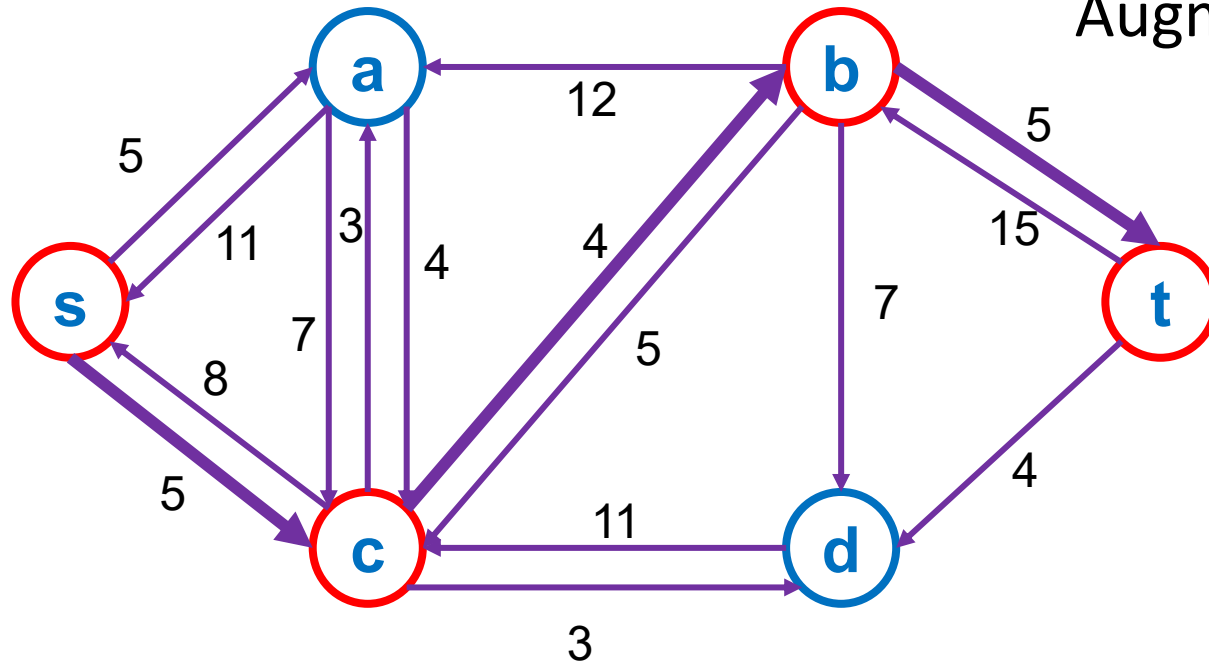
$$f'(u \rightarrow v) = \begin{cases} f(u \rightarrow v) + F & \text{if } u \rightarrow v \in P \\ f(u \rightarrow v) - F & \text{if } v \rightarrow u \in P \\ f(u \rightarrow v) & \text{otherwise} \end{cases}$$



However, when doing augment the path, we have to separate **flow reclamation** and **augmenting flow**:

- $a \rightarrow c$ from the original graph: **augment 4**
- $c \rightarrow a$ from the original graph: **reclaim 4**

Ford and Fulkerson's Algorithm (5/7)



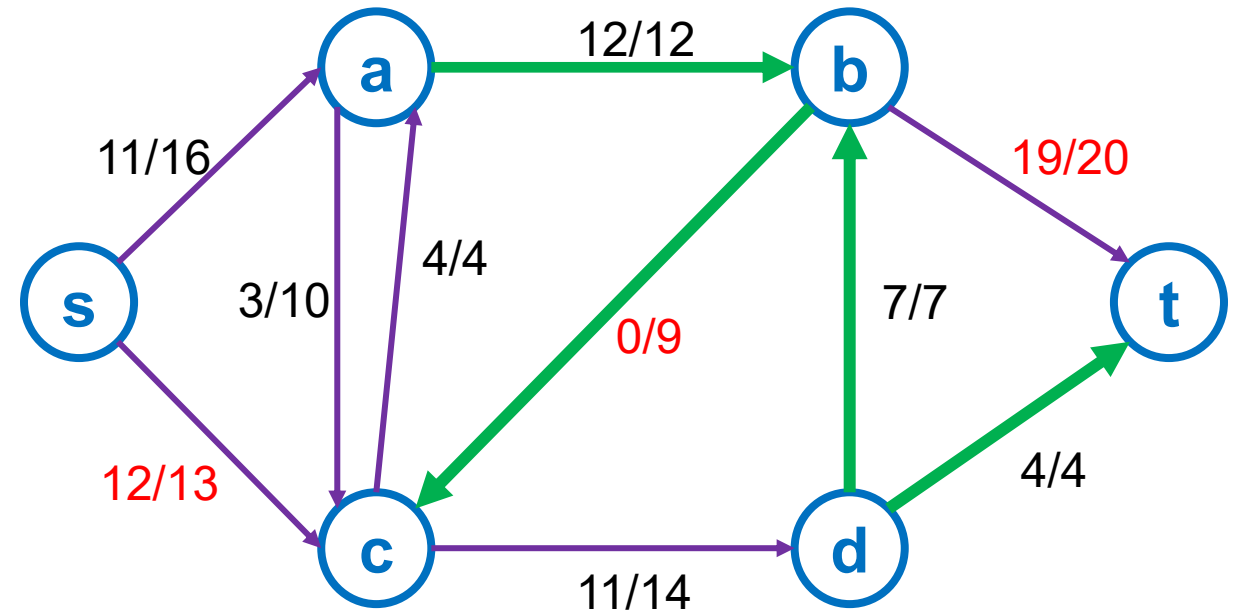
residual graph

Augmenting Path:

$s \rightarrow c \rightarrow b \rightarrow t$

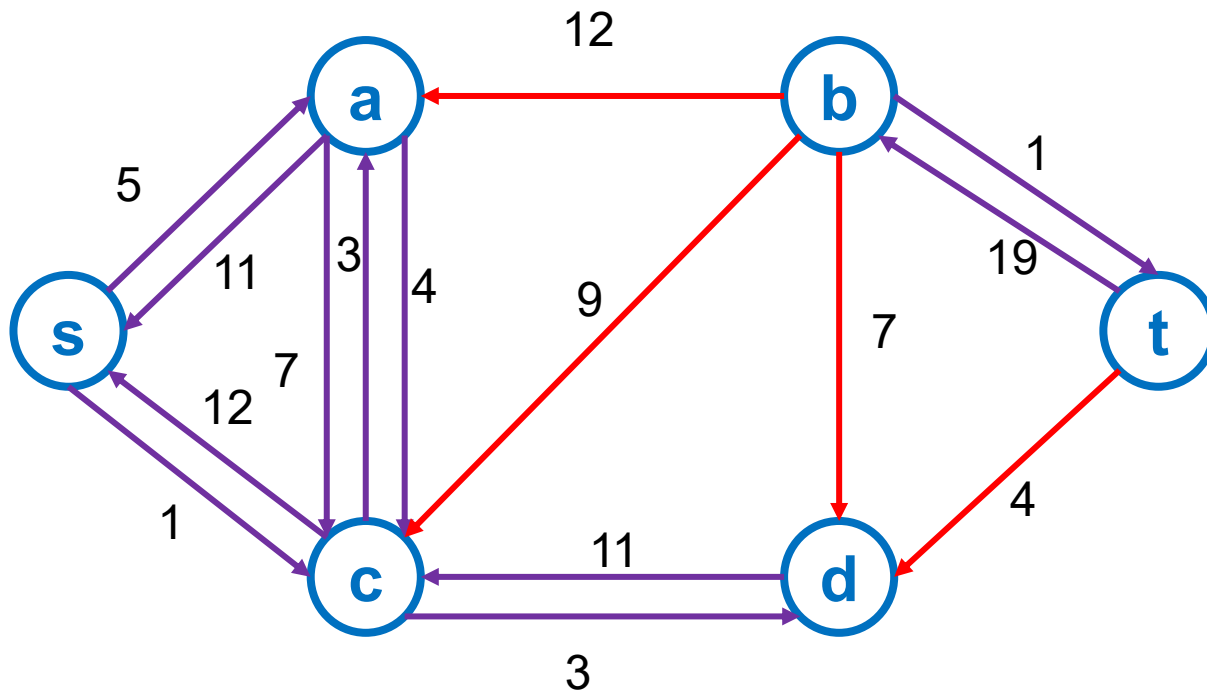
$F = 4$

$$f'(u \rightarrow v) = \begin{cases} f(u \rightarrow v) + F & \text{if } u \rightarrow v \in P \\ f(u \rightarrow v) - F & \text{if } v \rightarrow u \in P \\ f(u \rightarrow v) & \text{otherwise} \end{cases}$$



Ford and Fulkerson's Algorithm (6/7)

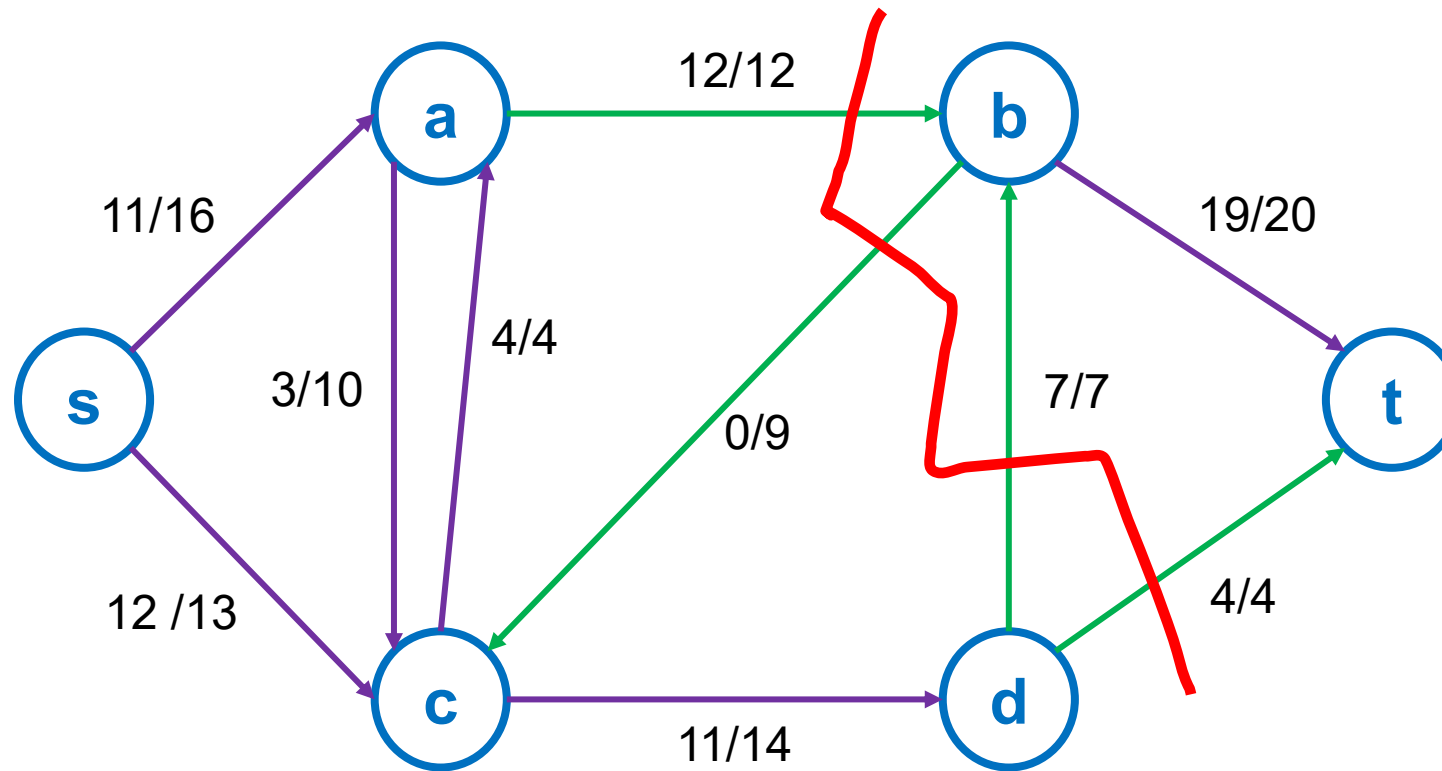
NO more augmenting paths. STOP



residual graph

Ford and Fulkerson's Algorithm (7/7)

Max flow = 12 + 7 + 4 = 23

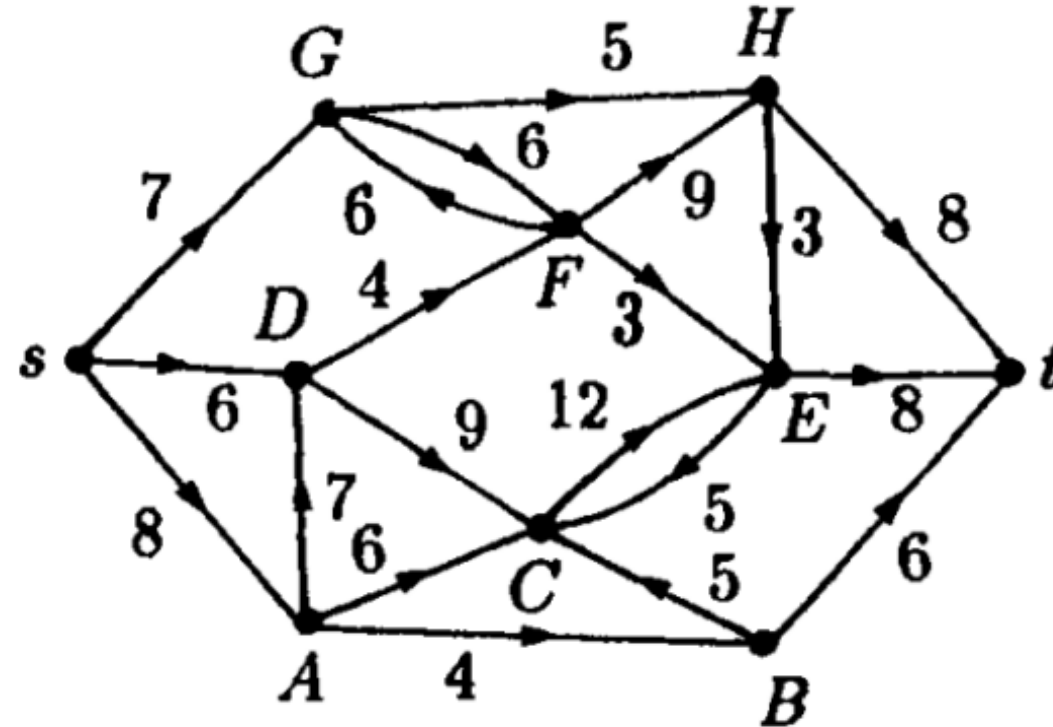


Complexity Analysis

- If every edge capacity is an integer, then Ford and Fulkerson's algorithm halts after at most $|f^*|$ iterations, where f^* is the actual maximum flow.
- In each iteration, building residual graph G_f + perform whatever-first-search cost $O(E)$ time.
- Thus, the algorithm runs in $O(E|f^*|)$ times in the worst case.
 - The running time of Ford-Fulkerson is **actually exponential** in the input size.

Exercise

- Find a maximum flow path for the following graph

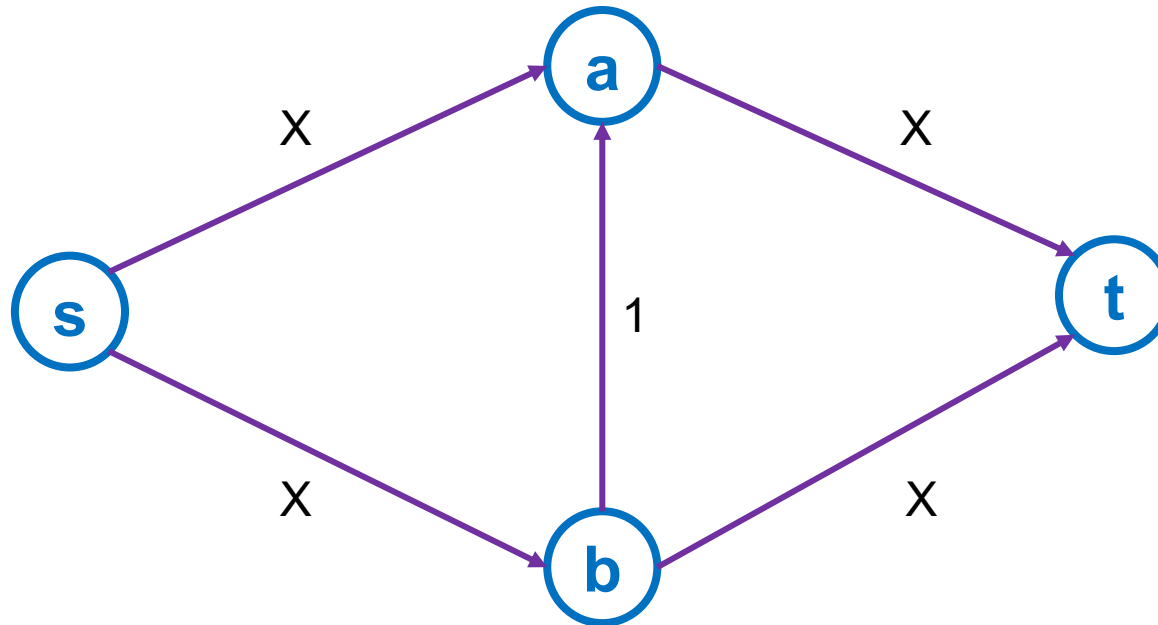


Outline

1. Flows
2. Cuts
3. The Maxflow-Mincut Theorem
4. Ford and Fulkerson's algorithm
5. Edmonds and Karp's Algorithms

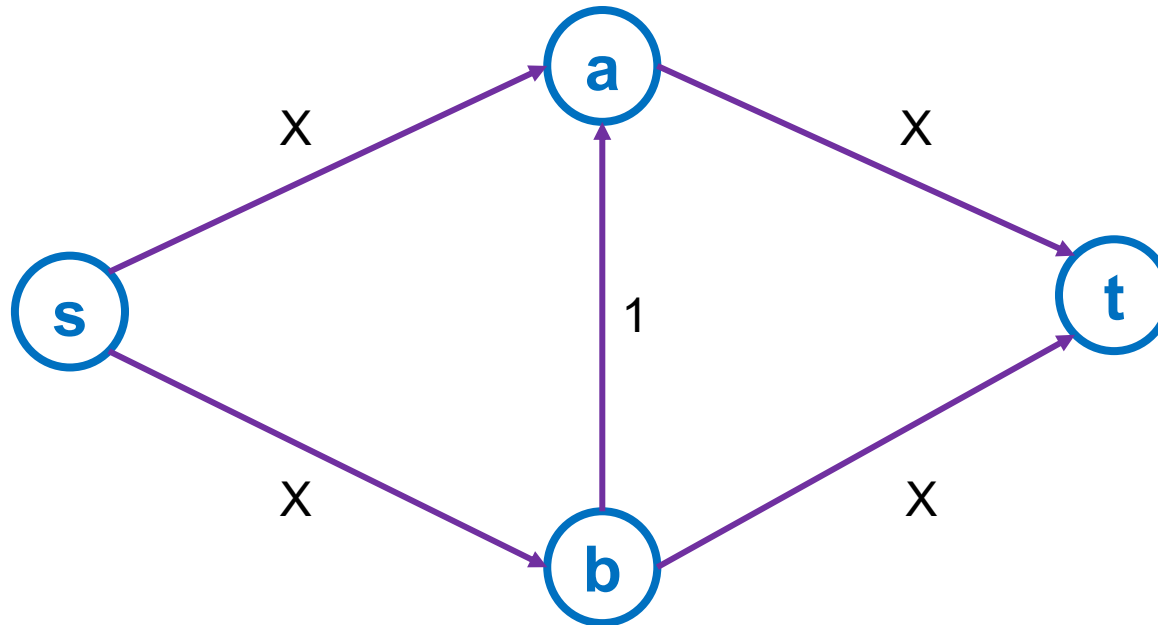
Bad examples for Ford and Fulkerson's Algorithm (1/4)

Consider the 4-node network, where X is some large integer:



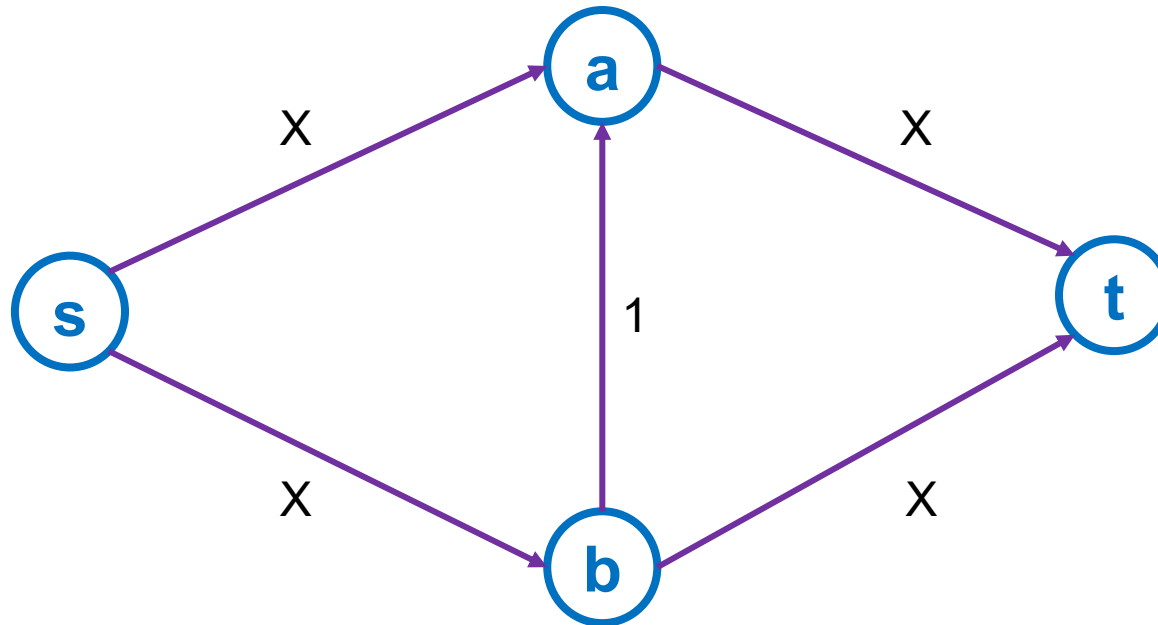
Bad examples for Ford and Fulkerson's Algorithm (2/4)

Ford-Fulkerson might alternate between pushing one unit of flow along $s \rightarrow b \rightarrow a \rightarrow t$ then pushing one unit of flow along $s \rightarrow a \rightarrow b \rightarrow t$



Bad examples for Ford and Fulkerson's Algorithm (3/4)

Ford-Fulkerson might alternate between pushing one unit of flow along $s \rightarrow b \rightarrow a \rightarrow t$ then pushing one unit of flow along $s \rightarrow a \rightarrow b \rightarrow t$

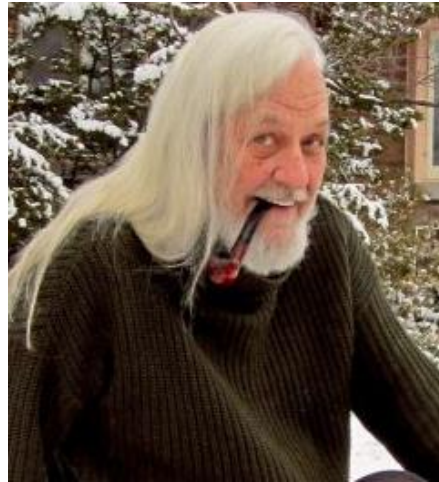


Thus this leads to a running time of $\Theta(X) = \Omega(|f^*|)$.

Bad examples for Ford and Fulkerson's Algorithm (4/4)

- Ford and Fulkerson's algorithm does not specify which path in the residual graph to augment.
- The poor worst-case behavior of the algorithm can be blamed on poor choices for the augmenting path.

History



Jack R. Edmonds
(1935 –)



Richard Manning Karp
(1935 –)



Yefim Dinitz
(? -)

- Edmonds–Karp algorithm is an implementation of the Ford–Fulkerson method for computing the maximum flow **efficiently** in a flow network in $O(VE^2)$ time.
- The algorithm was first published by Yefim Dinitz in 1970 [1][2]. and independently published by Jack Edmonds and Richard Karp in 1972. [3]

[1] Dinic, E. A. (1970). "Algorithm for solution of a problem of maximum flow in a network with power estimation". Soviet Mathematics - Doklady. 11. Doklady: 1277–1280.
[2] Yefim Dinitz (2006). "Dinitz' Algorithm: The Original Version and Even's Version". In Oded Goldreich; Arnold L. Rosenberg; Alan L. Selman (eds.). Theoretical Computer Science: Essays in Memory of Shimon Even. Springer.
[3] Edmonds, Jack; Karp, Richard M. (1972). "Theoretical improvements in algorithmic efficiency for network flow problems". Journal of the ACM.

Edmonds and Karp's two rules

First rule (**Fattest** Augmenting Paths):

Choose the augmenting path with largest bottleneck value.

Second rule (**Shortest** Augmenting Paths):

Choose the augmenting path with the smallest number of edges.

Edmonds and Karp's Algorithms

Algorithm **EdmondsKarp**(Graph G , Source s , Target t):

Initialize flow $f(u, v) = 0$ for all edges (u, v) in G

While there exists an augmenting path P from s to t in residual graph G_f :

Find the shortest augmenting path p using BFS

Find the minimum residual capacity c_f along path P

Augment flow f along P by c_f

Update residual graph G_f

Return f

Fattest Augmenting Paths

First rule (**Fattest** Augmenting Paths):

Choose the augmenting path with largest bottleneck value.

It's not hard to show that the maximum-bottleneck (s, t) -path in a directed graph can be computed in $O(E \log V)$ time using a “best-first” traversal, similar to Jarník's minimum-spanning-tree algorithm or Dijkstra's shortest-path algorithm.

Fattest Augmenting Paths

- Let f be any flow in G and f' be the maximum flow in the current residual graph G_f .
- We proved f' can be decomposed into at most E paths and cycles.
- By averaging argument, there is at least one of paths in this decomposition must carry at least $\frac{|f'|}{E}$ units of flow. It follows that fattest (s, t) -path in G_f carries at least $\frac{|f'|}{E}$ units of flow.

Fattest Augmenting Paths

Thus, residual maximum flow value decays exponentially with the number of iterations. After $E \cdot \ln|f^*|$ iterations, the maximum flow value in G_f is at most:

$$|f^*| \cdot \left(1 - \frac{1}{E}\right)^{E \cdot \ln|f^*|} < |f^*| e^{-\ln|f^*|} = 1.$$

We conclude that the Edmonds-Karp “fattest path” algorithm runs in $O(E^2 \log E \log|f^*|)$.

Shortest Augmenting Paths

Second rule (**Shortest** Augmenting Paths):

Choose the augmenting path with the smallest number of edges.

Shortest Augmenting Paths

Edmonds and Karp's second rule:

Choose the augmenting path with the smallest number of edges.

⇒ The shortest augmenting path can be found in $O(E)$ time by running BFS in residual graph. Resulting algorithm halts after a polynomial number of iterations (independent of actual edge capacities!!!)

Proof of Shortest Augmenting Paths

Proof.

- Let f_i = current flow after i augmentation steps
- Let G_i = corresponding residual graph
- For each vertex v , let $level_i(v)$ denote the unweighted shortest-path distance from s to v in G_i
 - If there is no path from s to v in G_i then $level_i(v) = \infty$.

Shortest Augmenting Paths

The polynomial upper bound relies on two observations:

- The first observation: Level of a vertex can only increase over time

Lemma 2. $\text{level}_i(v) \geq \text{level}_{i-1}(v)$ for all vertices v and all integers $i > 0$.

Shortest Augmenting Paths

The polynomial upper bound relies on two observations:

- The first observation: Level of a vertex can only increase over time

Lemma 2. $\text{level}_i(v) \geq \text{level}_{i-1}(v)$ for all vertices v and all integers $i > 0$.

- The second observation: an edge cannot appear or disappear too many times.

Lemma 3. During the execution of the Edmonds-Karp shortest-augmenting-path algorithm, each edge $u \rightarrow v$ disappears from the residual graph G_f at most $V/2$ times.

Proof on Lemma 2

Lemma 2. $\text{level}_i(v) \geq \text{level}_{i-1}(v)$ for all vertices v and all integers $i > 0$.

Proof. Fix an arbitrary positive integer $i > 0$ and an arbitrary vertex v .

We prove by induction on $\text{level}_i(v)$.

Assume for all vertex u such that $\text{level}_i(u) < \text{level}_i(v)$,

$$\text{level}_i(u) > \text{level}_{i-1}(u).$$

Proof on Lemma 2

- If $v = s$, then $level_i(s) = level_{i-1}(s) = 0$.
- If there is no path from s to v , then $level_i(v) = \infty \geq level_{i-1}(v)$.
- Otherwise, let $s \rightarrow \dots \rightarrow u \rightarrow v$ = shortest path from s to v , then we have $level_i(v) = level_i(u) + 1$. So, the inductive hypothesis implies that

$$level_i(u) \geq level_{i-1}(u).$$

We need to show that $level_{i-1}(u) \geq level_i(v) - 1$.

Proof on Lemma 2

Consider two subcases:

- 1) If $u \rightarrow v$ is an edge in G_{i-1} , then $level_{i-1}(v) \leq level_i(u) + 1$ (by BFS)
- 2) If $u \rightarrow v$ is not an edge in G_{i-1} , then its reversal $v \rightarrow u$ must be an edge in i -th augmenting path which by definition is the shortest path from s to t in G_{i-1} . It follow that $level_{i-1}(v) = level_{i-1}(u) - 1 \leq level_{i-1}(u) + 1$

From 1) and 2), we conclude that

$$level_i(v) = level_i(u) + 1 \geq level_{i-1}(u) + 1 \geq level_{i-1}(v)$$

Proof on Lemma 3

Lemma 3. During the execution of the Edmonds-Karp shortest-augmenting-path algorithm, each edge $u \rightarrow v$ disappears from the residual graph G_f at most $V/2$ times.

Proof. Suppose $u \rightarrow v$ is in two residual graphs G_i and G_{j+1} , but not in any of the intermediate residual graphs $G_{i+1}, \dots, G_j$, for some $i < j$. Then, $u \rightarrow v$ must be in the i -th augmenting path, so $level_i(v) = level_i(u) + 1$, and $v \rightarrow u$ must be on the j -th augmenting path, so $level_j(v) = level_j(u) - 1$.

Proof on Lemma 3

By **Lemma 2**, it follow that

$$level_j(u) = level_j(v) + 1 \geq level_i(v) + 1 = level_i(u) + \mathbf{2}.$$

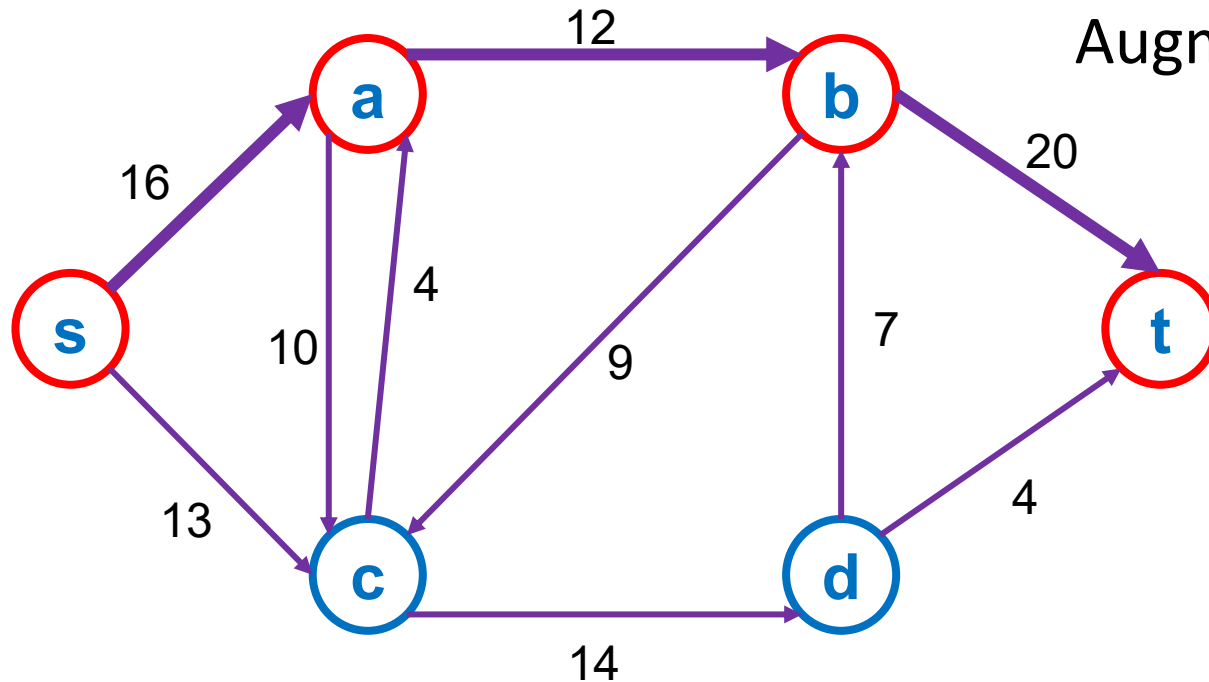
Because every level is either less than V or infinite, the number of disappearances is at most $\frac{V}{2}$.

Complexity of Edmonds and Karp's Algorithms

- Each edge disappears at most $V/2$ times, there are at most $EV/2$ edge disappearances overall.
- But at least one edge disappears on each iteration, so the algorithm must halt after at most $EV/2$ iterations.
- Thus, each iteration requires $O(E)$ time

So the overall algorithm runs in **$O(VE^2)$ time.**

Edmonds and Karp's Algorithms (1/4)

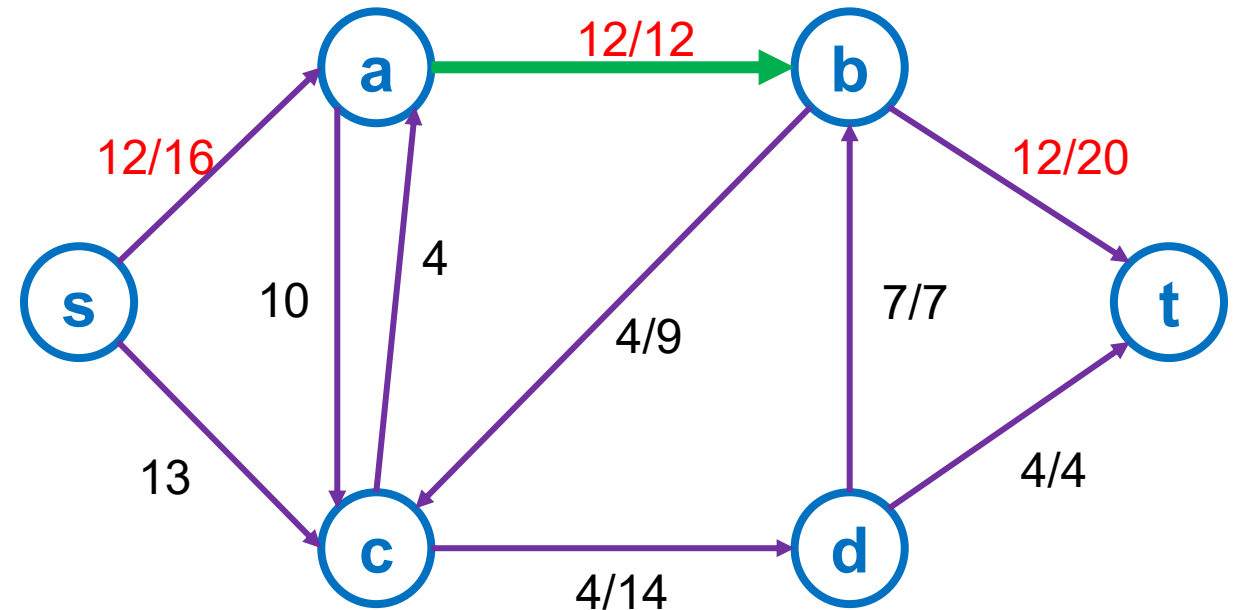


Augmenting Path:

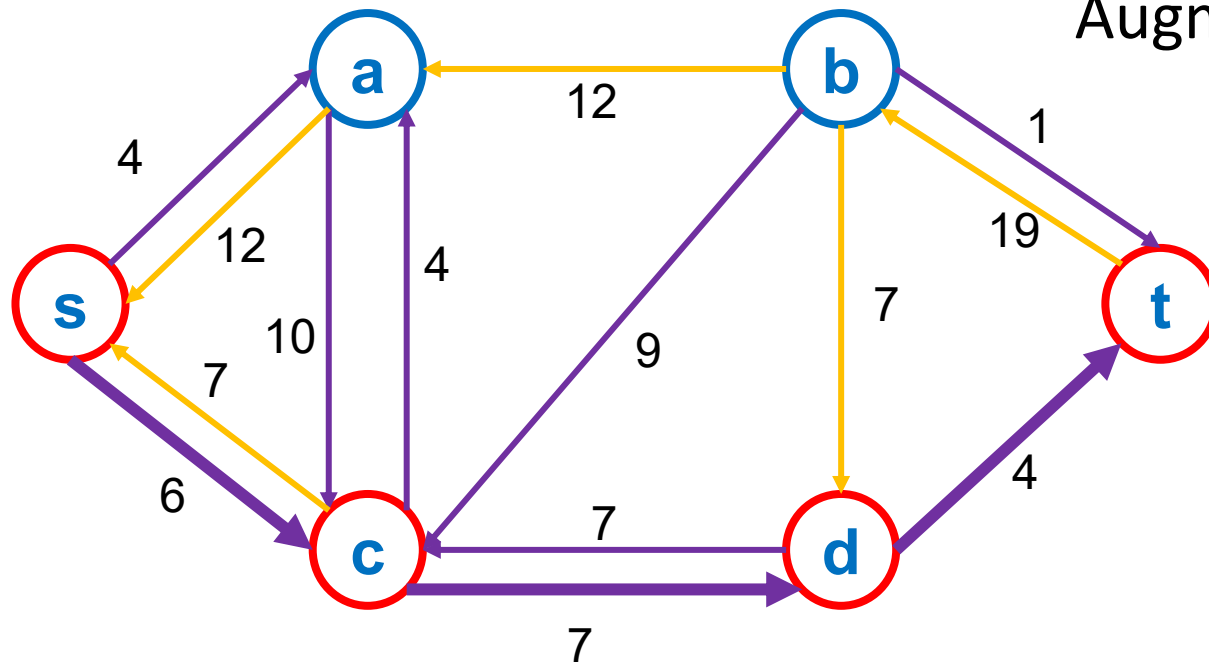
$s \rightarrow a \rightarrow b \rightarrow t$

$F = 4$

$$f'(u \rightarrow v) = \begin{cases} f(u \rightarrow v) + F & \text{if } u \rightarrow v \in P \\ f(u \rightarrow v) - F & \text{if } v \rightarrow u \in P \\ f(u \rightarrow v) & \text{otherwise} \end{cases}$$



Edmonds and Karp's Algorithms (2/4)

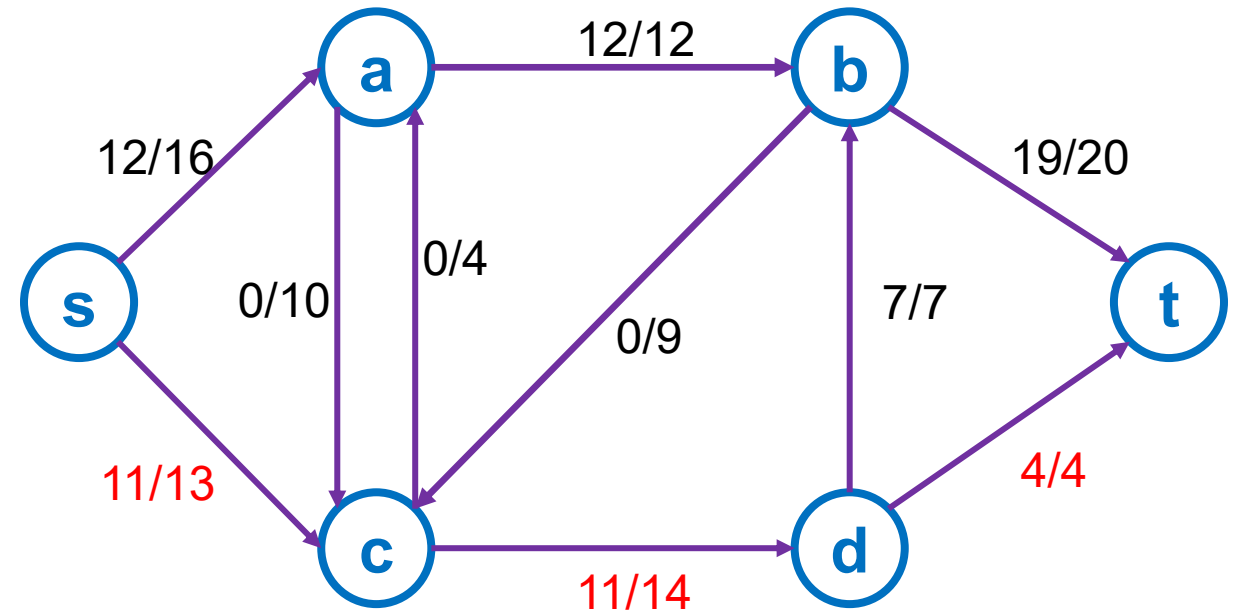


Augmenting Path:

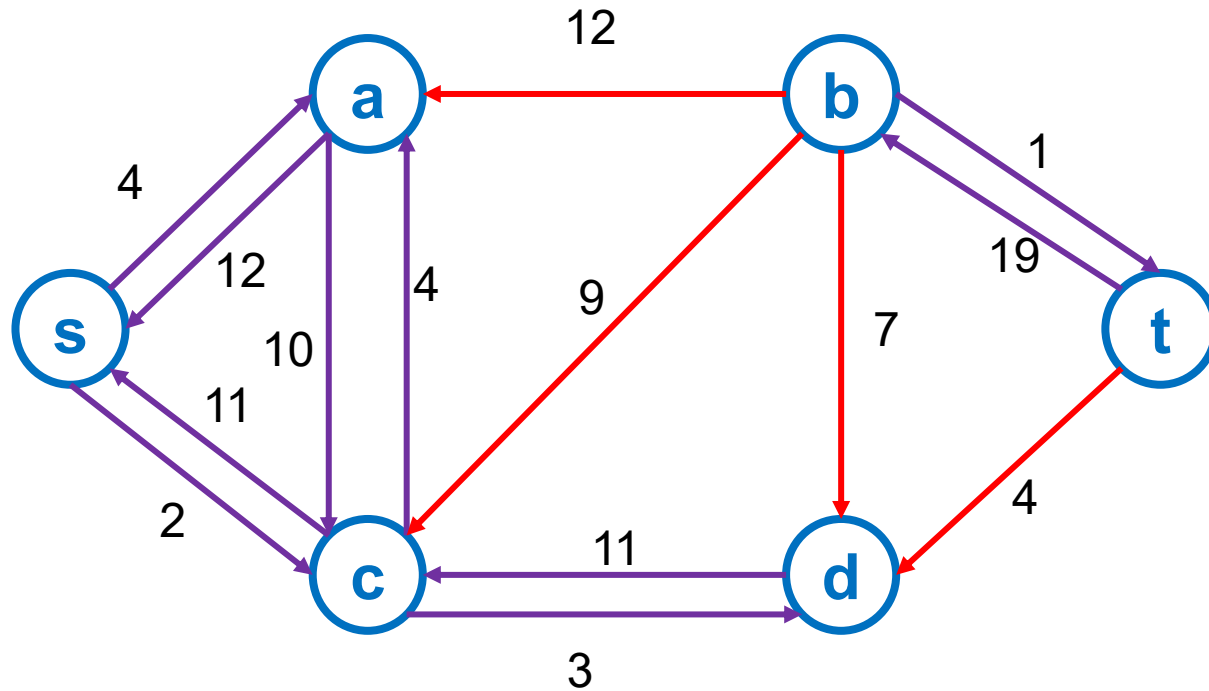
$s \rightarrow c \rightarrow d \rightarrow t$

$F = 4$

$$f'(u \rightarrow v) = \begin{cases} f(u \rightarrow v) + F & \text{if } u \rightarrow v \in P \\ f(u \rightarrow v) - F & \text{if } v \rightarrow u \in P \\ f(u \rightarrow v) & \text{otherwise} \end{cases}$$



Edmonds and Karp's Algorithms (3/4)

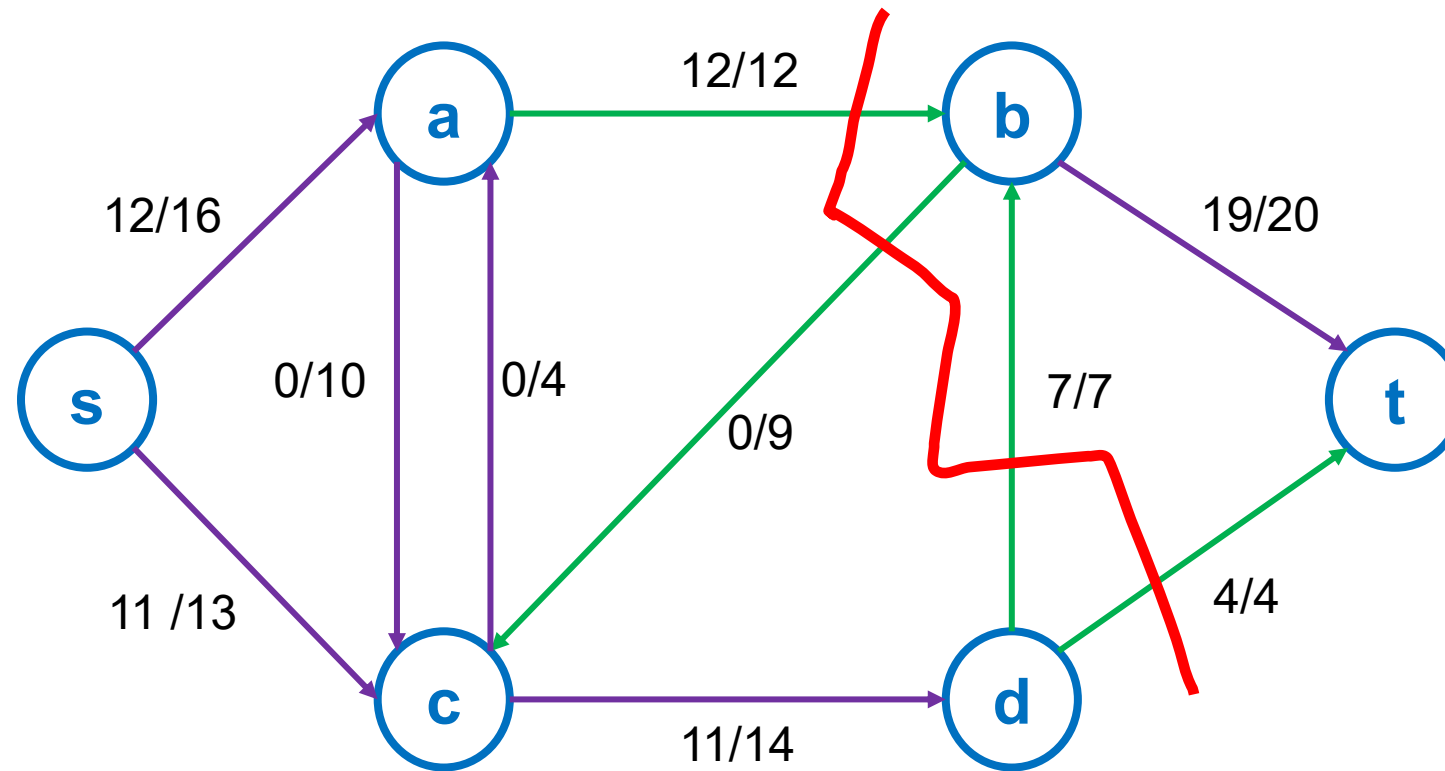


residual graph

NO more augmenting paths. STOP

Edmonds and Karp's Algorithms (4/4)

Max flow = 12 + 7 + 4 = 23



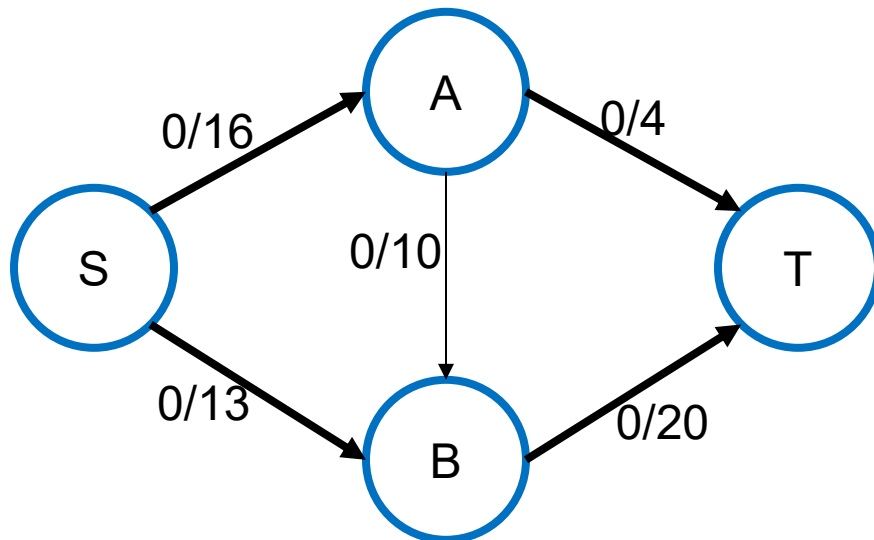
Further progress

Technique	Direct	With dynamic trees	Source
Blocking flow	$O(V^2E)$	$O(VE \log V)$	[Dinitz; Karzanov; Even and Itai; Sleator and Tarjan]
Network simplex	$O(V^2E)$	$O(VE \log V)$	[Dantzig; Goldfarb and Hao; Goldberg, Grigoriadis, and Tarjan]
Push-relabel (generic)	$O(V^2E)$	-	[Goldberg and Tarjan]
Push-relabel (FIFO)	$O(V^3)$	$O\left(VE \log \left(\frac{V^2}{E}\right)\right)$	[Goldberg and Tarjan]
Push-relabel (highest label)	$O(V^2\sqrt{E})$	-	[Cheriyān and Maheshwari; Tunçel]

Further progress

Technique	Direct	With dynamic trees	Source
Push-relabel-add games	-	$O\left(VE \log \frac{E}{V \log V} V\right)$	[Cheriyán and Hagerup; King, Rao, and Tarjan]
Pseudoflow	$O(V^2 E)$	$O(VE \log V)$	[Hochbaum]
Pseudoflow (highest label)	$O(V^3)$	$O\left(VE \log \left(\frac{V^2}{E}\right)\right)$	[Hochbaum and Orlin]
Incremental BFS	$O(V^2 E)$	$O\left(VE \log \left(\frac{V^2}{E}\right)\right)$	[Goldberg, Held, Kaplan, Tarjan, and Werneck]
Compact Network	-	$O(VE)$	[Orlin]

Linear Programming: Max Flow



Variables: SA, SB, AB, AT, BT (amount of flow through the edge)

Constraints:

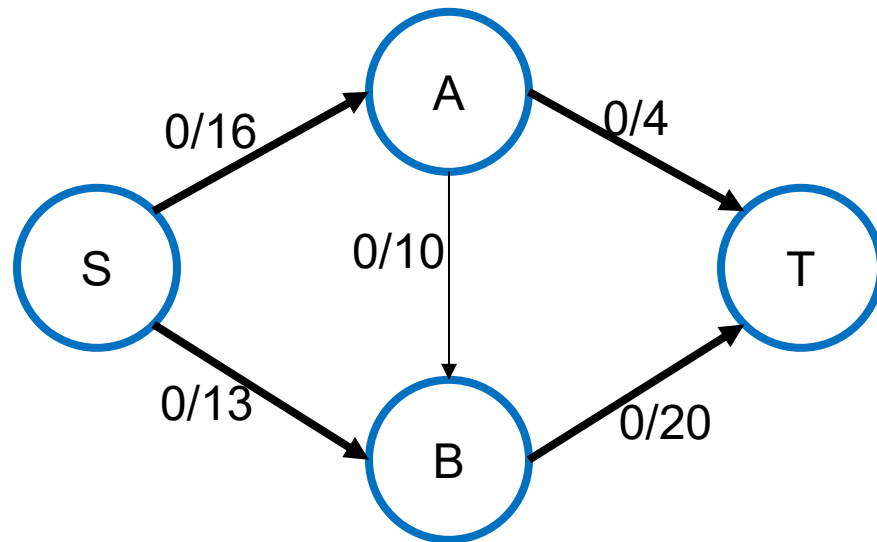
- Capacity

- flow conservation

Objective:

$$\max(SA + SB + AB + AT + BT)$$

Linear Programming: Min Cut



Variables: SA, SB, AB, AT, BT (not cut = 0, cut = 1)

$AinS, BinS$ (not in $S = 0$, in $S = 1$)

Constraints:

- Capacity

- $0 \leq SA, SB, AB, AT, BT$
- $1 \leq SA + AinS$ (A in S or SA is cut)
- $1 \leq SB + BinS$ (B in S or SB is cut)
- $0 \leq AB - AinS + BinS$

(A in S but not B means AB is cut)

- $0 \leq AT - AinS$ (A in S means AT is cut)
- $0 \leq BT - BinS$ (B in S means BT is cut)

Objective:

$$\min (16 SA + 4 AT + 10 AB + 13 SB + 20 BT)$$

Q & A